Supplementary Information

A flexible modelling framework for estimating thermal tolerance and sensitivity

Daniel W.A. Noble*[1], Pieter A. Arnold[1], Shinichi Nakagawa[2,4], Patrice Pottier†[3,4]

[1] *Division of Ecology and Evolution, Research School of Biology, Australian National University, Canberra, Australia*
[2] *Department of Biological Sciences, University of Alberta, Edmonton, Canada*
[3] *Department of Biological and Environmental Sciences, University of Gothenburg, Gothenburg, Sweden*
[4] *Evolution & Ecology Research Centre, School of Biological, Earth and Environmental Sciences, University of New South Wales, Sydney, New South Wales, Australia*

2026-10-11

Table of contents

1 General Introduction **3**

2 Introduction to `bayesTLS` R Package **3**
2.1 *Description of Functions* . . . 5

3 A Tutorial with Simulations **6**
3.0.1 *Simulating Data* . . . 6
3.0.2 *The classical two-stage TDT pipeline* . . . 11
3.0.3 *A joint Bayesian 4PL* . . . 15
3.0.4 *Deriving z, $CT_{max_{1hr}}$, and T_{crit} from the posterior* . . . 21
3.0.5 *Comparing the two pipelines* . . . 28
3.0.6 *A worked example: temperature effects on every shape parameter* . . . 32
3.0.7 Heat injury accumulation and survival under a fluctuating trace . . . 43
3.0.8 Validation: recovering known doses . . . 46

4 Empirical model specifications used in the manuscript **47**

5 Case Study 1: Zebrafish heat tolerance across an oxygen gradient **49**
5.1 Data and standardisation . . . 49
5.2 Joint 4PL with oxygen on CT_{max} and z . . . 50
5.3 Per-treatment thermal limits and OCLTT check . . . 51

*Corresponding author: `daniel.noble@anu.edu.au`
†Corresponding author: `patrice.pottier@bioenv.gu.se`

6 Case Study 2: Cereal aphids — comparing three species **53**
6.1 Data and standardisation . . . 54
6.2 Joint 4PL with species on CT_{max} and z . . . 54
6.3 Per-species thermal limits . . . 56
6.4 Heat injury across species under a field temperature trace . . . 62

7 Case Study 3: Snow gum leaf PSII — light vs dark recovery **63**
7.1 Data and standardisation . . . 63
7.2 Joint 4PL with recovery on CT_{max} and z . . . 64
7.3 Per-condition thermal limits . . . 67
7.4 Retained PSII function curves with observed overlay . . . 68
7.5 Model fit: Bayesian R^2 . . . 68

8 Case Study 4: Vinegar fly (*Drosophila suzukii*) TDT across sexes **69**

9 Case Study 4.2: Vinegar fly (*Drosophila suzukii*): Analysing sublethal measures **81**

10 Extended Simulation Results **91**
10.1 *Strict equivalence baseline* . . . 92
10.2 *Likelihood misspecification alone* . . . 94
10.3 *Asymptotes compress symmetrically* . . . 96
10.4 *Coverage for shifting* `up` *across temperatures* (β_{up}) . . . 98
10.5 *Coverage for constant-but-reduced* `up` (up_0) . . . 99
10.6 *Experimental Design Impacts* . . . 100

11 Manuscript Figure 5: cross-case-study summary **105**

12 Manuscript Figure 6: Heat injury and survival **113**

13 Fitting models with `freqTLS` **119**

14 Derivation of the correction factors for threshold summaries **120**
14.1 Why a correction is needed . . . 120
14.2 Step-by-step derivation of the $\log_{10} t_{50}$ correction . . . 121
14.3 From $\log_{10} t_{50}$ to the CT_{max} correction . . . 122
14.4 Generalisation to arbitrary survival thresholds $x/100$ . . . 122
14.5 The slope (and hence z) is robust to the correction . . . 123
14.6 When the correction vanishes . . . 123
14.7 Numerical sanity check . . . 123

15 References **125**

16 Session Information **126**

```
#
# Cached simulation outputs + model fits live on the public OSF deposit (node
# c6dxy). Download them ONLY if they are not already present locally. Note that, you will need
.have_results <- length(Sys.glob(here::here("output", "sim_twostage", "per_sim_*.rds"))) > 0
.have_models  <- length(Sys.glob(here::here("output", "models", "*.rds"))) > 0
```

```
.fetch_script <- here::here("scripts", "fetch_artifacts.R")
if (!(.have_results && .have_models) && file.exists(.fetch_script)) {
  try(system2("Rscript", .fetch_script, stdout = FALSE, stderr = FALSE), silent = TRUE)
}
```

1 General Introduction

This supplement demonstrates the joint Bayesian 4-parameter logistic (4PL) framework from the main manuscript. To cite the approach and the associated R package please use:

Noble, D.W.A., Arnold, P.A., Nakagawa, S. & Pottier, P. (2026). A flexible modelling framework for estimating thermal tolerance and sensitivity. *bioRxiv*. https://doi.org/10.64898/2026.07.16.738378

The supplement has two parts. The first introduces the `bayesTLS` R package using simulated data and empirical case studies. The second reports the detailed models fit for each case study, extended simulation results and provides the code used to reproduce the main manuscript figures.

Simulation outputs and model fits are large! There are several chunks below that read relevant simulation files via `readRDS()` and use these to reproduce output and figures. All the files are archived on the public OSF deposit (node `c6dxy`). They are fetched automatically when this document is rendered (the setup chunk above runs `scripts/fetch_artifacts.R`, which downloads only the files not already present locally), or you can pull them ahead of time with `make data`.

2 Introduction to `bayesTLS` R Package

We use the **`bayesTLS`** R package included with this project. The package fits 4PL models, extracts classical TLS quantities (z, $CT_{max_{1hr}}$, and, for lethal endpoints, T_{crit}) with posterior uncertainty, and can estimate z and $CT_{max_{1hr}}$ directly so group differences and random effects are easy to interpret. It also predicts heat-injury accumulation and survival under arbitrary temperature time series, optionally with a Sharpe–Schoolfield repair kernel (Arnold *et al.* 2025).

All the data used throughout the manuscript and supplement are already preloaded into **`bayesTLS`**, so are easily accessible.

The **`bayesTLS`** package is complemented by **`freqTLS`**, which fits the same models in a frequentist likelihood framework. `freqTLS` is faster and useful for large datasets, while `bayesTLS` is useful when posterior draws are needed for uncertainty propagation as it's much easier to propagate uncertainty into derived estimates. We show in Section 13 that the two packages recover the same z and $CT_{max_{1hr}}$ on one of the case studies.

This tutorial focuses on **`bayesTLS`**, which is available on CRAN. Installing `bayesTLS` pulls in its modelling dependencies (`brms`, `dplyr`, `ggplot2`, `posterior`, `tibble`, `patchwork`) automatically; the tutorial also uses a few small helper packages:

```r
# Install the released version of bayesTLS from CRAN.
install.packages("bayesTLS")

# Or install the development version from GitHub:
# install.packages("remotes")   # if needed
# remotes::install_github("daniel1noble/bayesTLS")

# Install helper packages used in the tutorial.
install.packages(c("here", "purrr", "tidyr"))
```

Every fit below passes `file = file.path(models_dir, "...")`, so re-running a chunk reloads the saved model instead of refitting it. For this tutorial, the folder models are stored are in `output/models`. If you don't already have this folder it will create it for you. Run this setup chunk once, before the analyses that follow:

```r
# Load bayesTLS and helper packages.
# Install pacman first if needed.
if (!requireNamespace("pacman", quietly = TRUE)) install.packages("pacman")
pacman::p_load(bayesTLS, brms, dplyr, tidyr, purrr, ggplot2, here)

set.seed(123)

# Cache fitted brms models in output/models.
models_dir <- here::here("output", "models")
if (!dir.exists(models_dir)) dir.create(models_dir, recursive = TRUE)
```

The functions in `bayesTLS` chain together as a single pipeline with the following logic:

1. **Standardise** raw experimental data to a common column schema (`temp`, `duration`, `n_total`, `n_surv`, …). This step is important because it also stores meta-data about the centering temperatures used which ensures calculations are made correctly.
2. **Fit** the joint Bayesian 4PL with the built-in `beta_binomial(link = "identity")` family and asymptote-bounded reparameterisation. We also have other families built in depending on the response variable distribution.
3. **Extract** z, CT_{max} (at a chosen reference exposure) and, for lethal endpoints, T_{crit} — all as transforms of the fitted posterior, with full uncertainty. `tls()` is the general extractor: it works on any fitted 4PL — the `fit_4pl()` workflow or a hand-coded `brms` model — and returns results **per moderator group** (`by =`, e.g. species, sex or oxygen treatment) as tidy-long summaries plus the underlying posterior draws. Its `target_surv` argument sets the survival threshold the curve is read at — the relative midpoint (default), the absolute LT50, or any LTx.
4. **Predict** survival across a temperature × duration grid, and heat-injury accumulation with predicted survival under fluctuating field temperature traces — both group-aware (`by =`) and with optional damage repair.
5. **Plot** each step — survival curves, tolerance landscapes and heat-injury trajectories — with a shared project theme so figures across the workflow read consistently.

Every exported function is documented (`?fit_4pl`, etc.) and tested against known simulation truths. The supplement first demonstrates the pipeline in controlled simulations and then applies it to case studies.

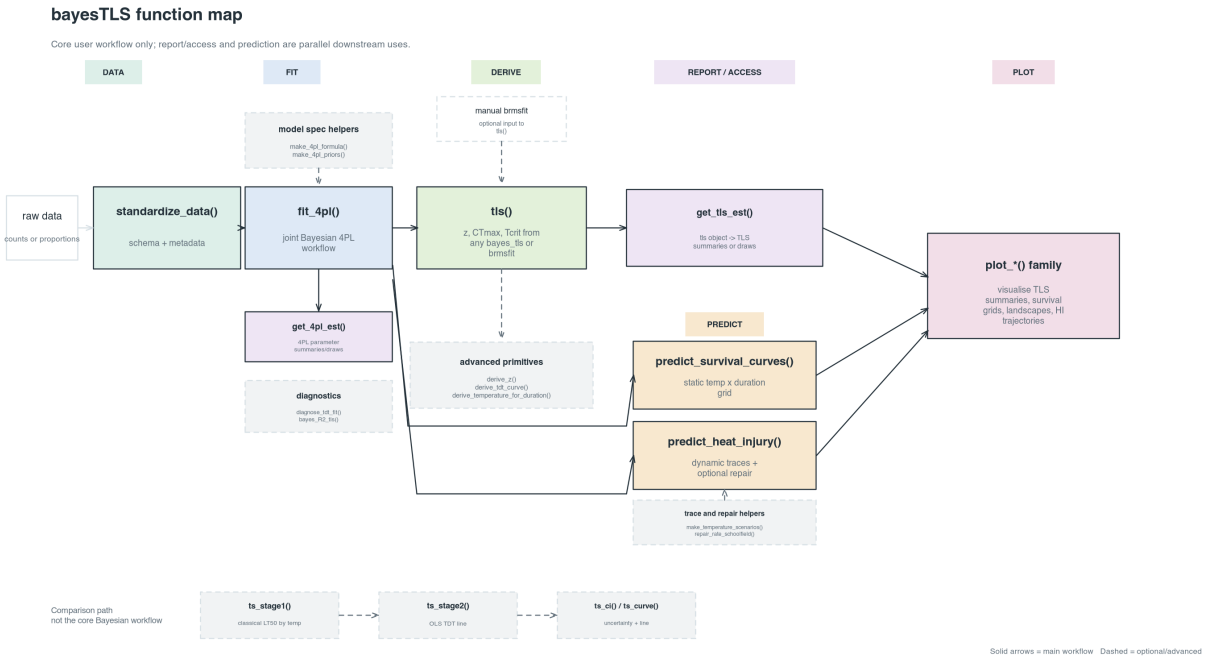


Figure S1: Core user-facing `bayesTLS` function map. Solid arrows show the main Bayesian workflow from data standardisation through model fitting and posterior-derived TLS quantities. Report/access functions are object-specific: `get_4pl_est()` reads the fitted `fit_4pl()` workflow and `get_tls_est()` reads a `tls()` object. Prediction functions also take the fitted workflow, while plotting functions visualise outputs from the report/access or prediction branches. Dashed arrows show optional or advanced routes, including manual `brms` input, model-specification helpers, diagnostics, derivation primitives and the conventional two-stage approach.

2.1 *Description of Functions*

2.1.0.1 *Core functions*

Table S1 lists the key functions that are part of the `bayesTLS` package. Generally, these are the ones users will use most. These functions are used for standardising data, modelling, deriving the TLS quantities of interest, and predicting heat injury and survival.

In addition, if `brms` 4PL models are fit manually to data we also provide a `tls()` function that works more generally across fitted models to derive relevant TLS quantities.

2.1.0.2 *Full package functions*

Table S1: Core `bayesTLS` functions for a standard workflow, following the five pipeline steps. `tls()` is the general entry point for deriving TDT quantities from any fitted 4PL, including hand-coded `brms` models. See Table S2 for the full list of exported functions.

Step	Function	Purpose
Standardise	standardize_data()	Reshape raw experimental data to the package schema (temp, durati
Fit	fit_4pl()	Fit the joint Bayesian 4PL in one call (builds priors and the brms fo
Extract	tls()	Derive z, CTmax and (for lethal endpoints 'lethal = TRUE') T_crit
Extract	get_tls_est()	Pull the derived TLS quantities (z, CTmax, T_crit) out of a 'tls()' o
Extract	get_4pl_est()	Pull the natural-scale 4PL parameters (low, up, k, mid) out of a fitte
Predict	predict_survival_curves()	Posterior survival across a temperature x duration grid, with credible
Predict	predict_heat_injury()	Heat-injury accumulation and predicted survival under a fluctuating
Plot	plot_tdt_curve()	Visualise the TDT (LT_x) curve on a log-time axis; grouped curves

There are many helper functions in the package that maybe useful for various users. Table S2 lists every exported function in the library, grouped by pipeline stage with a description of what they are designed to do.

3 A Tutorial with Simulations

This tutorial uses simulated data to show how the Bayesian hierarchical 4PL recovers thermal sensitivity (z) and $CT_{max_{1hr}}$ in both binomial and beta-binomial (overdispersed) settings. The joint model propagates uncertainty through downstream quantities and accommodates extensions that are less convenient in a two-stage analysis. The subsequent case studies apply the same workflow to data presented in the main manuscript.

3.0.1 *Simulating Data*

3.0.1.1 Step 1 — Setting up parameters

We choose values that might be typical of a thermal mortality assay. The four parameters are the lower and upper asymptotes (low, up), the slope on the $\log_{10}$-time axis (k), and a midpoint that varies linearly with temperature: $\text{mid}(T) = \beta_0 + \beta_1(T - \bar{T})$. Each value has a clear biological meaning (detailed in the code below).

```
# Define the generating parameters for the tutorial simulation.
true_params <- list(
  low      = 0.03,   # 3% of individuals survive even at the longest exposures
  up       = 0.97,   # 97% are alive at very short exposures
  k        = 8,      # steepness of the survival drop on the log10(t) axis
  m_beta0  = 1.5,    # midpoint at the grand-mean temperature: ~31.6 min
  m_beta1  = -0.15,  # midpoint drops 0.15 log10-min per °C; z = -1/beta1   6.7 °C
```

Table S2: Functions exported by the TDT analysis library, grouped by pipeline stage. Each function is documented with roxygen2 in its R file; runnable examples accompany every function.

Stage	Function	Purpose
Data	standardize_data()	Rename raw columns to project schema; attach metada
Model spec	make_4pl_priors()	Weakly informative priors for the disjoint-bounds 4PL.
Model spec	make_4pl_formula()	brms non-linear formula; identity link, all four sub-para
Fit	fit_4pl()	Joint Bayesian 4PL fit; returns a 'bayes_tls' object (us
Inspection	print.bayes_tls()	Compact one-screen header: data shape, T_bar, asymp
Inspection	summary.bayes_tls()	Delegates to 'summary()' on the underlying 'brmsfit'; r
Inspection	plot.bayes_tls()	Delegates to 'plot()' on the underlying 'brmsfit'; return
Inspection	has_fit()	Predicate: is a 'bayes_tls' workflow fitted (TRUE) or s
Inspection	extract_4pl_pars()	Posterior draws of the natural-scale 4PL parameters: lo
Inspection	get_brmsfit()	Return the underlying 'brmsfit' object held inside a 'ba
Inspection	print.tls()	Compact print method for a 'tls' object: shows the per
Accessors	get_tls_est()	Draws or summary of any or all TLS quantities (z, CT
Accessors	get_4pl_est()	Draws or summary of the natural-scale 4PL parameter
Accessors	get_surv_draws()	Per-draw posterior of survival probability from 'predict
Accessors	get_hi_draws()	Per-draw posterior of heat-injury integrals from 'predic
Diagnostics	diagnose_tdt_fit()	Rhat / ESS / divergences / treedepth, with pass-fail fla
Diagnostics	tdt_parameter_table()	Posterior summary on the natural scale (low, up, k, mi
Diagnostics	bayes_R2_tls()	Bayesian R^2 for a fitted 'bayes_tls' workflow (posteri
Predictions	predict_survival_curves()	Survival curves on a temp x duration grid with credibl
Predictions	derive_tdt_landscape()	Dense 2-D survival surface for a heatmap (per group v
Predictions	summarise_observed_survival()	Observed survival mean and SE per (temp, duration) c
TDT quantities	derive_tdt_curve()	Time to threshold survival across temperature. Default
TDT quantities	derive_temperature_for_duration()	Temperature at threshold survival for a fixed exposure
TDT quantities	derive_z()	Thermal sensitivity z read directly from the posterior (
TDT quantities	tls()	One call returning z, CTmax and (when 'lethal = TRU
TDT quantities	tls_z()	Convenience wrapper for 'tls(params = "z")' — z only.
TDT quantities	tls_ctmax()	Convenience wrapper for 'tls(params = "ctmax")' — C
TDT quantities	tls_tcrit()	Convenience wrapper for 'tls(params = "tcrit", lethal =
Two-stage	ts_stage1()	Classical Stage 1: per-temperature dose-response (bino
Two-stage	ts_stage2()	Classical Stage 2: OLS of log10(LT50) on temperature
Two-stage	ts_ci()	Two-stage uncertainty: delta-method Normal/t interva
Two-stage	ts_curve()	Median LT-vs-temperature line from a two-stage fit.
Heat injury	make_temperature_scenarios()	Three reference traces (flat / single spike / multi-spike)
Heat injury	planted_dose_from_trace()	Analytical HI under known z, CTmax, T_c (validation
Heat injury	predict_heat_injury()	Posterior HI and survival under a temperature trace (p

```
  T_bar    = 34      # grand-mean temperature
)
```

The implied thermal sensitivity is $z = -1/\beta_1 \approx 6.7$ °C: a temperature change of this magnitude shifts LT50 by a factor of 10. Savvy readers will note that this is technically relative z, but because our asymptotes are 0.03 and 0.97, despite these shifts from 0 and 1, we are still at a midpoint of 50% survival. So, in this case, they are the same. The same parameters determine the uncentred TDT intercept $\alpha = \beta_0 + \frac{1}{k}\ln\left(\frac{\text{up}-0.5}{0.5-\text{low}}\right) - \beta_1 \bar{T}$ and the 1-hour critical thermal limit $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$. We use these known values to assess how well the models are performing, but do a proper simulation in Section 10.

We first calculate α, z, and $CT_{max_{1hr}}$ from the generating parameters to provide reference values for the fitted models.

```
# Compute the true TDT summaries implied by the simulation parameters.
true_z      <- -1 / true_params$m_beta1

# Convert the centred midpoint model to the original temperature scale.
true_alpha  <- true_params$m_beta0 +
  (1 / true_params$k) *
  log((true_params$up - 0.5) / (0.5 - true_params$low)) -
  true_params$m_beta1 * true_params$T_bar
true_CTmax  <- (log10(60) - true_alpha) / true_params$m_beta1

# Tcrit uses the geometric midpoint of the default heat-injury rate range.
true_T_crit <- true_CTmax - 2.5 * true_z
```

3.0.1.2 Step 2 — Simulated study design

We mimic a well-replicated TDT experiment: five assay temperatures crossed with six log-spaced exposure durations, with 30 replicates of 30 individuals per cell. The resulting 900 trials and 27,000 individuals make parameter recovery clear, but the case studies below and simulations illustrate less information-rich designs.

```
# Build the temperature-by-duration sampling design for the simulation.
temps     <- c(30, 32, 34, 36, 38)             # 5 assay temperatures (°C)
durations <- c(1, 5, 15, 45, 135, 405)         # 6 durations (minutes, log-spaced)
n_rep     <- 30                                # 30 replicates per cell
n_per_rep <- 30                                # 30 individuals per replicate

design <- tidyr::expand_grid(
  T   = temps,
  t   = durations,
  rep = seq_len(n_rep)
) |>
  dplyr::mutate(
    log10_t = log10(t),
```

```
  T_c     = T - true_params$T_bar,                          # mean-centred T
  mid     = true_params$m_beta0 + true_params$m_beta1 * T_c,  # mid(T)
  p_true  = true_params$low +
    (true_params$up - true_params$low) /
    (1 + exp(true_params$k * (log10_t - mid))),
  n = n_per_rep
)
```

3.0.1.3 Step 3 — Look at the truth before simulating

Before generating noisy data, let's plot the *true* survival surface (Figure S2) so we know what the model is being asked to recover. Each line is the 4PL at one assay temperature, plotted against $\log_{10}$ exposure time.

```r
# Plot the true survival surface before sampling noise is added.
truth_grid <- tidyr::expand_grid(
  T = temps,
  t = 10 ^ seq(log10(0.5), log10(1000), length.out = 200)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - true_params$T_bar,
    mid     = true_params$m_beta0 + true_params$m_beta1 * T_c,
    p_true  = true_params$low +
      (true_params$up - true_params$low) /
      (1 + exp(true_params$k * (log10_t - mid)))
  )

ggplot2::ggplot(truth_grid,
                ggplot2::aes(log10_t, p_true, colour = factor(T))) +
  ggplot2::geom_hline(yintercept = 0.5, linetype = "dashed", colour = "grey50") +
  ggplot2::geom_line(linewidth = 0.8) +
  ggplot2::labs(
    x = expression(log[10]~exposure~time~(min)),
    y = "True survival probability",
    colour = "Temperature (°C)"
  ) +
  ggplot2::theme_minimal()
```

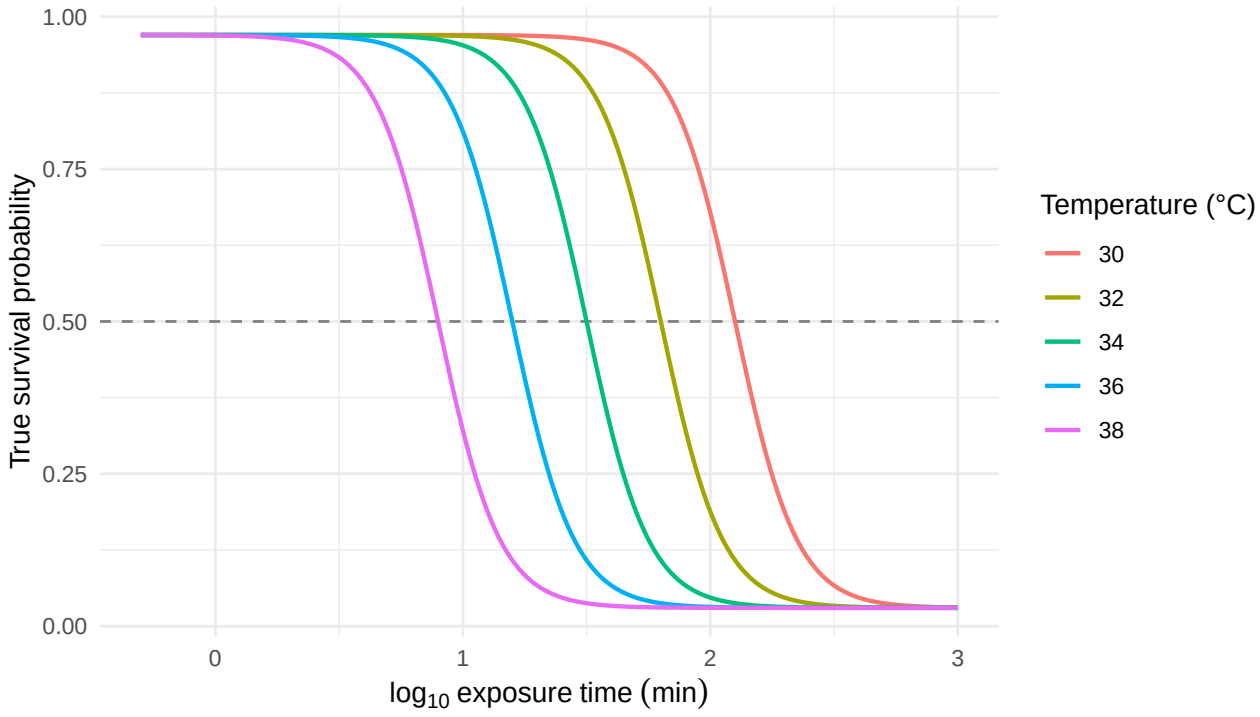


Figure S2: The known 4PL dose-response surface used to generate the simulated data, evaluated on a dense time grid at each of the five assay temperatures. The horizontal line marks 50% survival; where each curve crosses it gives the true LT50 at that temperature.

Notice the curves shift left as temperature rises: hotter temperatures kill faster, so the LT50 is reached at shorter durations.

3.0.1.4 Step 4 — Generate noisy data: binomial and beta-binomial

Survival counts may exhibit two forms of variation. Under **binomial** sampling, each replicate's survival count is drawn from Binomial(n, p), where p is given by the 4PL surface. **Beta-binomial** sampling adds *overdispersion*: replicate-to-replicate variation in p, for example from cohort effects or unmodelled heterogeneity. When TDT data show this extra variation, the beta-binomial likelihood is more appropriate (note that our 4PL fit function defaults to beta-binomial to protect against mis-fits - over- or under-dispersion).

```
# Simulate binomial and beta-binomial counts from the true 4PL surface.
# (a) Binomial: y ~ Binomial(n, p_true)
sim_bin <- design |>
  dplyr::mutate(y = rbinom(dplyr::n(), size = n, prob = p_true))

# (b) Beta-binomial: replicate-level p drawn from Beta(p*phi, (1-p)*phi)
phi <- 5  # smaller phi = more overdispersion
sim_bb <- design |>
  dplyr::mutate(
    p_draw = rbeta(dplyr::n(), p_true * phi, (1 - p_true) * phi),
    y      = rbinom(dplyr::n(), size = n, prob = p_draw)
  )
```

3.0.2 *The classical two-stage TDT pipeline*

We first run the simulated data through the **classical two-stage TDT pipeline** (Ørsted *et al.* 2022; Rezende *et al.* 2014), which is dominant in the thermal-tolerance literature.

1. **Stage 1.** At each assay temperature, fit a logistic dose-response curve to the count data and read off a point estimate of $\log_{10} \mathrm{LT50}_T$.
2. **Stage 2.** Regress those Stage-1 point estimates on assay temperature with ordinary least squares, then derive $z = -1/\beta_1$ and $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$ from the fitted line.

3.0.2.1 Stage 1 — Dose-response curves at each temperature

We fit a binomial GLM with a logit link separately at each assay temperature. This is a common Stage-1 specification in the TDT literature: a two-parameter logistic with asymptotes constrained to 0% and 100% (e.g., Ørsted *et al.* 2024). The midpoint $\log_{10} \mathrm{LT50}_T = -\beta_0/\beta_1$ falls out of the GLM's two coefficients.

```
# Fit the per-temperature dose-response curves for the classical Stage-1 analysis.
# Stage 1 uses the package function ts_stage1(): a per-temperature binomial GLM
# (the field-default Stage 1). We fit both a binomial and a beta-binomial GLM:
# the two families differ chiefly in the LT50 standard errors (which the
# classical Stage 2 discards), so the LT50 *point* estimates are near-identical
```

Table S3: Per-temperature Stage-1 estimates of $\log_{10}$ LT50, fit at each of the five assay temperatures with a binomial GLM (logit link) and, for comparison, a beta-binomial GLM. Truth values are the simulated midpoints (which coincide with $\log_{10}$ LT50 here because the simulation asymptotes are near 0 and 1 and change equally).

T (°C)	Truth log10(LT50)	Binomial log10(LT50)	Beta-binomial log10(LT50)
30	2.1	2.07	2.023
32	1.8	1.78	1.77
34	1.5	1.49	1.529
36	1.2	1.21	1.211
38	0.9	0.92	0.932

```
# -- the binomial and beta-binomial columns of tbl-stage1-lt50 confirm this.
stage1_bin <- ts_stage1(sim_bin, temp = "T", duration = "t",
                        n_surv = "y", n_total = "n", family = "binomial")
stage1_bb  <- ts_stage1(sim_bb,  temp = "T", duration = "t",
                        n_surv = "y", n_total = "n", family = "betabinomial")
```

```
# Format the Stage-1 LT50 estimates and simulation truth.
stage1_table <- dplyr::full_join(
  stage1_bin |> dplyr::select(temp, bin_log10_lt50 = log10_lt50),
  stage1_bb  |> dplyr::select(temp, bb_log10_lt50  = log10_lt50),
  by = "temp"
) |>
  dplyr::mutate(
    truth_log10_lt50 = true_params$m_beta0 +
      true_params$m_beta1 * (temp - true_params$T_bar)
  ) |>
  dplyr::select(temp, truth_log10_lt50, bin_log10_lt50, bb_log10_lt50) |>
  dplyr::rename(
    `T (°C)`              = temp,
    `Truth log10(LT50)`   = truth_log10_lt50,
    `Binomial log10(LT50)` = bin_log10_lt50,
    `Beta-binomial log10(LT50)` = bb_log10_lt50
  )

tinytable::tt(stage1_table, digits = 3, escape = TRUE)
```

With the simulation's true asymptotes near 0 and 1, the two-parameter dose-response function forced-asymptote constraint introduces negligible bias here, so each Stage-1 estimate sits close to the simulation truth.

3.0.2.2 Stage 2 — log-linear TDT regression

Stage 2 regresses Stage-1 $\log_{10}$ LT50 on assay temperature with ordinary least squares. The classical TDT quantities — $z = -1/\beta_1$ and $CT_{max_{1hr}} = (\log_{10} 60 - \alpha)/\beta_1$ — fall out of the fitted line.

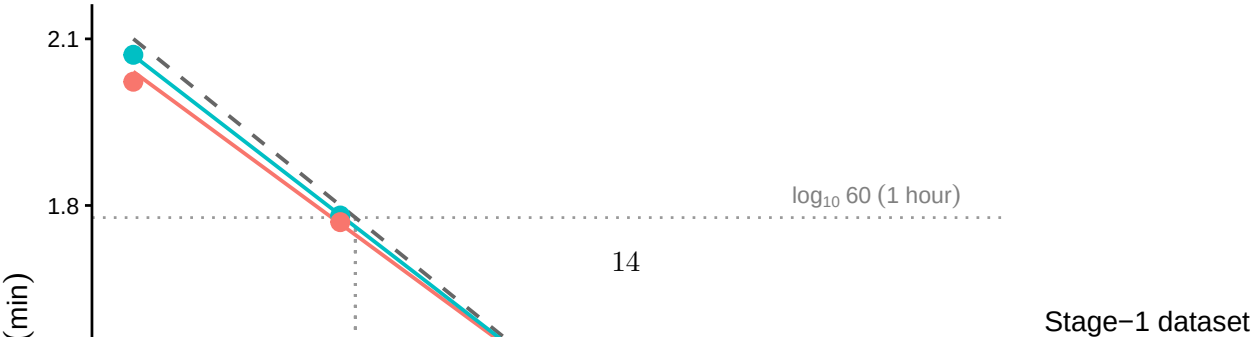
```
# Regress log10(LT50) on temperature and derive z, CTmax and Tcrit.
stage2_bin <- ts_stage2(stage1_bin, t_ref = 60, time_multiplier = 1)
stage2_bb  <- ts_stage2(stage1_bb,  t_ref = 60, time_multiplier = 1)
```

Figure S3 visualises the Stage-2 fit. Each point is one Stage-1 $\log_{10}$ LT50 estimate from Table S3; the solid line is the OLS regression through them. The slope is β_1, from which $z = -1/\beta_1$ — i.e., the temperature increase that causes LT50 to change by a factor of 10. The vertical dotted line marks the simulation truth's $CT_{max_{1hr}}$ (the temperature at which the *true* $\log_{10}$ LT50(T) line crosses $\log_{10} 60 \approx 1.78$, i.e. 1 hour), so we can check how close the OLS line passes to it. The dashed grey line is the simulation truth.

```
# Plot the Stage-2 regression and CTmax reference point.
stage1_combined <- dplyr::bind_rows(
  stage1_bin |> dplyr::mutate(dataset = "Binomial"),
  stage1_bb  |> dplyr::mutate(dataset = "Beta-binomial")
)

truth_line <- tibble::tibble(
  T = seq(min(temps), max(temps), length.out = 100)
) |>
  dplyr::mutate(
    log10_lt50 = true_params$m_beta0 +
      true_params$m_beta1 * (T - true_params$T_bar)
  )

ggplot2::ggplot(stage1_combined,
                ggplot2::aes(x = temp, y = log10_lt50, colour = dataset)) +
  ggplot2::geom_hline(yintercept = log10(60),
                      linetype = "dotted", colour = "grey60") +
  ggplot2::geom_segment(
    ggplot2::aes(x = true_CTmax, xend = true_CTmax,
                 y = log10(60), yend = -Inf),
    inherit.aes = FALSE,
    linetype = "dotted", colour = "grey60"
  ) +
  ggplot2::geom_line(data = truth_line,
                     ggplot2::aes(x = T, y = log10_lt50),
                     inherit.aes = FALSE,
                     linetype = "dashed", colour = "grey40",
                     linewidth = 0.7) +
  ggplot2::geom_smooth(method = "lm", se = FALSE, linewidth = 0.7) +
  ggplot2::geom_point(size = 3) +
  ggplot2::annotate("text", x = max(temps), y = log10(60),
                    label = "log[10]~60~(1*' hour')", parse = TRUE,
                    hjust = 1, vjust = -0.4, colour = "grey50", size = 3) +
  ggplot2::annotate("text", x = true_CTmax, y = -Inf,
                    label = "CT[max[1*hr]]", parse = TRUE,
                    hjust = -0.15, vjust = -0.7, colour = "grey50", size = 3) +
  ggplot2::labs(
    x = "Assay temperature (°C)",
    y = expression(log[10]~LT[50]~(min)),
    colour = "Stage-1 dataset"
  ) +
  ggplot2::theme_classic()
```



Point estimates of z and $CT_{max_{1hr}}$ are then used as-is, often without uncertainty associated with these estimates. We compare these to the joint Bayesian results to show the 4PL gives the same values under these ideal conditions.

> **!** Important
>
> Classical two-stage analyses often treat Stage-1 LT_{50} estimates as known when fitting Stage-2 models. In otherwords, uncertainty in LT_{50} values are never accounted for in the Stage-2 regression. This can have consequences when estmating slopes and intercepts in Stage-2.
> Despite being a potential issue, it is also challenging to correctly propagate error to z and $CT_{max_{1hr}}$. Incorrectly doing so in Stage-2 may have bigger impacts. Without propagating uncertainty properly these will systematically bias sampling uncertainty in z and $CT_{max_{1hr}}$ compromising inferences and leading to optimistic predictions. The Delta-method or bootstrap procedures can be used to propagate uncertainty, though, these are rarely used in practice. In addition, even if uncertainty was correctly propagated, a likelihood approach makes it harder to propogate uncertainty through downstream predictions (e.g., heat injury).

3.0.3 *A joint Bayesian 4PL*

Instead of fitting separate logistic curves and then regressing their LT50 estimates, the joint fit estimates low, up, k, β_0, and β_1 simultaneously from the raw counts. We use the same simulated data to derive z and $CT_{max_{1hr}}$ directly from this model from the posteriors of these parameters.

3.0.3.1 Specifying the 4PL in `brms`

The model has two pieces. The first is the 4PL itself — probability as a function of $\log_{10}$ exposure time:

$$p \;=\; \text{low} \;+\; \frac{\text{up} - \text{low}}{1 + \exp\!\big(k\,(\log_{10} t - \text{mid})\big)}, \tag{S1}$$

where low and up are the lower and upper asymptotes, k is the slope on the $\log_{10} t$ axis, and mid is the value of $\log_{10} t$ at which the curve crosses the midpoint between low and up. The second piece lets the midpoint vary linearly with mean-centred temperature:

$$\text{mid} \;=\; \beta_0 + \beta_1\,(T - \bar{T}), \tag{S2}$$

so that β_1 encodes how fast the dose-response curve shifts on the time axis as assay temperature changes; $\bar{T}$ is the grand-mean temperature, included so that β_0 is interpretable as the midpoint at the average assay temperature.

The function library exposes this model as a single call: `fit_4pl()` constructs the brms `bf()` formula, the default weakly-informative priors, and the sampling controls, and returns a workflow object containing the fit. Internally the formula uses the disjoint-bounds reparameterisation introduced in this section — low $= 0.001 +$ inv_logit(lowraw) $\cdot\, 0.498$ and up $= 0.501 +$ inv_logit(upraw) $\cdot$

0.498 — so the 4PL output is guaranteed to lie in $(0, 1)$ regardless of where MCMC takes the unconstrained `lowraw`, `upraw`, and `logk` parameters. The default also puts a `temp_c` slope on every 4PL sub-parameter; when temperature has no real effect on a given parameter, the slope shrinks toward zero under the prior. We inspect the brms formula `fit_4pl()` builds with:

```
# Show the 4PL brms formula fit_4pl() builds. The asymptote-bound mapping is
# derived from the package's own bound helper (compute_4pl_bounds) rather than
# hardcoded, so the displayed coefficients track the pad/gap defaults and cannot
# drift from what fit_4pl() actually uses.
f <- make_4pl_formula()
b <- getFromNamespace("compute_4pl_bounds", "bayesTLS")(0, 1)
cat("4PL dose-response (disjoint-bounds reparameterisation; survival stays in (0, 1)):\n\n")
```

```
4PL dose-response (disjoint-bounds reparameterisation; survival stays in (0, 1)):
```

```
cat("  ", paste(deparse(f$formula[[2]]), collapse = ""),
    "  ~  low + (up - low) / (1 + exp(k * (logd - mid)))\n", sep = "")
```

```
  n_surv | trials(n_total)  ~  low + (up - low) / (1 + exp(k * (logd - mid)))
```

```
cat(sprintf("      low = %.3f + inv_logit(lowraw) * %.3f\n", b$low_min, b$low_w))
```

```
      low = 0.001 + inv_logit(lowraw) * 0.498
```

```
cat(sprintf("      up  = %.3f + inv_logit(upraw)  * %.3f\n", b$up_min,  b$up_w))
```

```
      up  = 0.501 + inv_logit(upraw)  * 0.498
```

```
cat("      k   = exp(logk)\n\n")
```

```
      k   = exp(logk)
```

```
cat("  Sub-parameter linear predictors (default: a temp_c slope on each):\n")
```

```
  Sub-parameter linear predictors (default: a temp_c slope on each):
```

```
for (nm in c("lowraw", "upraw", "logk", "mid"))
  cat(sprintf("      %-6s ~ %s\n", nm, paste(deparse(f$pforms[[nm]][[3]]), collapse = "")))
```

```
      lowraw ~ temp_c
      upraw  ~ temp_c
      logk   ~ temp_c
      mid    ~ temp_c
```

> **!** Important
>
> The default model puts a `temp_c` slope on all four 4PL sub-parameters and includes no random effects. How random effects are added depends on the parameterisation (see the manuscript for differences). In the **midpoint** parameterisation (used here) pass `random_effects = c("Date", "Tank", ...)` to `fit_4pl()` to place random intercepts on `mid`. In the **direct** CTmax/z parameterisation they instead go *inside* the `ctmax`/`z` formulas, e.g. `ctmax = ~ 0 + group + (1 | batch)`. The simulation here has no batch structure so we leave them off.

3.0.3.2 Standardising and fitting

The first step is to pass the simulated data through `standardize_data()` so the columns match the schema the rest of the library expects (`temp`, `duration`, `logd`, `temp_c`, `n_total`, `n_surv`). This function also keeps track of your reference temperature, time units and centering so it's important to use for calculations to be done correctly.

```
# Convert the simulated counts into the bayesTLS modelling schema.
std_bin <- standardize_data(sim_bin,
                            temp = "T", duration = "t",
                            n_total = "n", n_surv = "y",
                            duration_unit = "minutes")
std_bb  <- standardize_data(sim_bb,
                            temp = "T", duration = "t",
                            n_total = "n", n_surv = "y",
                            duration_unit = "minutes")
```

We now call `fit_4pl()` twice — once with the built-in `binomial(link = "identity")` family for the non-overdispersed simulation, and once with the default `beta_binomial(link = "identity")` for the overdispersed case. We store the model fits using brms's `file = ...` mechanism, so subsequent code calls just reload the cached fits unless the formula or data changes.

```
# Fit the tutorial 4PL models with and without overdispersion.
# Reuse cached fits; delete the .rds files to force refitting.
wf_bin <- fit_4pl(std_bin,
                  family = binomial(link = "identity"),
                  chains = 4, iter = 2000, cores = 4, seed = 123,
                  file       = file.path(models_dir, "sim_4pl_binomial_v2"),
                  file_refit = "never")

wf_bb  <- fit_4pl(std_bb,
                  chains = 4, iter = 2000, cores = 4, seed = 123,
                  file       = file.path(models_dir, "sim_4pl_betabinomial_v2"),
                  file_refit = "never")

# Show brms summaries and diagnostics for each fit.
summary(wf_bin)
```

```
 Family: binomial
  Links: mu = identity
Formula: n_surv | trials(n_total) ~ (0.001 + inv_logit(lowraw) * 0.498) + ((0.501 + inv_logit(u
         lowraw ~ temp_c
         upraw ~ temp_c
         logk ~ temp_c
         mid ~ temp_c
   Data: data (Number of observations: 900)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Regression Coefficients:
                Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
lowraw_Intercept    -2.82      0.09    -3.01    -2.66 1.00     2519     2773
lowraw_temp_c       -0.01      0.03    -0.07     0.05 1.00     2787     2728
upraw_Intercept      2.68      0.07     2.56     2.82 1.00     3357     2710
upraw_temp_c        -0.02      0.02    -0.07     0.03 1.00     3350     2790
logk_Intercept       2.12      0.03     2.07     2.17 1.00     2971     2979
logk_temp_c          0.00      0.01    -0.02     0.02 1.00     3151     2954
mid_Intercept        1.50      0.00     1.49     1.51 1.00     3931     2847
mid_temp_c          -0.15      0.00    -0.15    -0.15 1.00     5742     2551

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
summary(wf_bb)
```

```
 Family: beta_binomial
  Links: mu = identity
Formula: n_surv | trials(n_total) ~ (0.001 + inv_logit(lowraw) * 0.498) + ((0.501 + inv_logit(u
         lowraw ~ temp_c
         upraw ~ temp_c
         logk ~ temp_c
         mid ~ temp_c
   Data: data (Number of observations: 900)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Regression Coefficients:
                Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
lowraw_Intercept    -3.05      0.25    -3.61    -2.63 1.00     2222     2317
lowraw_temp_c        0.10      0.09    -0.06     0.28 1.00     2086     2207
upraw_Intercept      2.79      0.14     2.52     3.08 1.00     3021     2743
upraw_temp_c         0.03      0.05    -0.06     0.13 1.00     3072     2656
logk_Intercept       1.99      0.06     1.89     2.10 1.00     2980     2596
logk_temp_c         -0.01      0.02    -0.06     0.03 1.00     2681     2243
```

```
mid_Intercept        1.52      0.01     1.50     1.54 1.00     3323     2694
mid_temp_c          -0.15      0.00    -0.16    -0.14 1.00     5046     3113

Further Distributional Parameters:
    Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
phi     4.88      0.43     4.08     5.74 1.00     3502     3194

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```r
# Extract the underlying brmsfit objects for brms helpers.
fit_bin <- get_brmsfit(wf_bin)
fit_bb  <- get_brmsfit(wf_bb)
```

We also summarise the proportion of outcome variation explained by each fitted model using Bayesian R^2. For any fitted workflow this is a single `bayes_R2_tls()` call. We define a small helper that maps it over a named list of fits and formats the result into a `median [95% CrI]` table, then reuse it in every case study below.

```r
# bayes_R2_tls() pulls the brmsfit and returns a tidy one-row tibble
# (estimate / est_error / lower / upper) -- one call per fit:
# bayes_R2_tls(wf_bin)

# For the tables we map that single call over a named list of fits and format it
# as "median [95% CrI]"; this thin formatter is reused in every case study below.
bayes_r2_table <- function(fits, digits = 3) {
  purrr::imap_dfr(fits, function(wf, nm) {
    r <- bayes_R2_tls(wf)
    tibble::tibble(
      Model         = nm,
      `Bayesian R²` = format_interval(r$estimate, r$lower, r$upper, digits = digits)
    )
  })
}
```

```r
# Format Bayesian R2 for the tutorial fits.
bayes_r2_table(list(
  "Binomial 4PL"      = wf_bin,
  "Beta-binomial 4PL" = wf_bb
)) |> tinytable::tt(escape = TRUE)
```

3.0.3.3 Checking parameter recovery

We assess recovery of true values by comparing posterior medians and 95% credible intervals with the known generating values (Table S5). Given the large simulated sample, the posterior should concentrate near those values.

Table S4: Bayesian R^2 for the two tutorial joint 4PL fits (binomial and beta-binomial), via `bayes_R2_tls()` (a tidy wrapper around `brms::bayes_R2()`). Cells are the posterior median with the 95% credible interval.

Model	Bayesian R^2
Binomial 4PL	0.988 [0.987, 0.988]
Beta-binomial 4PL	0.926 [0.924, 0.928]

Table S5: Posterior medians and 95% credible intervals for the four 4PL parameters and the temperature slope, compared to the known truth used to generate the simulated data. Recovery is judged successful when the credible interval covers the truth.

Parameter	Truth	Binomial	Beta-binomial
low (lower asymptote)	0.03	0.029 [0.024, 0.034]	0.024 [0.014, 0.035]
up (upper asymptote)	0.97	0.967 [0.963, 0.971]	0.97 [0.962, 0.977]
k (slope)	8	8.316 [7.893, 8.771]	7.327 [6.589, 8.176]
mid intercept (at T_bar)	1.5	1.501 [1.493, 1.51]	1.518 [1.497, 1.538]
mid temp_c slope	-0.15	-0.15 [-0.152, -0.146]	-0.15 [-0.157, -0.142]

```
# tdt_parameter_table() returns the per-parameter posterior summary on the
# natural 4PL scale (median + 95% CrI) directly from a fit, so we don't roll our
# own. We compare the four 4PL parameters and the temperature slope to the known
# truth; the z row it also returns is the derived quantity tabulated in
# @sec-joint-derive. format_interval() renders each cell as "median [2.5%, 97.5%]".
truth <- tibble::tibble(
  Parameter = c("low (lower asymptote)", "up (upper asymptote)", "k (slope)",
                "mid intercept (at T_bar)", "mid temp_c slope"),
  Truth     = round(c(true_params$low, true_params$up, true_params$k,
                      true_params$m_beta0, true_params$m_beta1), 3)
)
recovery_cell <- function(wf) tdt_parameter_table(wf) |>
  dplyr::filter(parameter != "z (°C)") |>
  dplyr::transmute(Parameter = parameter,
                   cell = format_interval(median, lower, upper, digits = 3))
recovery <- truth |>
  dplyr::left_join(recovery_cell(wf_bin) |> dplyr::rename(Binomial = cell),
                   by = "Parameter") |>
  dplyr::left_join(recovery_cell(wf_bb)  |> dplyr::rename(`Beta-binomial` = cell),
                   by = "Parameter")

tinytable::tt(recovery, digits = 3, escape = TRUE)
```

We can also overlay the posterior fitted curves on the simulated data (Figure S4). If the model has recovered the true surface, the fitted lines should track the simulated proportions at every assay

temperature.

```
# Plot fitted survival curves against the simulated observations.
# Use the library's predict_survival_curves() and plot_survival_curves().
psc_bin <- predict_survival_curves(
  wf_bin, temps = temps,
  durations = 10 ^ seq(log10(0.5), log10(800), length.out = 100),
  ndraws = 200
)

plot_survival_curves(psc_bin, observed = std_bin, log_time = TRUE) +
  ggplot2::labs(x = expression(exposure~time~(min)))
```

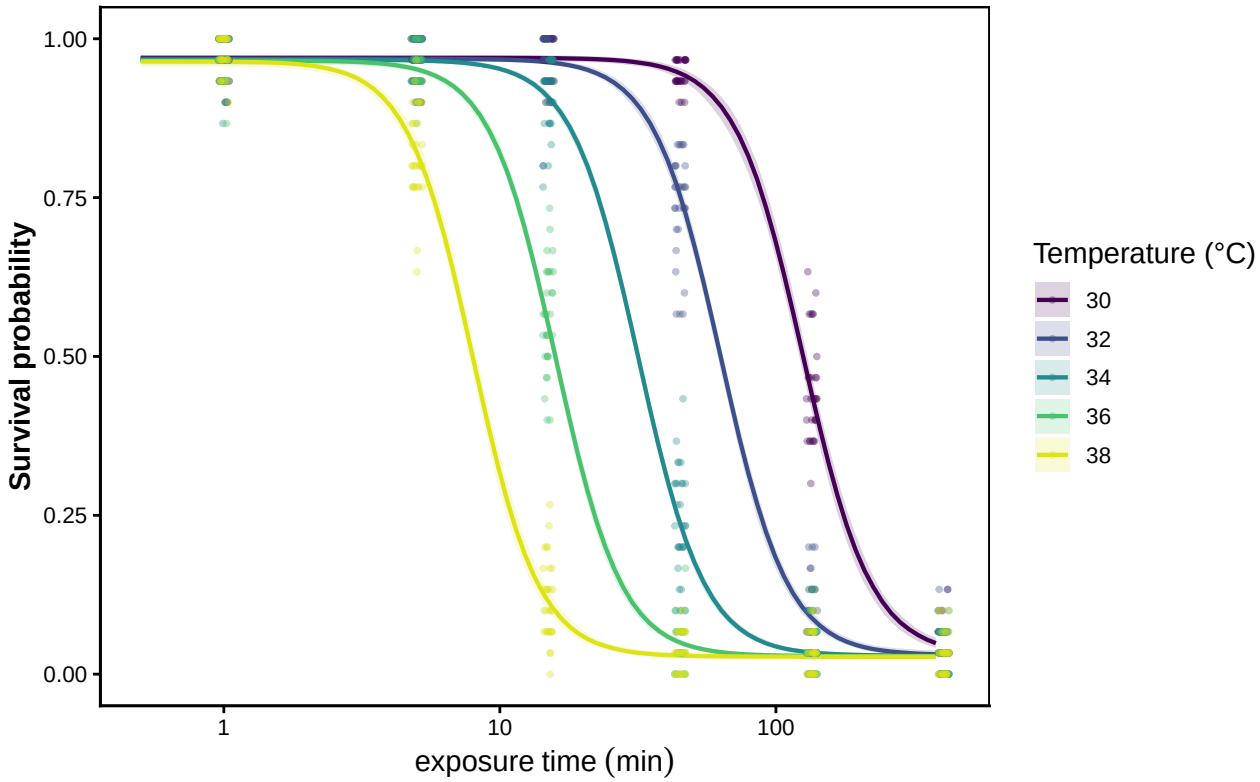


Figure S4: Posterior fitted 4PL curves (solid lines, with shaded 95% credible bands) overlaid on the simulated binomial data (points). One panel-coloured line per assay temperature. Where the fitted curves track the simulated proportions across all five temperatures, the model has recovered the underlying dose-response surface.

3.0.4 *Deriving z, $CT_{max_{1hr}}$, and T_{crit} from the posterior*

The classical TLS quantities are transformations of the fitted joint posterior. The general extractor `tls()` returns the first two quantities below, and the third when `lethal = TRUE`, via its `target_surv`, `t_ref` and `lethal` arguments:

- z — thermal sensitivity is read directly from the posterior as $z = -1/(\mathrm{d}\log_{10}\mathrm{LT}_{\mathrm{threshold}}/\mathrm{d}T)$

per draw (Equation 3 of the manuscript, the midpoint–temperature relation). Under the relative threshold, $\log_{10} \mathrm{LT}(T) = \mathrm{mid}(T)$ and the derivative is simply the linear slope, so $z = -1/\beta_1$ exactly.

- $CT_{max_{1hr}}$ — temperature at which the threshold is reached after 1 h of exposure, obtained from the fitted 4PL at the reference time, per posterior draw.
- T_{crit} *(only when* `lethal = TRUE`*)* — the temperature at which heat injury starts to accumulate. By default, the damage rate at temperature T in the TDT framework is $r(T) = 100 \cdot 10^{(T - CT_{max,1hr})/z}$ % LT50-dose per hour, so inverting for a chosen rate floor r^* gives

$$T_{crit}(r^*) \;=\; CT_{max,1hr} \;+\; z \cdot \log_{10}(r^*/100). \tag{S3}$$

The threshold one wishes to use is set by the `target_surv` argument of `tls()`. The default, `"relative"`, evaluates at (low + up)/2 per draw, which equals classical absolute 50% survival when asymptotes are 0 and 1 but accommodates shifting upper asymptotes (for example, background mortality unrelated to heat stress). Passing `target_surv = "absolute"` or any numeric value in $(0, 1)$ instead uses an absolute survival threshold. Because the simulated asymptotes are near 0 and 1, both definitions give essentially the same results here.

How do we estimate T_{crit}? Different r^* values imply different T_{crit} thresholds, and the literature does not converge on one value. Rather than fix one r^*, we adopt a plausible operational range — $r^* \in [0.1, 1]$ % HI per hour, the heat-injury rate at which we assume net damage begins to accrue — corresponding to approximately 100 to 1000 hours (about 4 to 42 days) of continuous exposure to reach LT_{50}. We integrate over a uniform prior on $\log_{10} r^*$ across this range. The resulting posterior for T_{crit} includes parameter uncertainty in $CT_{max,1hr}$ and z as well as operational uncertainty in r^*. Its median is $CT_{max,1hr} - 2.5 \cdot z$, corresponding to $r^* \approx 0.3$ % HI per hour.

> ⚠ T_{crit} is meaningful only for lethal-endpoint data
>
> The rate-multiplier T_{crit} in Equation S3 interprets z as the temperature sensitivity of *damage accumulation*, which is appropriate for lethal-endpoint dose-response data. It treats T_{crit} as the temperature where damage begins to accumulate, and so can be used to make a prediction about T_{crit}. Users with lethal-endpoint data therefore set `lethal = TRUE` in `tls()` to additionally derive an *estimate* of T_{crit}; with `lethal = FALSE` it returns only z and $CT_{max_{1hr}}$. Note that this is only a prediction and does not negate the need to follow up with empirical studies as validations (see Faber *et al.* 2026).
> For **sublethal** endpoints, fitted z instead describes performance loss and can be very different from z derived from lethal data. Substituting that value into Equation S3 can shift T_{crit} substantially and produce unrealistic T_{crit} estimates. For example, a smaller sublethal z can produce a T_{crit} several °C higher than the lethal estimate despite a similar $CT_{max_{1hr}}$. Interpreting T_{crit} as an intersection between thermal-performance and thermal-death-time curves requires both lethal and sublethal data (Faber *et al.* 2026; Jørgensen *et al.* 2021; Ørsted *et al.* 2022).

Returned quantities (e.g., $CT_{max,1hr}$ and z) inherit the same posterior, so every credible interval reflects joint uncertainty in all fitted 4PL parameters. The opt-in T_{crit} pools that same posterior with the additional uniform prior on $\log_{10} r^*$.

> **Note**
>
> The default `fit_4pl()` puts a `temp_c` slope on every 4PL sub-parameter, so low, up and k can each carry a linear effect of temperature in addition to the linear-mid term — when those slopes shrink to zero the model reduces to the constant-shape case. However, when the data carry a real temperature signal on any of the shape parameters, the asymmetry correction $\frac{1}{k(T)} \ln\left(\frac{\text{up}(T)-0.5}{0.5-\text{low}(T)}\right)$ varies with T and the resulting $\log_{10}\text{LT50}(T)$ curve is bent rather than straight:
>
> $$\log_{10}\text{LT50}(T) \;=\; \beta_0 + \beta_1(T-\bar{T}) \;+\; \frac{1}{k(T)} \ln\left(\frac{\text{up}(T)-0.5}{0.5-\text{low}(T)}\right). \tag{S4}$$
>
> The correction factor is needed when calculating an 'absolute' threshold (e.g., 50% mortality - `target_surv = "absolute"`). Under an *absolute* threshold the asymmetry-correction term $\frac{1}{k(T)} \ln\left(\frac{\text{up}(T)-0.5}{0.5-\text{low}(T)}\right)$ bends the curve when up, low or k carry T effects; then a per-draw **local** $z(T) = -1/(\mathrm{d}\log_{10}\text{LT}_{0.5}/\mathrm{d}T)$ is computed at each assay temperature. The reported z is the per-draw mean of those local values over the assay range — with the full per-temperature $z(T)$ available via `z_local = TRUE`.
> If using a 'relative threshold' (`target_surv = "relative"`) then the calculations do not require the correction factor. $CT_{max_{1hr}}$ is the temperature at which $\log_{10}\text{LT}(T)$ crosses t_{ref}, per draw. For z, under the relative threshold $\log_{10}\text{LT}(T) = \text{mid}(T)$ is exactly linear, so $z = -1/\beta_1$ per draw. The worked example in Section 3.0.6 demonstrates this with a simulation where `up` and k carry strong T effects.
> Under `target_surv = "absolute"`, `tls()` includes the asymmetry-correction term automatically, evaluating it per draw from the same posterior of low, up and k so that any temperature-dependence of those parameters is propagated. The draws used to compute z and $CT_{max_{1hr}}$ are subsampled once across all parameters, so the two quantities share draws — their correlation, and the joint pairing used for T_{crit}, is preserved.

```
# tls() is the general one-call extractor: from any fitted 4PL it returns z,
# CTmax and -- when lethal = TRUE -- T_crit, as a tidy-long summary ($summary)
# plus the underlying posterior draws ($draws). One call per fit; t_ref = 60 min
# sets the 1-hour reference and lethal = TRUE adds T_crit (the endpoint is lethal).
tl_bin <- tls(wf_bin, target_surv = "relative", t_ref = 60, lethal = TRUE, ndraws = 1000)
tl_bb  <- tls(wf_bb,  target_surv = "relative", t_ref = 60, lethal = TRUE, ndraws = 1000)

# We can now get all TLS estimates (z, CTmax, Tcrit): posterior median and 95% interval as foll
get_tls_est(tl_bin, what = "summary")
```

```
# A tibble: 3 x 4
  quantity median lower upper
  <chr>     <dbl> <dbl> <dbl>
1 z          6.69  6.55  6.83
2 CTmax     32.1  32.1  32.2
3 Tcrit     15.5  12.2  18.5
```

Table S6 reads each quantity's median and interval straight off these `tls()` summaries, and Figure S5 plots the matching `$draws`.

Table S6: Posterior medians and 95% credible intervals for thermal sensitivity (z), the critical thermal limit at a 1-hour reference duration ($CT_{max_{1hr}}$), and the rate-multiplier critical temperature (T_{crit}, integrated over $r^* \in [0.1, 1]$ % HI per hour). All three derived via `tls()` from a single posterior. Each row's truth value is computed analytically from `true_params`.

Quantity	Truth	Binomial	Beta-binomial
z (°C)	6.67	6.69 [6.55, 6.83]	6.67 [6.37, 7.03]
CTmax at 1 hr (°C)	32.15	32.15 [32.08, 32.21]	32.26 [32.1, 32.41]
T_crit (°C)	15.48	15.48 [12.19, 18.54]	15.57 [12.08, 18.89]

```
# get_tls_est(x, "summary") is the public accessor for a tls object (used in the
# chunk above too); it returns one row per quantity (z, CTmax, Tcrit) with median
# / lower / upper, in that order -- so we read the table off it rather than the
# raw $summary slot. format_interval() (vectorised) renders each cell as
# "median [2.5%, 97.5%]".
bin_sum <- get_tls_est(tl_bin, what = "summary")
bb_sum  <- get_tls_est(tl_bb,  what = "summary")
tls_recovery <- tibble::tibble(
  Quantity        = c("z (°C)", "CTmax at 1 hr (°C)", "T_crit (°C)"),
  Truth           = round(c(true_z, true_CTmax, true_T_crit), 2),
  Binomial        = format_interval(bin_sum$median, bin_sum$lower, bin_sum$upper),
  `Beta-binomial` = format_interval(bb_sum$median,  bb_sum$lower,  bb_sum$upper)
)

tinytable::tt(tls_recovery, digits = 3, escape = TRUE)
```

```r
# Plot posterior distributions of the recovered TLS quantities. tls()$draws is
# already tidy-long (quantity, .draw, value), so no reshaping is needed -- bind
# the two fits and relabel the quantities for the facets.
post_long <- dplyr::bind_rows(
    tl_bin$draws |> dplyr::mutate(model = "Binomial"),
    tl_bb$draws  |> dplyr::mutate(model = "Beta-binomial")) |>
  dplyr::mutate(quantity = factor(quantity,
                                  levels = c("z", "CTmax", "Tcrit"),
                                  labels = c("z (°C)",
                                             "CTmax_1hr (°C)",
                                             "T_crit (°C)")))

truth_long <- tibble::tibble(
  quantity = factor(c("z", "CTmax", "Tcrit"),
                    levels = c("z", "CTmax", "Tcrit"),
                    labels = levels(post_long$quantity)),
  truth    = c(true_z, true_CTmax, true_T_crit)
)

ggplot2::ggplot(post_long, ggplot2::aes(value, fill = model)) +
  ggplot2::geom_density(alpha = 0.5, colour = NA) +
  ggplot2::geom_vline(data = truth_long,
                      ggplot2::aes(xintercept = truth),
                      linetype = "dashed", colour = "black") +
  ggplot2::facet_wrap(~ quantity, scales = "free") +
  ggplot2::labs(x = NULL, y = "Posterior density", fill = "Model") +
  ggplot2::theme_classic()
```

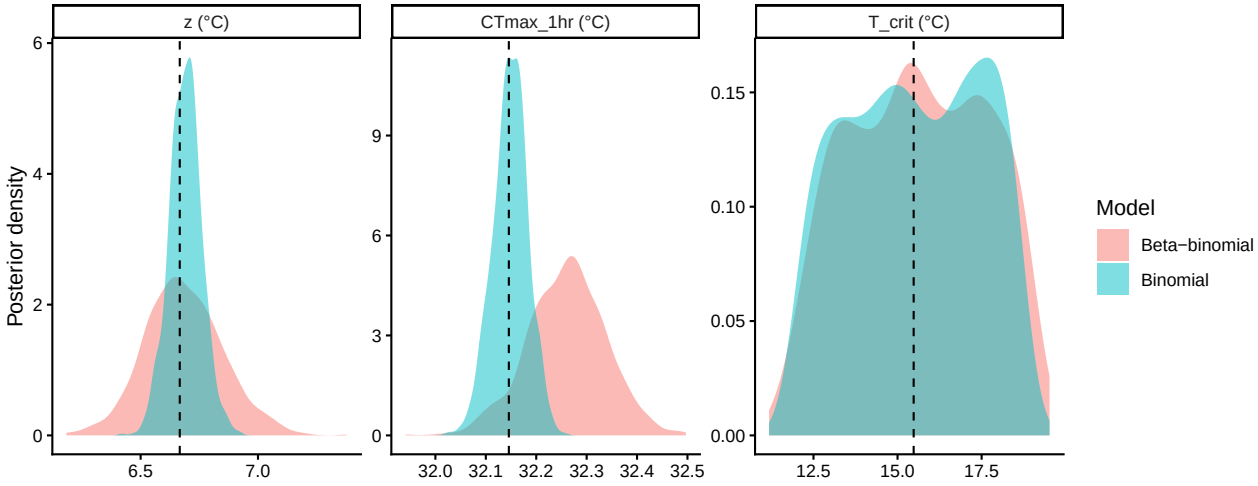


Figure S5: Posterior densities of z, $CT_{max_{1hr}}$, and T_{crit} from the binomial (blue) and beta-binomial (orange) joint fits. Vertical dashed lines mark the simulation truth. All three posteriors concentrate around their respective truths, illustrating that the 4PL model reproduces classical TLS quantities with posterior uncertainty and no separate Stage-2 regression.

Both fits recover all three simulation truths (Table S6, Figure S5), with credible intervals reflecting joint uncertainty in the 4PL parameters. In the joint approach, every classical TLS quantity is derived from the same fitted posterior.

Posterior summary statistics — mean, SD, median, and 95% credible interval — follow directly from the `$draws` slot of the same `tls()` output:

```
# Posterior summaries of z, CTmax and T_crit per likelihood. get_tls_est(x, "draws") returns tl
# value), so we summarise each quantity x fit directly. The inline numbers below
# read off this table.
uncert_summary <- dplyr::bind_rows(
    get_tls_est(tl_bin, "draws") |> dplyr::mutate(model = "Binomial"),
    get_tls_est(tl_bb,  "draws") |> dplyr::mutate(model = "Beta-binomial")) |>
  dplyr::group_by(quantity, likelihood = model) |>
  dplyr::summarise(mean = mean(value), sd = stats::sd(value),
                   median = stats::median(value),
                   lower = stats::quantile(value, 0.025, names = FALSE),
                   upper = stats::quantile(value, 0.975, names = FALSE),
                   .groups = "drop") |>
  dplyr::mutate(
    quantity   = factor(quantity, c("z", "CTmax", "Tcrit"),
                        c("z (°C)", "CTmax at 1 hr (°C)", "T_crit (°C)")),
    likelihood = factor(likelihood, c("Binomial", "Beta-binomial"))
  ) |>
  dplyr::arrange(likelihood, quantity)

# Scalars for inline reporting, read straight off uncert_summary (single source).
g <- function(q, lik, col)
  uncert_summary[[col]][uncert_summary$quantity == q & uncert_summary$likelihood == lik]
sd_z_bin  <- round(g("z (°C)",             "Binomial",      "sd"), 2)
sd_z_bb   <- round(g("z (°C)",             "Beta-binomial", "sd"), 2)
sd_ct_bin <- round(g("CTmax at 1 hr (°C)", "Binomial",      "sd"), 2)
sd_ct_bb  <- round(g("CTmax at 1 hr (°C)", "Beta-binomial", "sd"), 2)
ci_z_bin  <- round(g("z (°C)",             "Binomial", "upper") - g("z (°C)",             "Bino
ci_ct_bin <- round(g("CTmax at 1 hr (°C)", "Binomial", "upper") - g("CTmax at 1 hr (°C)", "Bino

uncert_summary |>
  dplyr::rename(Quantity = quantity, Likelihood = likelihood, Mean = mean,
                SD = sd, Median = median, Lower = lower, Upper = upper) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

Both z and $CT_{max_{1hr}}$ are estimated precisely, and their credible intervals come directly from the joint posterior. A frequentist joint fit could similarly quantify transformation uncertainty using the profiled likelihoods, the Delta method or using bootstrap approaches.

Under the binomial likelihood, the posterior SD for z is 0.07 °C and its 95% credible interval is 0.27 °C wide; $CT_{max_{1hr}}$ has SD 0.03 °C and interval width 0.14 °C.

Table S7: Posterior summaries of z, $CT_{max_{1hr}}$, and T_{crit} from the joint Bayesian 4PL: posterior mean, standard deviation, median, and 95% credible interval, reported separately for the binomial and beta-binomial fits.

Quantity	Likelihood	Mean	SD	Median	Lower	Upper
z (°C)	Binomial	6.69	0.0691	6.69	6.55	6.83
CTmax at 1 hr (°C)	Binomial	32.15	0.0347	32.15	32.08	32.21
T_crit (°C)	Binomial	15.48	1.9572	15.48	12.19	18.54
z (°C)	Beta-binomial	6.68	0.1663	6.67	6.37	7.03
CTmax at 1 hr (°C)	Beta-binomial	32.26	0.0776	32.26	32.1	32.41
T_crit (°C)	Beta-binomial	15.6	2.0115	15.57	12.08	18.89

The beta-binomial fit is slightly wider for both quantities (SD for z: 0.17 °C; for $CT_{max_{1hr}}$: 0.08 °C) because ϕ captures extra-binomial variation and propagates it into the derived quantities. A two-stage pipeline requires an additional bootstrap or delta-method calculation to achieve comparable propagation.

Sometimes we want to calculate a new quantity from several TLS estimates at once. For example, we might want a contrast, a ratio, or an index that uses z, $CT_{max_{1hr}}$ and T_{crit} together. The important thing is to keep values from the same posterior draw together. `get_tls_est(., "draws")` returns the draws in long format, with `.draw` identifying which posterior draw each value came from. We pivot that table wider so each row is one draw and the matching z, $CT_{max_{1hr}}$ and T_{crit} values sit side by side. Any calculation made row-by-row then keeps the uncertainty and correlations among these quantities intact:

```
# Start with draws in long format, then pivot wider so each row is one posterior
# draw. This keeps z, CTmax and T_crit values from the same draw together.
tl_bin <- tls(wf_bin, target_surv = "relative", t_ref = 60, lethal = TRUE, ndraws = 1000)
d_bin  <- get_tls_est(tl_bin, "draws") |>
  tidyr::pivot_wider(names_from = quantity, values_from = value) |>
  dplyr::rename(T_crit = Tcrit)
head(d_bin)
```

```
# A tibble: 6 x 4
  .draw     z CTmax T_crit
  <int> <dbl> <dbl>  <dbl>
1     1  6.80  32.2   17.2
2     2  6.74  32.1   14.7
3     3  6.67  32.1   15.1
4     4  6.71  32.1   16.4
5     5  6.61  32.1   17.8
6     6  6.56  32.2   17.9
```

```r
# The summary form is the same table shown by tls()$summary above.
get_tls_est(tl_bin, "summary")
```

```
# A tibble: 3 x 4
  quantity median lower upper
  <chr>     <dbl> <dbl> <dbl>
1 z          6.69  6.56  6.83
2 CTmax     32.1  32.1  32.2
3 Tcrit     15.4  12.2  18.6
```

```r
# z and CTmax co-vary across posterior draws.
round(stats::cor(d_bin$z, d_bin$CTmax), 2)
```

```
[1] -0.57
```

```r
# Row-by-row calculations preserve joint uncertainty. CTmax_1hr - 2.5*z is
# T_crit at the central rate (r* ~ 0.3% HI/h); the full T_crit column is wider
# because it also includes rate uncertainty.
qs <- c(0.025, 0.5, 0.975)
stats::quantile(d_bin$CTmax - 2.5 * d_bin$z, qs)   # paired z & CTmax, central rate
```

```
    2.5%      50%    97.5%
15.03464 15.42960 15.79365
```

```r
stats::quantile(d_bin$T_crit,                qs)   # + rate uncertainty -> wider
```

```
    2.5%      50%    97.5%
12.18986 15.42527 18.57926
```

> **!** Important
>
> For real data, it is important to compare binomial and beta-binomial fits rather than assuming either likelihood is adequate. When extra dispersion is present, the choice can affect uncertainty and inference.

3.0.5 *Comparing the two pipelines*

3.0.5.1 Point-estimate agreement on the same data

Table S8 puts the two-stage point estimates next to the joint Bayesian posterior medians for both simulated datasets.

Table S8: Point estimates (two-stage) and posterior medians (joint Bayesian) for thermal sensitivity (z) and $CT_{max_{1hr}}$, recovered by each of four estimator-dataset combinations from the same simulated datasets. Truth values are the simulation parameters.

Quantity	Truth	Two-stage (binomial)	Joint Bayesian (binomial)	Two-stage (beta-binomial)
z (°C)	6.67	6.96	6.69	7.3
CTmax at 1 hr (°C)	32.15	32.03	32.15	31.9

```
# Compare tutorial estimates from the joint and two-stage workflows. The joint
# medians come straight off the tls() summaries derived above; match() pulls the
# z and CTmax rows by name (so the result is independent of row order).
compare_table <- tibble::tibble(
  quantity   = c("z (°C)", "CTmax at 1 hr (°C)"),
  truth      = c(true_z, true_CTmax),
  ts_bin     = c(stage2_bin$summary$z, stage2_bin$summary$CTmax_1hr),
  joint_bin  = tl_bin$summary$median[match(c("z", "CTmax"), tl_bin$summary$quantity)],
  ts_bb      = c(stage2_bb$summary$z, stage2_bb$summary$CTmax_1hr),
  joint_bb   = tl_bb$summary$median[match(c("z", "CTmax"), tl_bb$summary$quantity)]
)

compare_table |>
  dplyr::rename(
    Quantity                          = quantity,
    Truth                             = truth,
    `Two-stage (binomial)`            = ts_bin,
    `Joint Bayesian (binomial)`       = joint_bin,
    `Two-stage (beta-binomial)`       = ts_bb,
    `Joint Bayesian (beta-binomial)`  = joint_bb
  ) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

All four estimators give similar point estimates for z and $CT_{max_{1hr}}$, close to the simulation truth. Under this simple generating scenario, the joint Bayesian 4PL reproduces the classical pipeline's estimates while retaining a coherent posterior for downstream inference.

3.0.5.2 One model, two parameterisations

`fit_4pl()` can write the same model in two ways. The default **midpoint** version estimates how the curve midpoint changes with temperature, then `tls()` converts that slope into z and CT_{max}. The **direct** version estimates z and CT_{max} inside the model itself; request it with `ctmax = ~ 1` and `z = ~ 1`. If the same data are fit with the same likelihood, the two versions should give the same z and $CT_{max_{1hr}}$ estimates.

```
# Refit the binomial tutorial data with the DIRECT (z, CTmax) parameterisation.
# Same data, same family -- only the way the midpoint is written changes.
```

```r
# Cached like the fits above; the .rds is created on first render.
wf_direct <- fit_4pl(std_bin,
                     ctmax = ~ 1, z = ~ 1,
                     family = binomial(link = "identity"),
                     chains = 4, iter = 2000, cores = 4, seed = 123,
                     file       = file.path(models_dir, "sim_4pl_direct"),
                     file_refit = "never")

# Summarise the direct model, where z and CTmax are modelled explicitly.
summary(wf_direct)
```

```
 Family: binomial
  Links: mu = identity
Formula: n_surv | trials(n_total) ~ low + (up - low)/(1 + exp(exp(logk) * (logd - mid)))
         low ~ (0.001 + inv_logit(lowraw) * 0.498)
         up ~ (0.501 + inv_logit(upraw) * 0.498)
         mid ~ (1.77815 - (temp_c - CTmaxdev)/exp(logz))
         lowraw ~ temp_c
         upraw ~ temp_c
         logk ~ temp_c
         CTmaxdev ~ 1
         logz ~ 1
   Data: data (Number of observations: 900)
  Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
         total post-warmup draws = 4000

Regression Coefficients:
                  Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
lowraw_Intercept      -2.86      0.09    -3.04    -2.69 1.00     2655     2766
lowraw_temp_c         -0.00      0.03    -0.06     0.07 1.00     2629     2934
upraw_Intercept        2.89      0.07     2.74     3.03 1.00     2861     2904
upraw_temp_c           0.01      0.03    -0.04     0.06 1.00     2769     2799
logk_Intercept         2.06      0.02     2.01     2.11 1.00     3050     2968
logk_temp_c            0.00      0.01    -0.01     0.02 1.00     3072     3072
CTmaxdev_Intercept    -1.87      0.03    -1.94    -1.81 1.00     2695     2875
logz_Intercept         1.89      0.01     1.87     1.91 1.00     2788     2836

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

A single `tls()` call turns either fit into a tidy table of the derived quantities — one row per quantity, with the posterior median and 95% interval. It is the one-call extractor: pass it *any* fitted 4PL and it returns z, $CT_{max_{1hr}}$ and (for a lethal endpoint) T_{crit}, whatever the parameterisation.

Table S9: Posterior medians and 95% credible intervals for z and $CT_{max_{1hr}}$ recovered from the same simulated (binomial) data fit two ways — the default midpoint parameterisation and the direct (z, CT_{max}) parameterisation — both extracted with the single-call `tls()`. Truth values are the simulation parameters.

Quantity	Truth	Midpoint (default)	Direct (z, CTmax)
z (°C)	6.67	6.69 [6.56, 6.83]	6.64 [6.51, 6.77]
CTmax at 1 hr (°C)	32.15	32.15 [32.08, 32.21]	32.13 [32.06, 32.19]

```
# One call per fit -- no separate extract step. target_surv picks the threshold,
# t_ref sets the reference duration (minutes).
tls_mid <- tls(wf_bin,    target_surv = "relative", t_ref = 60)
tls_dir <- tls(wf_direct, target_surv = "relative", t_ref = 60)

# What a single tls() call returns for the direct fit:
get_tls_est(tls_dir, what = "summary", params = c("z", "CTmax"))
```

```
# A tibble: 2 x 4
  quantity median lower upper
  <chr>     <dbl> <dbl> <dbl>
1 z          6.64  6.51  6.77
2 CTmax     32.1  32.1  32.2
```

Table S9 lays the two parameterisations side by side, reading z and $CT_{max_{1hr}}$ straight off each `tls()` summary.

```
# Pull the z and CTmax rows from each tls() summary via the accessor (params=
# selects quantities; rows keep their z, CTmax order) and lay them side by side.
# format_interval() is vectorised.
mid <- get_tls_est(tls_mid, "summary", c("z", "CTmax"))
dir <- get_tls_est(tls_dir, "summary", c("z", "CTmax"))

param_equiv <- tibble::tibble(
  Quantity             = c("z (°C)", "CTmax at 1 hr (°C)"),
  Truth                = round(c(true_z, true_CTmax), 2),
  `Midpoint (default)` = format_interval(mid$median, mid$lower, mid$upper, digits = 2),
  `Direct (z, CTmax)`  = format_interval(dir$median, dir$lower, dir$upper, digits = 2)
)

tinytable::tt(param_equiv, digits = 3, escape = TRUE)
```

The two parameterisations recover essentially the same z and $CT_{max_{1hr}}$ — each estimate sits well within the other's credible interval (Table S9).

3.0.6 *A worked example: temperature effects on every shape parameter*

The default `fit_4pl()` places a `temp_c` slope on low, up, and k, estimating temperature effects on curve shape alongside the linear midpoint. Strictly speaking, this isn't necessary if you are interested in *relative* measures so long as they are on the midpoint. However, these are needed for *absolute* measures because of the need for the correction factor to adjust to the desired probability threshold.

Here, we test recovery when the constant-shape assumption fails, we re-simulate beta-binomial data on the design in Section 3.0.1.2 with `up` declining and k increasing sharply with T. We then apply the same `fit_4pl()` and `tls()` workflow used above.

```
# Prepare the extended simulation dataset for the package workflow example.
# Same factorial design as @sec-sim-step2; re-declared here so this chunk is
# self-contained.
temps     <- c(30, 32, 34, 36, 38)
durations <- c(1, 5, 15, 45, 135, 405)
n_rep     <- 30
n_per_rep <- 30
phi       <- 5

# Set truth on the raw parameter scale used by fit_4pl().
true_params_ext <- list(
  low_raw  = qlogis(0.05 / 0.49),                # low = 0.05
  up_raw_0 = qlogis((0.92 - 0.51) / 0.49),       # up = 0.92 at T_bar
  up_raw_T = -0.60,                               # strong decline in up with T
  log_k_0  = log(8),                              # k = 8 at T_bar
  log_k_T  = 0.30,                                # strong rise in k with T
  m_beta0  = 1.5,
  m_beta1  = -0.15,
  T_bar    = 34
)

design <- tidyr::expand_grid(
  T   = temps,
  t   = durations,
  rep = seq_len(n_rep)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - true_params_ext$T_bar,
    n       = n_per_rep
  )

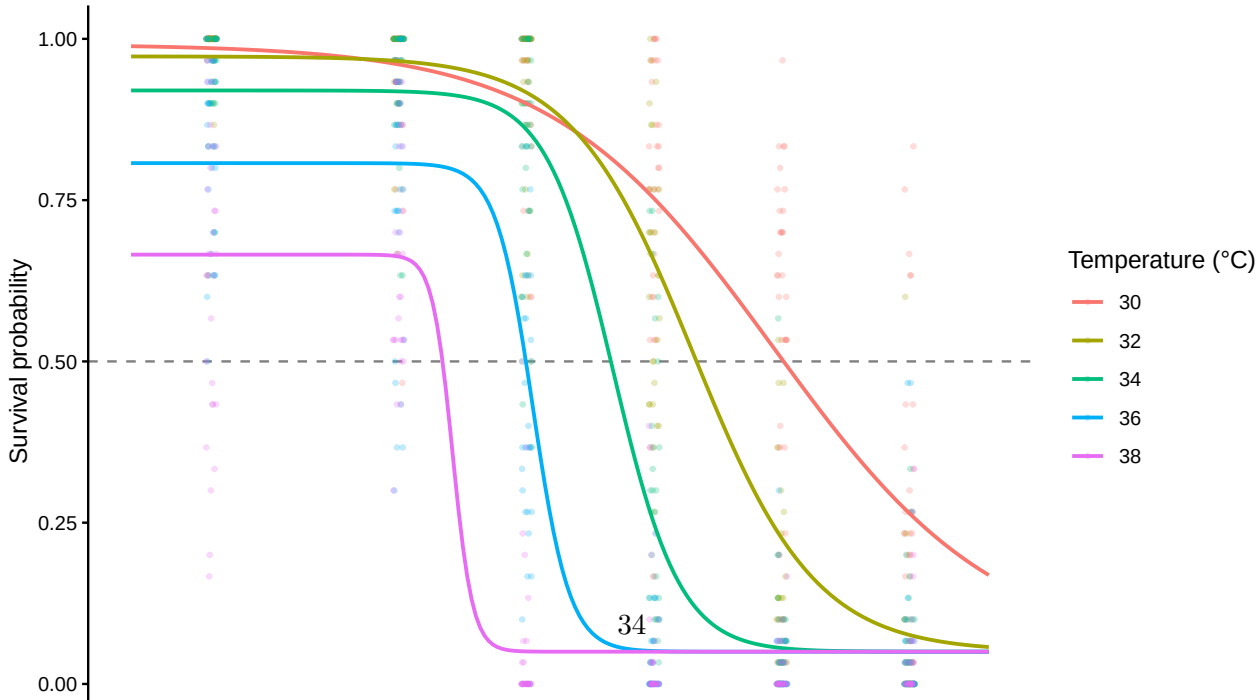
sim_bb_ext <- design |>
  dplyr::mutate(
    low    = 0.49 * plogis(true_params_ext$low_raw),
    up     = 0.51 + 0.49 * plogis(true_params_ext$up_raw_0 +
```

```
                                    true_params_ext$up_raw_T * T_c),
  k      = exp(true_params_ext$log_k_0 + true_params_ext$log_k_T * T_c),
  mid    = true_params_ext$m_beta0 + true_params_ext$m_beta1 * T_c,
  p_true = low + (up - low) / (1 + exp(k * (log10_t - mid))),
  p_draw = rbeta(dplyr::n(), p_true * phi, (1 - p_true) * phi),
  y      = rbinom(dplyr::n(), size = n, prob = p_draw)
)
```

Here the lower asymptote remains at low $= 0.05$, while `up` falls from 0.99 at $T = 30$ to 0.67 at $T = 38$, and k rises from 2.4 to 26.6. The extreme-temperature curves therefore differ in shape, not merely position (Figure S6).

```r
# Plot the true surface for the extended simulation example.
truth_grid_ext <- tidyr::expand_grid(
  T = temps,
  t = 10 ^ seq(log10(0.5), log10(800), length.out = 200)
) |>
  dplyr::mutate(
    log10_t = log10(t),
    T_c     = T - true_params_ext$T_bar,
    low     = 0.49 * plogis(true_params_ext$low_raw),
    up      = 0.51 + 0.49 * plogis(true_params_ext$up_raw_0 +
                                   true_params_ext$up_raw_T * T_c),
    k       = exp(true_params_ext$log_k_0 + true_params_ext$log_k_T * T_c),
    mid     = true_params_ext$m_beta0 + true_params_ext$m_beta1 * T_c,
    p_true  = low + (up - low) / (1 + exp(k * (log10_t - mid)))
  )

ggplot2::ggplot() +
  ggplot2::geom_hline(yintercept = 0.5, linetype = "dashed",
                      colour = "grey50") +
  ggplot2::geom_jitter(
    data = sim_bb_ext,
    ggplot2::aes(x = log10_t, y = y / n, colour = factor(T)),
    width = 0.02, height = 0, alpha = 0.25, size = 0.7
  ) +
  ggplot2::geom_line(
    data = truth_grid_ext,
    ggplot2::aes(x = log10_t, y = p_true, colour = factor(T)),
    linewidth = 0.8
  ) +
  ggplot2::labs(x = expression(log[10]~exposure~time~(min)),
                y = "Survival probability",
                colour = "Temperature (°C)") +
  ggplot2::theme_classic()
```



Before fitting, we compute the analytical targets: $\log_{10}\mathrm{LT50}(T)$, local $z(T)$ (negative reciprocal of the local slope of $\log_{10}\mathrm{LT50}$ on T), and $CT_{max_{1hr}}$ (the temperature at which $\log_{10}\mathrm{LT50}$ crosses $\log_{10}60$).

```
# Derive the true TLS quantities for the extended simulation example.
# True log10(LT50) at any T from the T-varying 4PL: solve p(log10_t) = 0.5
# for log10_t. With the 4PL,
#   log10_t = mid + (1/k) * log((up - 0.5) / (0.5 - low))
true_log10_LT50 <- function(T_value) {
  T_c <- T_value - true_params_ext$T_bar
  low <- 0.49 * plogis(true_params_ext$low_raw)
  up  <- 0.51 + 0.49 * plogis(true_params_ext$up_raw_0 +
                              true_params_ext$up_raw_T * T_c)
  k   <- exp(true_params_ext$log_k_0 + true_params_ext$log_k_T * T_c)
  mid <- true_params_ext$m_beta0 + true_params_ext$m_beta1 * T_c
  mid + (1 / k) * log((up - 0.5) / (0.5 - low))
}

# Local z(T) via central finite difference of true log10(LT50)(T). half_step
# matches the package's pointwise derivative step (h = 1e-3) for an exact
# like-for-like comparison against the value derive_z() returns.
true_local_z <- function(T_at, half_step = 1e-3) {
  -1 / ((true_log10_LT50(T_at + half_step) -
         true_log10_LT50(T_at - half_step)) / (2 * half_step))
}

# True CTmax_1hr: solve log10(LT50)(T) = log10(60) directly with uniroot()
# (a root-find, rather than a grid search, so the truth is exact).
true_CTmax_1hr <- stats::uniroot(
  function(T_value) true_log10_LT50(T_value) - log10(60),
  interval = c(20, 45)
)$root

truth_ext <- tibble::tibble(
  T               = temps,
  log10_LT50_true = vapply(temps, true_log10_LT50, numeric(1)),
  LT50_min_true   = 10 ^ log10_LT50_true,       # Inverse of log10
  local_z_true    = vapply(temps, true_local_z, numeric(1))
)

true_mean_local_z <- mean(truth_ext$local_z_true)
```

Across the assay range the true $\log_{10}\mathrm{LT50}(T)$ curve is bent: the implied local z varies from 6.05 °C at $T = 30$ to 6.53 °C at $T = 38$ (truth mean $z = 6.28$ °C). The truth implies $CT_{max_{1hr}} = 32.21$ °C. With straight-line $\log_{10}\mathrm{LT50}(T)$ — the constant-shape case — local z would not vary across temperature at all.

We now fit with exactly the same `fit_4pl()` call as before. The default formula already puts a

`temp_c` slope on each of the four 4PL sub-parameters, so the model has the capacity to recover the truth's temperature signal on `up` and k without any modification.

```
# Fit the default package 4PL to the extended simulation data.
std_ext <- standardize_data(sim_bb_ext,
                            temp = "T", duration = "t",
                            n_total = "n", n_surv = "y",
                            duration_unit = "minutes")

wf_ext <- fit_4pl(std_ext,
                  chains = 4, iter = 2000, cores = 4, seed = 123,
                  file       = file.path(models_dir,
                                         "sim_4pl_ext_betabin_DEFAULT_strong_v1"),
                  file_refit = "never")     # see fit-4pl-tutorial note above
```

Table S10 reports recovery of the four 4PL sub-parameters on their natural scale at each assay temperature. The posterior keeps low near 0.05, while recovering the decline in `up` and increase in k across the assay range.

```
# Format parameter recovery for the extended simulation example.
ext_post  <- posterior::as_draws_df(get_brmsfit(wf_ext)) |> as.data.frame()
ext_meta  <- wf_ext$meta
ext_bnds  <- ext_meta$bounds         # compute_4pl_bounds(0, 1)
ext_temp_mean <- ext_meta$temp_mean  # what the fit centred on

ext_param_long <- purrr::map_dfr(temps, function(T_value) {
  T_c_fit  <- T_value - ext_temp_mean
  T_c_true <- T_value - true_params_ext$T_bar

  draws_at_T <- tibble::tibble(
    low = ext_bnds$low_min +
          stats::plogis(ext_post$b_lowraw_Intercept +
                        ext_post$b_lowraw_temp_c * T_c_fit) * ext_bnds$low_w,
    up  = ext_bnds$up_min  +
          stats::plogis(ext_post$b_upraw_Intercept +
                        ext_post$b_upraw_temp_c * T_c_fit)  * ext_bnds$up_w,
    k   = exp(ext_post$b_logk_Intercept + ext_post$b_logk_temp_c * T_c_fit),
    mid = ext_post$b_mid_Intercept + ext_post$b_mid_temp_c * T_c_fit
  )

  truth_at_T <- tibble::tibble(
    low = 0.49 * plogis(true_params_ext$low_raw),
    up  = 0.51 + 0.49 * plogis(true_params_ext$up_raw_0 +
                               true_params_ext$up_raw_T * T_c_true),
    k   = exp(true_params_ext$log_k_0 +
              true_params_ext$log_k_T * T_c_true),
    mid = true_params_ext$m_beta0 + true_params_ext$m_beta1 * T_c_true
```

```
  )

  purrr::map_dfr(c("low", "up", "k", "mid"), function(p) {
    q <- stats::quantile(draws_at_T[[p]], c(0.025, 0.5, 0.975),
                         na.rm = TRUE, names = FALSE)
    tibble::tibble(
      Parameter         = p,
      `T (°C)`          = T_value,
      Truth             = truth_at_T[[p]],
      Median            = q[2],
      Lower             = q[1],
      Upper             = q[3]
    )
  })
})

ext_param_long |>
  tinytable::tt(digits = 3, escape = TRUE)
```

We derive the TDT quantities as in Section 3.0.4, but read at the **absolute** threshold. `tls()` inverts the fitted surface numerically per draw, so the same workflow handles the bent curve; for the per-temperature detail we call the `derive_z()` and `derive_tdt_curve()` primitives alongside it:

```
# Extract TDT quantities from the extended simulation fit under the ABSOLUTE
# threshold: with temperature effects on up/k the log10(LT50) curve is bent, so
# its local z varies with T and the asymmetry correction is evaluated per draw.
# (The relative threshold reads mid(T), which is linear and would give a constant
# z that cannot match the bent truth.)
#
# tls() returns the pooled z and CTmax. For this bent-curve deep-dive we also call
# two advanced primitives directly: derive_z() for the per-temperature local z(T),
# and derive_tdt_curve() for the smooth LT50(T) curve.
tl_ext    <- tls(wf_ext, target_surv = "absolute", t_ref = 60, time_multiplier = 1,
                 ndraws = 1000)
z_ext     <- derive_z(wf_ext, target_surv = "absolute", ndraws = 1000)
curve_ext <- derive_tdt_curve(wf_ext, temp_grid = seq(min(temps), max(temps), length.out = 60)
                              target_surv = "absolute", ndraws = 1000)
```

Table S11 compares the extracted z and $CT_{max_{1hr}}$ to the analytical truth derived above.

```
# Format extracted TDT quantities for the extended simulation fit (z and CTmax;
# T_crit is omitted here -- it is the rate-multiplier prediction demonstrated in
# the tutorial, not the focus of this bent-curve recovery check).
# tls()$summary is tidy-long (quantity, median, lower, upper); pull z and CTmax.
ext_sum    <- get_tls_est(tl_ext, "summary")
```

Table S10: Posterior recovery of the four 4PL sub-parameters on the natural scale at each assay temperature. The fit uses the disjoint-bounds reparameterisation (low, up via inv_logit, k via exp); raw posterior columns are transformed back to the biological scale before summarising. Posterior medians and 95% credible intervals are reported alongside the analytical truth.

Parameter	T (°C)	Truth	Median	Lower	Upper
low	30	0.05	0.0625	0.0293	0.1084
up	30	0.991	0.9935	0.989	0.9962
k	30	2.41	2.4043	2.1161	2.7386
mid	30	2.1	2.1065	2.0379	2.1701
low	32	0.05	0.0536	0.0311	0.0797
up	32	0.973	0.9791	0.9697	0.9863
k	32	4.39	4.5723	4.1102	5.1149
mid	32	1.8	1.8061	1.7625	1.847
low	34	0.05	0.0457	0.0328	0.0596
up	34	0.92	0.932	0.9166	0.9463
k	34	8	8.7122	7.1951	10.6985
mid	34	1.5	1.5056	1.4818	1.5292
low	36	0.05	0.0391	0.0317	0.0475
up	36	0.807	0.8168	0.7885	0.842
k	36	14.577	16.5544	12.261	22.8179
mid	36	1.2	1.2049	1.1814	1.2328
low	38	0.05	0.0335	0.0248	0.0444
up	38	0.666	0.6592	0.6201	0.6995
k	38	26.561	31.5023	20.6197	48.8663
mid	38	0.9	0.9041	0.8624	0.9533

Table S11: Classical TDT quantities from `tls(target_surv = "absolute")` on the T-varying fit, compared against analytical truth. Under the absolute threshold the single-summary z is the per-draw **local** $z(T)$ of the bent $\log_{10}\mathrm{LT50}(T)$ curve, pooled (averaged) over the assay temperatures; the truth row reports the mean of the analytical local $z(T)$ at those temperatures. $CT_{max_{1hr}}$ comes from direct inversion of the fitted 4PL at 60 min per draw, including the per-draw asymmetry correction.

Quantity	Truth	Median	Lower	Upper
z (°C)	6.28	6.18	5.74	6.84
CTmax at 1 hr (°C)	32.21	32.27	32	32.49

```
z_ext_row  <- ext_sum[ext_sum$quantity == "z", ]
ct_ext_row <- ext_sum[ext_sum$quantity == "CTmax", ]
tibble::tibble(
  Quantity     = c("z (°C)", "CTmax at 1 hr (°C)"),
  Truth        = c(round(true_mean_local_z, 3), round(true_CTmax_1hr, 3)),
  Median       = c(z_ext_row$median, ct_ext_row$median),
  Lower        = c(z_ext_row$lower,  ct_ext_row$lower),
  Upper        = c(z_ext_row$upper,  ct_ext_row$upper)
) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

Both posterior intervals cover their analytical truths. Under the absolute threshold the single-summary z is the per-draw local $z(T)$ averaged over the assay temperatures; it recovers the truth (mean local z = 6.28 °C; posterior median 6.18 °C, 95% CrI [5.74, 6.84] °C). Because the curve is bent, the *full* per-temperature local $z(T)$ profile varies across the assay range (examined below, and returned directly by `derive_z(target_surv = "absolute")$local_summary`) — unlike the relative threshold, which reads the linear mid(T) and would give a single constant z. Direct 4PL inversion at the reference duration also recovers $CT_{max_{1hr}}$ (truth 32.21 °C; posterior median 32.27 °C, 95% CrI [32, 32.49] °C), with the asymmetry correction evaluated per draw rather than assumed constant in T.

The bent $\log_{10}\mathrm{LT50}(T)$ curve itself is produced by `derive_tdt_curve()` (here `curve_ext`) and can be plotted directly (Figure S7).

```r
# Plot the extended example LT50 curve.
truth_overlay <- {
  Ts <- seq(min(temps) - 2, max(temps) + 2, by = 0.05)
  tibble::tibble(temp = Ts,
                 duration_median = 10 ^ vapply(Ts, true_log10_LT50,
                                               numeric(1)))
}

plot_tdt_curve(curve_ext) +
  ggplot2::geom_line(data = truth_overlay,
                     ggplot2::aes(x = temp, y = duration_median),
                     linetype = "dashed", colour = "black",
                     inherit.aes = FALSE)
```

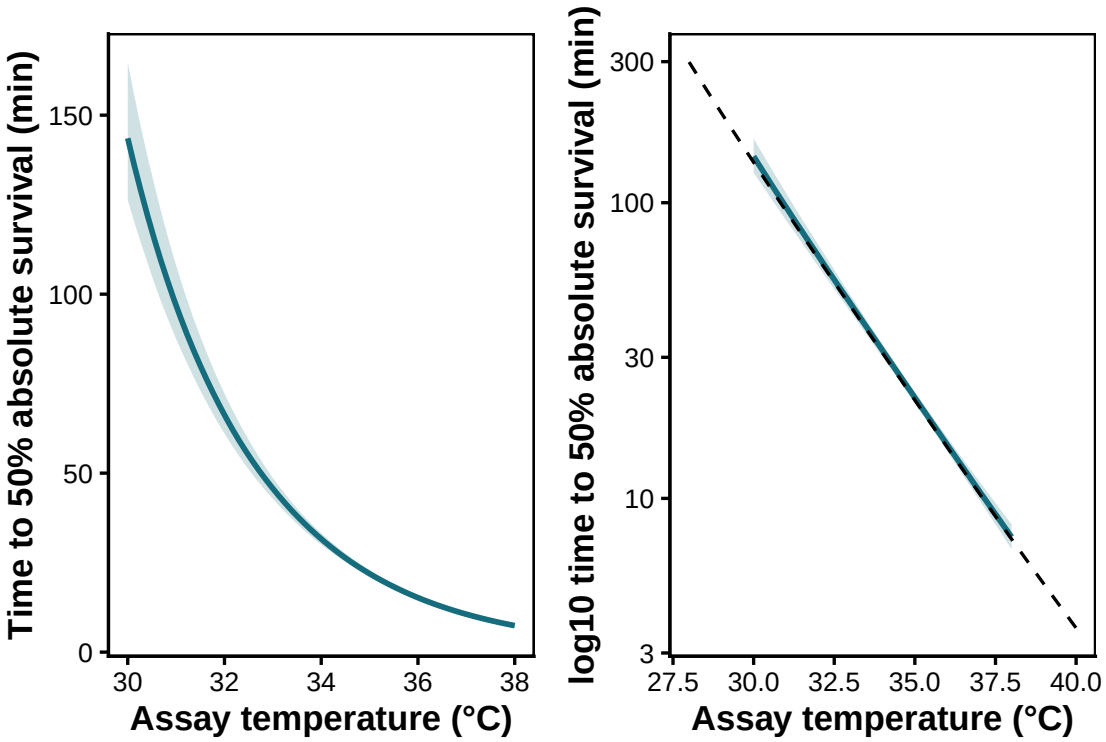


Figure S7: Posterior LT50(T) curve (median + 95% CrI, blue) recovered by `derive_tdt_curve()` from the model fit, with the analytical truth overlaid (dashed black). The curve is visibly bent rather than straight in T, and the posterior tracks the truth.

We can easily visualize variation in z with temperature. `derive_z(target_surv = "absolute")` returns the per-draw **local** $z(T)$ at each assay temperature directly, in `$local_draws` (per draw) and `$local_summary` (per temperature). We use that summary below (Figure S8).

```r
# Per-temperature local z(T) straight from derive_z(target_surv = "absolute").
# z_ext$local_summary has columns temp / z_median / z_lower / z_upper; we
# rename to the names the figure below expects and join the analytical truth.
local_z_summary <- z_ext$local_summary |>
```

T (°C)	z local truth (°C)	Median	Lower	Upper
30	6.05	5.74	5.04	6.38
32	6.22	6.12	5.72	6.6
34	6.27	6.27	5.87	6.78
36	6.34	6.32	5.89	6.88
38	6.53	6.49	6.03	7.09

```
  dplyr::transmute(T_at           = temp,
                   z_local_median = z_median,
                   z_local_lower  = z_lower,
                   z_local_upper  = z_upper)

local_z_summary |>
  dplyr::left_join(
    tibble::tibble(T_at = temps, z_local_truth = truth_ext$local_z_true),
    by = "T_at"
  ) |>
  dplyr::select(T_at, z_local_truth, z_local_median,
                z_local_lower, z_local_upper) |>
  dplyr::rename(
    `T (°C)`              = T_at,
    `z local truth (°C)`  = z_local_truth,
    Median                = z_local_median,
    Lower                 = z_local_lower,
    Upper                 = z_local_upper
  ) |>
  tinytable::tt(digits = 3, escape = TRUE)
```

```r
# Plot how local z changes across temperature in the extended example.
truth_curve_z <- {
  Ts <- seq(min(temps), max(temps), by = 0.1)
  tibble::tibble(T_at = Ts,
                 z_local_truth = vapply(Ts, true_local_z, numeric(1)))
}

ggplot2::ggplot(local_z_summary,
                ggplot2::aes(x = T_at, y = z_local_median)) +
  ggplot2::geom_ribbon(ggplot2::aes(ymin = z_local_lower,
                                    ymax = z_local_upper),
                       fill = "#146C7C", alpha = 0.2) +
  ggplot2::geom_line(colour = "#146C7C", linewidth = 1) +
  ggplot2::geom_line(data = truth_curve_z,
                     ggplot2::aes(x = T_at, y = z_local_truth),
                     linetype = "dashed", colour = "black",
                     inherit.aes = FALSE) +
  ggplot2::labs(x = "Assay temperature (°C)",
                y = expression(local~italic(z)~"(°C per 10-fold change in time)")) +
  ggplot2::theme_classic()
```

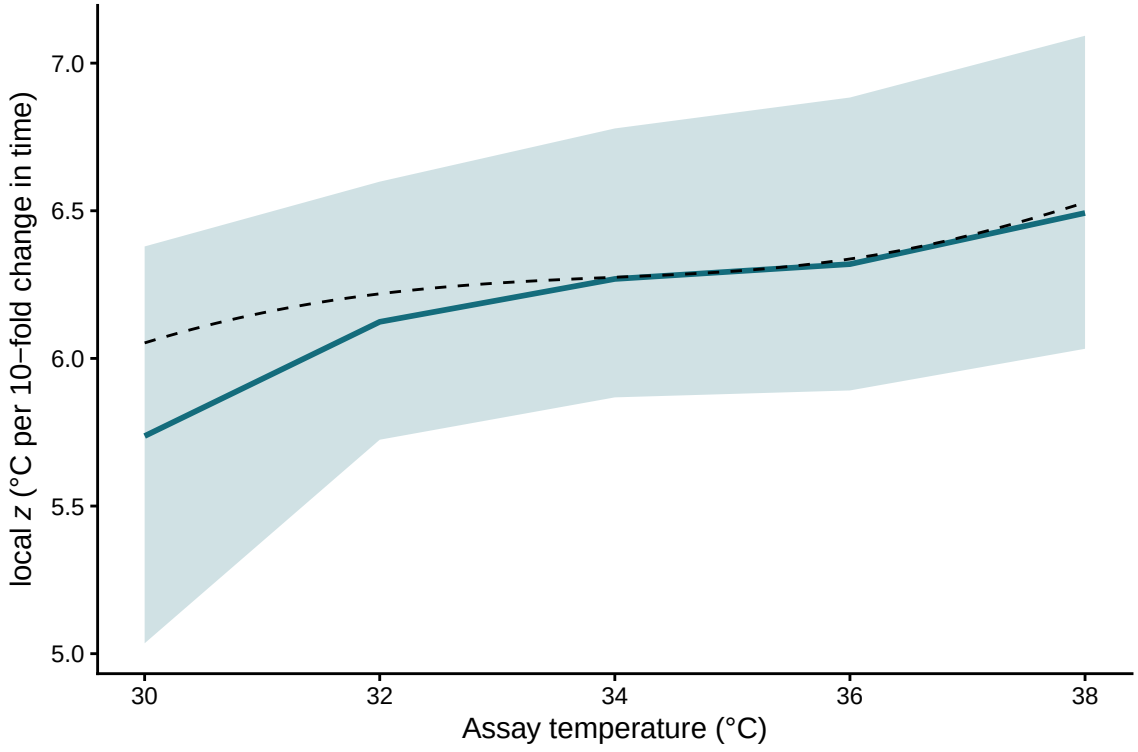


Figure S8: Local $z(T)$ recovered from the per-draw $\log_{10}\mathrm{LT50}(T)$ curve (blue median + 95% CrI), with the analytical truth overlaid (dashed black). Where the constant-shape simple-case fit would give the same z at every assay temperature, here local z varies with T in lock-step with the bent $\log_{10}\mathrm{LT50}(T)$ curve.

The same `fit_4pl()` fit, read through `tls()` and the `derive_*()` primitives, recovers the bent $\log_{10}\mathrm{LT50}(T)$ curve, the assay-averaged z, and per-draw $CT_{max_{1hr}}$. Under the constant-shape truth in Section 3.0.4, near-zero slopes on `up`, low, and k reduce this calculation to the closed-form case.

3.0.7 Heat injury accumulation and survival under a fluctuating trace

TLS models can translate dynamic field temperatures into predicted heat-injury (HI) accumulation toward a threshold such as LT_{50} (Arnold *et al.* 2025; Jørgensen *et al.* 2021; Noble *et al.* 2026; Ørsted *et al.* 2022; Rezende *et al.* 2020). The joint 4PL posterior propagates uncertainty in z and $CT_{max_{1hr}}$ through the HI integral and uses the fitted surface to predict survival fraction $S_{\mathrm{pred}}(t)$ along the same trajectory (the predicted-survival relation, Equations 9–10 of the manuscript). `predict_heat_injury()` returns both trajectories in one call.

3.0.7.1 A simulated temperature profile across 5 days

To mimic conditions for a moderately heat-tolerant ectotherm, we generate 5 days of hourly temperatures with a sinusoidal diurnal cycle, ranging from 12 °C overnight to a midday peak of 24 °C. We set the net-accumulation threshold T_c to 20 °C, below which HI does not increase (Jørgensen *et al.* 2021; Ørsted *et al.* 2022). The simulated organism has $CT_{max_{1hr}} \approx 32.1$ °C and $z \approx 6.7$ °C, so the daily peak remains below the 1-hour critical limit, yet damage accrues at every hour above T_c and the integrated dose ultimately proves lethal.

```
# Create example temperature traces for heat-injury prediction.
hi_trace <- tibble::tibble(
  time = 0:(24 * 5 - 1),
  temp   = 18 + 6 * sin(2 * pi * (0:(24 * 5 - 1) - 6) / 24)   # range 12-24 °C
)
hi_T_c <- 20    # damage threshold (°C)
```

```r
# Plot the example temperature traces used for heat-injury prediction.
ggplot2::ggplot(hi_trace, ggplot2::aes(time, temp)) +
  ggplot2::geom_line(colour = "darkred", linewidth = 0.6) +
  ggplot2::geom_hline(yintercept = hi_T_c, linetype = "dashed", colour = "grey40") +
  ggplot2::geom_hline(yintercept = true_CTmax, linetype = "dotted", colour = "grey40") +
  ggplot2::annotate("text", x = max(hi_trace$time), y = hi_T_c, label = "T_c",
                    hjust = 1, vjust = -0.4, size = 3, colour = "grey40") +
  ggplot2::annotate("text", x = max(hi_trace$time), y = true_CTmax,
                    label = "CTmax_1hr",
                    hjust = 1, vjust = -0.4, size = 3, colour = "grey40") +
  ggplot2::ylim(10, 45) +
  ggplot2::labs(x = "Hour", y = "Temperature (°C)") +
  ggplot2::theme_classic()
```

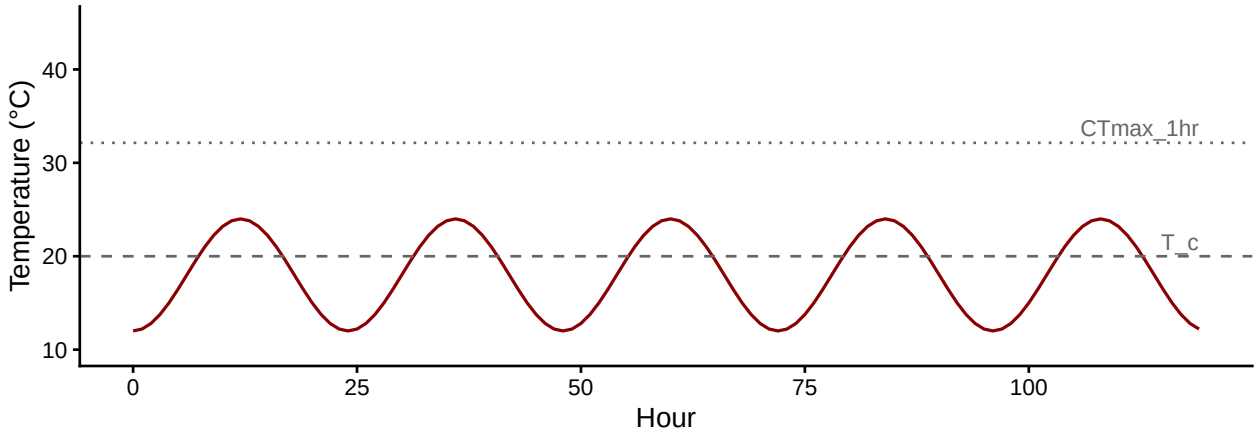


Figure S9: 5-day diurnal temperature trace. The dashed line is the damage-accumulation threshold $T_c = 20$ °C; the dotted line is the simulated organism's $CT_{max_{1hr}} \approx 32.1$°C — well above the daily peaks, yet the dose integrated over many sub-critical hours is ultimately lethal (Figure S10).

3.0.7.2 Posterior HI and survival via `predict_heat_injury()`

`predict_heat_injury()` takes a temperature trace plus a fitted workflow, propagates the full posterior of $(\text{low}, \text{up}, k, \beta_0, \beta_1)$ through the HI integral, and returns the median and 95% credible band of cumulative HI and predicted survival at every time point. We supply `T_c` to enforce zero damage accumulation below the threshold (matching the heat-injury integral, Equation 8 of the manuscript). The model time units here are minutes (because `standardize_data()` was called with `duration_unit = "minutes"`), so the trace's `time` is in hours and damage rates are computed accordingly internally.

```r
# Predict heat injury and survival for the example temperature traces.
# The trace's `time` is in hours; `predict_heat_injury()` reconciles it against
# the model's own time unit (minutes here) internally, so we pass `trace_unit`
# rather than rescaling the trace by hand.
hi <- predict_heat_injury(
  trace        = hi_trace,
```

```
  workflow     = wf_bb,
  target_surv  = 0.5,
  T_c          = hi_T_c,
  trace_unit   = "hours",
  ndraws       = 500
)
```

```
# Plot heat-injury and survival trajectories for the example traces.
# `plot_heat_injury()` returns the package's canonical two-panel figure:
# cumulative HI on top (with the 100% = one-LT50 reference line) over the
# predicted survival fraction, both with 95% credible bands.
plot_heat_injury(hi)
```

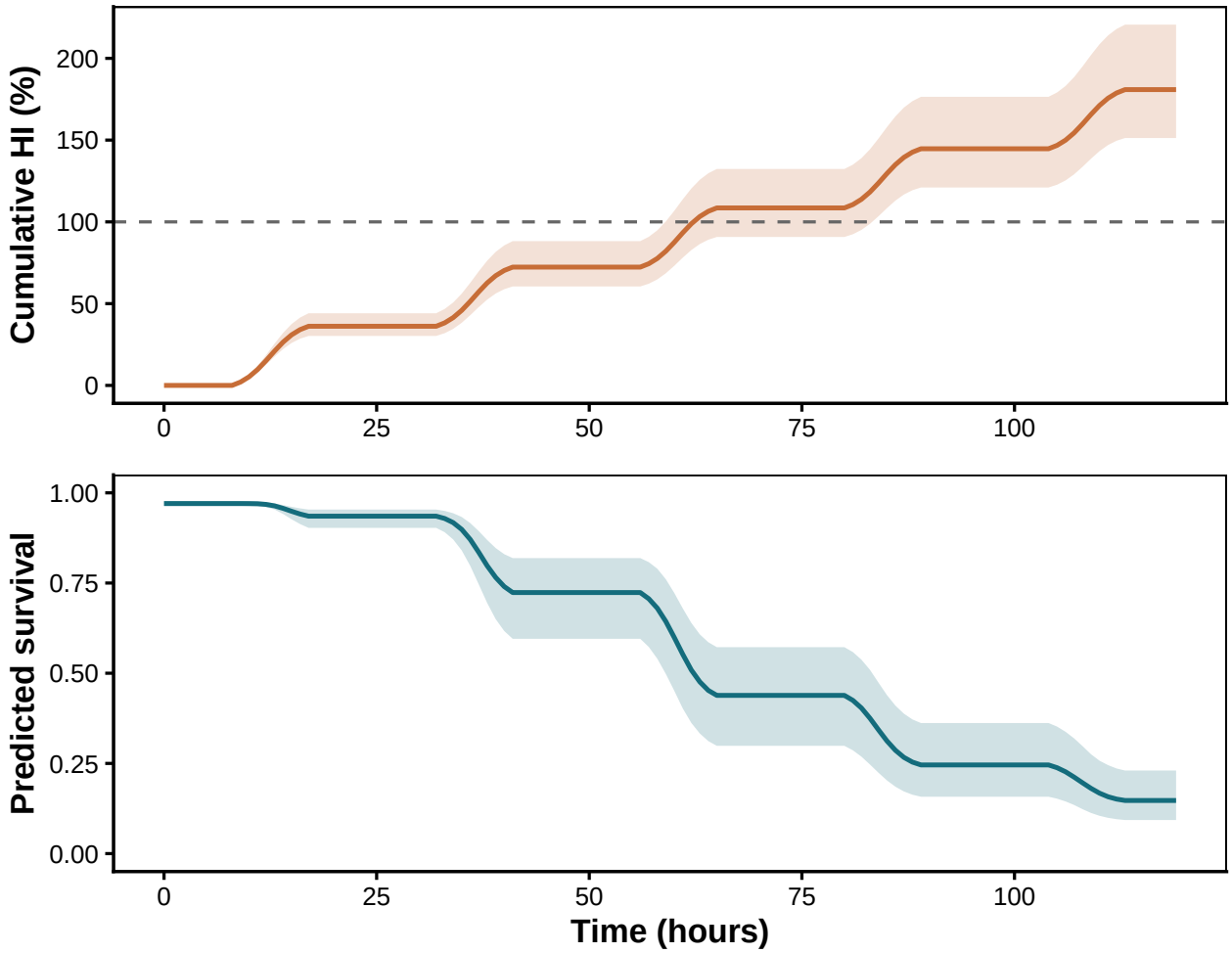


Figure S10: Posterior heat-injury accumulation (top) and predicted survival fraction (bottom) under the 5-day field trace, drawn by `plot_heat_injury()`. Solid lines are posterior medians across 500 draws from `wf_bb`; shaded bands are 95% credible intervals. The dashed line marks the LT50 threshold (HI = 100%). The two trajectories tell the same story: $S_{\text{pred}}(t)$ falls as HI(t) crosses 100%.

By the end of day 5 the posterior median HI is 181% (95% CI: 151% to 221%), and median predicted survival is 0.15 (95% CI: 0.09 to 0.23).

3.0.8 Validation: recovering known doses

Before trusting `predict_heat_injury()` on real field data, we validate it against analytical truth on three reference traces with known thermal loads. `make_temperature_scenarios()` builds the traces; `planted_dose_from_trace()` implements the classical HI integral (Equation 8 of the manuscript) directly given known `z` and `CTmax_1hr`. Posterior medians from `predict_heat_injury()` should sit near the analytical truth, and the 95% credible intervals should cover it.

```
# Validate heat-injury predictions against an analytical planted-dose calculation.
# Build validation traces with known planted doses.
val_scens <- make_temperature_scenarios(
  baseline    = 20,
  spike_temp  = 28,
  n_hours     = 96,
  spike_times_single = 24,
  spike_times_multi  = c(24, 48, 72)
)
val_T_c <- 24

# Compute analytical truth from the generating parameters.
planted <- purrr::map(val_scens, planted_dose_from_trace,
                      z = true_z, CTmax_1hr = true_CTmax, T_c = val_T_c)

# Predict the same doses from the fitted beta-binomial workflow.
predicted <- purrr::map(val_scens, function(sc) {
  predict_heat_injury(
    trace       = sc,
    workflow    = wf_bb,
    target_surv = 0.5,
    T_c         = val_T_c,
    trace_unit  = "hours",
    ndraws      = 300
  )
})

# Compare planted and posterior-predicted final heat injury.
val_table <- tibble::tibble(
  Scenario     = c("Flat", "Single spike", "Multi spike"),
  `Planted HI (%)` = c(tail(planted$flat$hi_cumulative,         1),
                       tail(planted$single_spike$hi_cumulative, 1),
                       tail(planted$multi_spike$hi_cumulative,  1)),
  `Posterior median (%)` = c(tail(predicted$flat$summary$hi_median,        1),
                             tail(predicted$single_spike$summary$hi_median, 1),
```

Scenario	Planted HI (%)	Posterior median (%)	Lower (%)	Upper (%)	Truth in CrI?
Flat	0	0	0	0	TRUE
Single spike	24	23	21	26	TRUE
Multi spike	72	70	62	78	TRUE

```
                                tail(predicted$multi_spike$summary$hi_median,  1)),
  `Lower (%)` = c(tail(predicted$flat$summary$hi_lower,        1),
                  tail(predicted$single_spike$summary$hi_lower, 1),
                  tail(predicted$multi_spike$summary$hi_lower,  1)),
  `Upper (%)` = c(tail(predicted$flat$summary$hi_upper,        1),
                  tail(predicted$single_spike$summary$hi_upper, 1),
                  tail(predicted$multi_spike$summary$hi_upper,  1))
)
val_table$`Truth in CrI?` <- with(val_table,
                                  `Lower (%)` <= `Planted HI (%)` &
                                  `Planted HI (%)` <= `Upper (%)`)
tinytable::tt(val_table, digits = 2, escape = TRUE)
```

The flat trace stays below T_c for its entire duration and so accrues zero HI, matching the analytical truth exactly. The single- and multi-spike traces deliver known doses analytically; `predict_heat_injury()` recovers each with the truth lying inside its credible interval.

4 Empirical model specifications used in the manuscript

The manuscript case studies all use the same 4PL response surface, but we show how sub-lethal meausures can be fit using variant models (Section 9). As discussed, there are two ways to parameterise the 4PL model: indirect and direct. In the indirect approach we derive estimates from the **midpoint**. As such, temperature enters the midpoint (and possibly all other parameters) directly:

$$\mathrm{mid}_{ij} = \beta_0 + \beta_1 (T_{ij} - \bar{T}) + \mathbf{x}_{ij}^{\top}\gamma + b_j, \tag{S5}$$

so relative-threshold z and CT_{max} are derived from the fitted midpoint line. In the **direct** parameterisation, the same midpoint is written in terms of CT_{max} and z:

$$\begin{aligned} \mathrm{mid}_{ij}(T) &= \log_{10} t_{\mathrm{ref}} + \frac{CT_{max,ij}(t_{\mathrm{ref}}) - T_{ij}}{z_{ij}}, \\ CT_{max,ij}(t_{\mathrm{ref}}) &= \alpha_0 + \mathbf{x}_{ij}^{\top}\alpha + b_j, \\ \ln z_{ij} &= \zeta_0 + \mathbf{x}_{ij}^{\top}\zeta + u_j. \end{aligned} \tag{S6}$$

Here $\mathbf{x}_{ij}$ contains fixed covariates such as species, oxygen treatment, recovery condition or sex, while b_j and u_j denote optional group-level effects. Direct fits place random effects inside the relevant `ctmax =` or `z =` formula; midpoint fits place them on `mid` or through an explicit `mid =` formula.

The benefit of direct parameterisation is that random effect and fixed effects are interpreted for z and CT_{max} whereas it would be interpreted on the midpoint otherwise. In both parameterisations, `bayesTLS` constrains the lower and upper asymptotes to disjoint intervals and estimates the steepness parameter on a positive scale. Temperature effects can also be included on the asymptotes and steepness where the data support them in case users want an absolute measure; however, the model can be simplified if all one wants is the relative z and CT_{max}.

We fit the empirical examples below with the Bayesian `bayesTLS` implementation through `brms` and Stan because posterior draws make uncertainty propagation through `tls()` and `predict_heat_injury()` straightforward. The same likelihoods can also be fit in a frequentist likelihood framework with `freqTLS`; for the same data and parameterisation these fits should give similar point estimates and comparable confidence intervals.

The manuscript examples use the following model specifications:

- **Cereal aphids**: lethal survival counts were fit with a beta-binomial 4PL, placing species directly on $CT_{max_{1hr}}$ and z (`ctmax = ~ 0 + species`, `z = ~ 0 + species`). No random effects were included. The fitted species-specific posterior was then projected under the unmodified Wuhan May-June 2016 air-temperature trace with `predict_heat_injury(by = "species")`.
- **Zebrafish oxygen experiment**: lethal survival counts were fit with a beta-binomial 4PL, placing oxygen treatment directly on $CT_{max_{1hr}}$ and z (`ctmax = ~ 0 + oxygen`, `z = ~ 0 + oxygen`). The lower asymptote was allowed to differ by oxygen treatment but not temperature (`low = ~ 0 + oxygen`), while the upper asymptote and steepness were shared (`up = ~ 1`, `k = ~ 1`). No random effects were included. Because the design lacks a benign-temperature control for the hyperoxia treatment, we report z and CT_{max} but not T_{crit} (`lethal = FALSE`).
- **Snow gum PSII**: retained photosystem-II function was fit as a continuous proportion with a Beta likelihood. Recovery treatment was placed directly on $CT_{max_{1hr}}$ and z, with a plant-level random intercept on CT_{max} (`ctmax = ~ 0 + recovery + (1 | plant)`, `z = ~ 0 + recovery`). Because this endpoint is sublethal, we report z and CT_{max} but not T_{crit}.
- **Vinegar fly mortality**: mortality counts were fit with beta-binomial 4PLs. We fit separate sex-specific models for comparison with Ørsted *et al.* (2024) and a joint midpoint model with sex and its temperature interaction on `mid` (`mid = ~ sex * temp_c`) to test sex differences. We also compare these with a direct parameterisation. The separate-fit comparison with Ørsted *et al.* (2024) uses the absolute LT50 threshold and a 4-hour reference exposure; the joint-model summary uses the relative midpoint threshold (also at a 4-hour reference to be more comparable to the study). The heat-injury forecasts use the relative threshold from the sex-specific lethal fits and the shaded Rennes microclimate trace from the Ørsted *et al.* (2024) NicheMapR (Kearney & Porter 2017) workflow.

The exact formulas, priors, sampler settings and diagnostics are shown in the code chunks for each case study. Note that we suppress the intercept in these models (`~ 0 + group`) because it makes the priors easier to set: this coding gives each group its own CT_{max} and z, so we can apply the same weakly-informative prior to every group. The alternative (`~ 1 + group`) instead estimates one reference group's values plus the *differences* of the other groups from it, which would place a different prior on the reference group than on the rest. This is a parameterisation choice, not a requirement for identifiability — `tls()` recovers the same per-group quantities under either coding. The direct zebrafish and aphid examples use the package's default four-chain sampler settings; the

snow gum, vinegar-fly and sublethal-extension models use the longer or more conservative settings annotated beside each fit.

5 Case Study 1: Zebrafish heat tolerance across an oxygen gradient

Saruhashi *et al.* (2026) tested the oxygen- and capacity-limited thermal tolerance (OCLTT) hypothesis — that oxygen availability sets the upper thermal limit — by scoring survival of zebrafish (*Danio rerio*) larvae across assay temperatures (26 °C controls plus 38–40 °C) and exposure durations (3.8–240 min) under different oxygen treatments. We take the diploid larvae and compare **normoxia** with **hyperoxia**, letting both the critical temperature (CT_{max}) and the thermal sensitivity (z) depend on oxygen inside one joint 4PL. The 26 °C control rows are retained for normoxia, but hyperoxia was only assayed at stressful temperatures (38–40 °C), so prediction plots below show only observed treatment-temperature combinations. While this is a **sparse** experiemntal design, we show that the 4PL can still be used to recover tolerance and sensitivity where it would be challenging with a classical two-stage analysis.

5.1 Data and standardisation

```
# Bundled Saruhashi et al. (2026) survival data; diploid larvae, normoxia vs
# hyperoxia (we drop hypoxia, whose z is weakly identified by these durations).
data(zebrafish_o2)
zf_raw <- zebrafish_o2 |>
  dplyr::filter(ploidy == "diploid", oxygen %in% c("normoxia", "hyperoxia")) |>
  dplyr::mutate(oxygen = droplevels(oxygen))

# Counts -> the bayesTLS modelling schema. `oxygen` is carried through as the moderator.
zf_std <- standardize_data(zf_raw, temp = "temp", duration = "duration_min",
                           n_total = "n_total", n_surv = "n_surv",
                           duration_unit = "minutes")
str(zf_std)
```

```
tibble [380 x 15] (S3: tbl_df/tbl/data.frame)
 $ cohort       : chr [1:380] "ZB100" "ZB100" "ZB100" "ZB100" ...
 $ ploidy       : Factor w/ 2 levels "diploid","triploid": 1 1 1 1 1 1 1 1 1 1 ...
 $ oxygen       : Factor w/ 2 levels "normoxia","hyperoxia": 1 1 1 1 1 1 1 1 1 1 ...
 $ o2_nominal   : int [1:380] 100 100 100 100 100 100 100 100 100 100 ...
 $ o2_measured  : num [1:380] 96 105.4 96 100 97.8 ...
 $ temp         : num [1:380] 26 38 26 38 26 38 26 38 26 38 ...
 $ temp_measured: num [1:380] 26.3 37.7 26.3 38.2 26.4 38.3 26.4 37.8 26.4 37.8 ...
 $ duration_min : num [1:380] 50 50 50 50 50 50 89 89 89 89 ...
 $ n_total      : int [1:380] 5 4 4 4 4 4 5 5 6 6 ...
 $ n_surv       : num [1:380] 5 4 4 4 4 4 5 5 6 6 ...
 $ duration     : num [1:380] 50 50 50 50 50 50 89 89 89 89 ...
 $ logd         : num [1:380] 1.7 1.7 1.7 1.7 1.7 ...
```

```
 $ n_dead       : num [1:380] 0 0 0 0 0 0 0 0 0 0 ...
 $ survival     : num [1:380] 1 1 1 1 1 1 1 1 1 1 ...
 $ temp_c       : num [1:380] -8.58 3.42 -8.58 3.42 -8.58 ...
 - attr(*, "tdt_meta")=List of 6
  ..$ temp_mean     : num 34.6
  ..$ duration_unit : chr "minutes"
  ..$ random_effects: NULL
  ..$ response_type : chr "count"
  ..$ response_var  : NULL
  ..$ proportion_eps: NULL
```

5.2 Joint 4PL with oxygen on CT_{max} and z

We fit CT_{max} and z directly as functions of `oxygen` using cell-means coding (`~ 0 + oxygen`), so each treatment gets its own coefficient. Because hyperoxia lacks a 26 °C control and the lower asymptote was otherwise weakly identified, we simplify the shape model: `low` differs by oxygen treatment but has no temperature slope, while `up` and k are shared. This keeps the dose-response floor from drifting with temperature in a biologically implausible way.

```
wf_zf <- fit_4pl(zf_std,
                 ctmax = ~ 0 + oxygen,
                     z = ~ 0 + oxygen,
                   low = ~ 0 + oxygen,
                    up = ~ 1,
                     k = ~ 1,
                t_ref = 60,
                control    = list(adapt_delta = 0.95),
                file_refit = "on_change",
                file       = file.path(models_dir, "zf_oxygen_normhyper"))
summary(wf_zf)
```

```
 Family: beta_binomial
  Links: mu = identity
Formula: n_surv | trials(n_total) ~ low + (up - low)/(1 + exp(exp(logk) * (logd - mid)))
         low ~ (0.001 + inv_logit(lowraw) * 0.498)
         up ~ (0.501 + inv_logit(upraw) * 0.498)
         mid ~ (1.77815 - (temp_c - CTmaxdev)/exp(logz))
         lowraw ~ 0 + oxygen
         upraw ~ 1
         logk ~ 1
         CTmaxdev ~ 0 + oxygen
         logz ~ 0 + oxygen
   Data: data (Number of observations: 380)
  Draws: 4 chains, each with iter = 4000; warmup = 2000; thin = 1;
         total post-warmup draws = 8000
```

```
Regression Coefficients:
                         Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS
lowraw_oxygennormoxia       -0.56      0.82    -2.88     0.57 1.00     3094
lowraw_oxygenhyperoxia      -3.23      0.96    -5.14    -1.38 1.00     7947
upraw_Intercept              4.66      0.68     3.43     6.11 1.00     8353
logk_Intercept               2.37      0.26     1.77     2.80 1.00     2920
CTmaxdev_oxygennormoxia      4.33      0.21     3.98     4.82 1.00     3759
CTmaxdev_oxygenhyperoxia     4.77      0.08     4.60     4.92 1.00     4721
logz_oxygennormoxia          1.73      0.17     1.38     2.06 1.00     6486
logz_oxygenhyperoxia         0.91      0.09     0.72     1.08 1.00     4454
                         Tail_ESS
lowraw_oxygennormoxia        1948
lowraw_oxygenhyperoxia       5801
upraw_Intercept              5393
logk_Intercept               2133
CTmaxdev_oxygennormoxia      2880
CTmaxdev_oxygenhyperoxia     4007
logz_oxygennormoxia          5560
logz_oxygenhyperoxia         2739

Further Distributional Parameters:
    Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
phi     0.58      0.10     0.41     0.79 1.00     7793     5602

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

5.3 Per-treatment thermal limits and OCLTT check

`tls(by = "oxygen")` derives z and $CT_{max_{1hr}}$ for each treatment from the shared posterior, so each quantity carries full uncertainty. We set `lethal = FALSE` and do not report T_{crit} here because the design has too few temperatures to estimate it correctly.

```
zf_tls <- tls(wf_zf, by = "oxygen", lethal = FALSE)
zf_tls$summary
```

```
# A tibble: 4 x 5
  oxygen    quantity median lower upper
  <fct>     <chr>     <dbl> <dbl> <dbl>
1 normoxia  z          5.63  3.99  7.87
2 normoxia  CTmax     38.9  38.6  39.4
3 hyperoxia z          2.49  2.06  2.93
4 hyperoxia CTmax     39.4  39.2  39.5
```

The joint fit recasts Saruhashi *et al.* (2026)'s OCLTT result as two linked components of the thermal tolerance landscape. Hyperoxia raises $CT_{max_{1hr}}$ to 39.4 °C, above the normoxic 38.9 °C, confirming

that hyperoxic water shifts the thermal limit upward. The model also estimates z from the same response surface, asking a second biological question: does oxygen change how quickly tolerance time declines once temperatures are stressful? This distinction is useful because Saruhashi *et al.* (2026) conclude that oxygen effects depend on stress intensity and exposure duration. In these data, the strongest support is for the upward shift in $CT_{max_{1hr}}$ under hyperoxia, while the normoxic z is less precise because survival plateaus across much of the tested duration range. The 4PL therefore does more than confirm that hyperoxia improves heat tolerance; it identifies which part of the OCLTT signal is well supported by the data (a higher thermal limit) and which part would require more duration information to estimate sharply (oxygen differences in thermal sensitivity).

```
psc_zf <- predict_survival_curves(wf_zf)
# predict_survival_curves() crosses the pooled assay temperatures with both
# oxygen levels, so it also predicts hyperoxia at 26 °C (never tested there).
# Keep only the (oxygen, temperature) combinations actually assayed, then draw
# the full fitted curves (clip_to_observed = FALSE) for both treatments.
psc_zf$summary <- dplyr::semi_join(
  psc_zf$summary, dplyr::distinct(zf_std, oxygen, temp), by = c("oxygen", "temp"))
plot_survival_curves(psc_zf, observed = zf_std, log_time = TRUE,
                     clip_to_observed = FALSE)
```

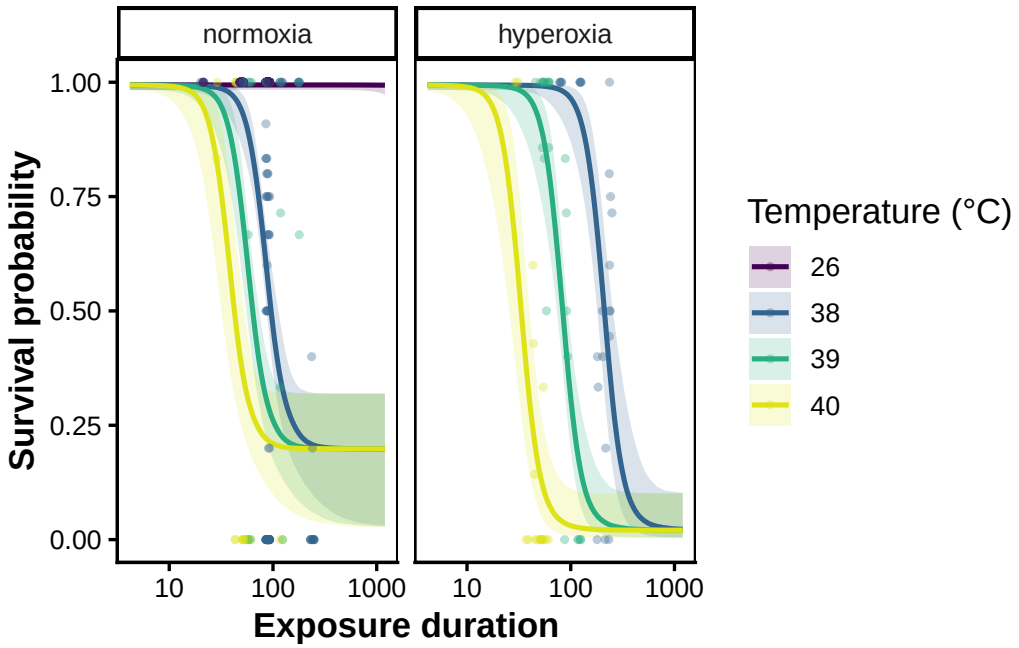


Figure S11: Zebrafish survival surface. Fitted joint-4PL survival probability against exposure duration for normoxia and hyperoxia (posterior median and 95% credible band), with the observed cell proportions overlaid. Curves are shown only for treatment-temperature combinations present in the data; hyperoxia was not assayed at 26 °C.

We can also look at the whole thermal tolerance landscape — the posterior survival surface across temperature *and* exposure duration — for each oxygen level:

```r
# Dense 2-D survival surface over temperature x duration. We show both oxygen
# treatments over a common 36-40 °C window (normoxia was also assayed at a benign
# 26 °C, far from the lethal transition); the 26 °C points are dropped from the
# overlay so the two panels are directly comparable. ndraws subsamples the
# posterior to keep the grid quick to render.
zf_land <- derive_tdt_landscape(wf_zf, ndraws = 500, temp_grid = seq(36, 40, length.out = 80))
plot_tdt_landscape(zf_land, observed = dplyr::filter(zf_std, temp >= 36),
                   clip_to_observed = FALSE)
```

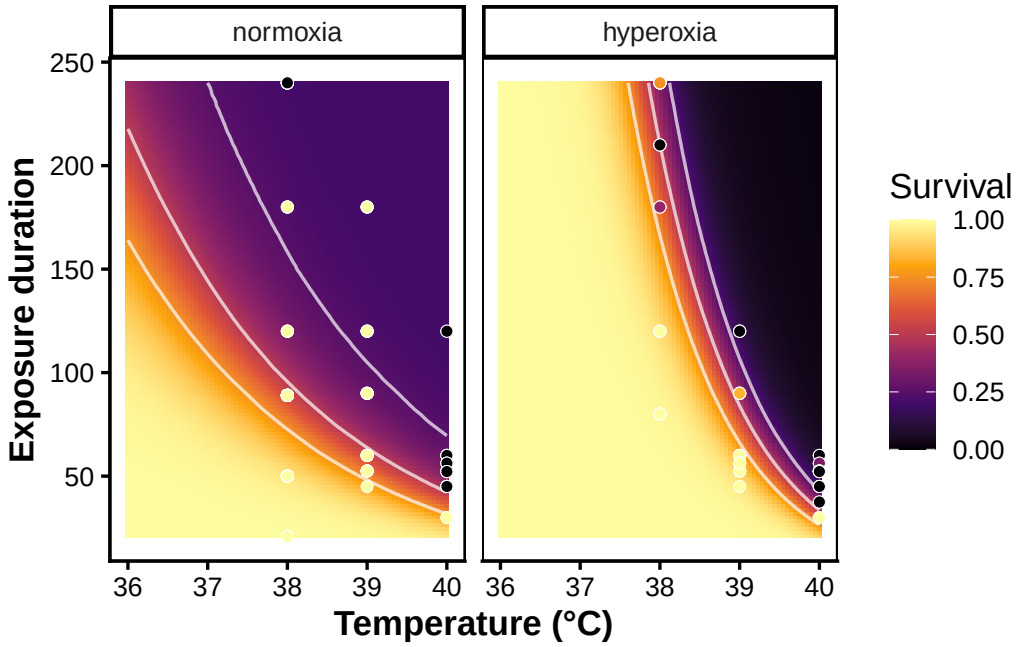


Figure S12: Zebrafish thermal tolerance landscape. Posterior-median survival probability across the temperature × exposure-duration plane for normoxia and hyperoxia. White contours mark the 25%, 50% and 75% survival isoclines, and observed cell proportions are overlaid (points). The rightward shift of the contours under hyperoxia is the higher $CT_{max_{1hr}}$ seen in the survival curves. Both panels use a common 36–40 °C window (the lethal transition); normoxia was additionally assayed at a benign 26 °C where all larvae survive, omitted here for comparability.

6 Case Study 2: Cereal aphids — comparing three species

Li *et al.* (2023) measured survival of three cereal-aphid species — *Metopolophium dirhodum*, *Sitobion avenae* and *Rhopalosiphum padi* — across temperatures and exposure durations, and linked their thermal tolerance landscapes to field abundance. We compare the three species at a single age (6 days), modelling CT_{max} and z as functions of species in one joint 4PL. This case study shows how to do a **three-group** comparison, extends the model to multiple predictors, validates the fit against Li et al.'s published estimates, and demonstrates how to use `predict_heat_injury()` across multiple groups under a real field temperature trace.

6.1 Data and standardisation

```
# Bundled Li et al. (2023) survival data; heat branch, 6-day-old aphids.
data(aphid_tdt)
aphid_raw <- aphid_tdt |>
  dplyr::filter(branch == "heat", age == "6") |>
  dplyr::mutate(species = droplevels(species))

aphid_std <- standardize_data(aphid_raw, temp = "temp", duration = "duration_min",
                              n_total = "n_total", n_surv = "n_surv",
                              duration_unit = "minutes")   # keeps `species`
str(aphid_std)
```

```
tibble [436 x 12] (S3: tbl_df/tbl/data.frame)
 $ species     : Factor w/ 3 levels "M_dirhodum","S_avenae",..: 1 1 1 1 1 1 1 1 1 1 ...
 $ age         : Factor w/ 3 levels "2","6","12": 2 2 2 2 2 2 2 2 2 2 ...
 $ branch      : Factor w/ 2 levels "heat","cold": 1 1 1 1 1 1 1 1 1 1 ...
 $ temp        : num [1:436] 40 40 40 40 40 40 40 40 40 40 ...
 $ duration_min: num [1:436] 2 2 4 4 4 5 5 5 6 6 ...
 $ n_total     : int [1:436] 4 10 9 10 10 10 10 10 10 9 ...
 $ n_surv      : num [1:436] 4 9 9 10 10 8 6 5 2 6 ...
 $ duration    : num [1:436] 2 2 4 4 4 5 5 5 6 6 ...
 $ logd        : num [1:436] 0.301 0.301 0.602 0.602 0.602 ...
 $ n_dead      : num [1:436] 0 1 0 0 0 2 4 5 8 3 ...
 $ survival    : num [1:436] 1 0.9 1 1 1 ...
 $ temp_c      : num [1:436] 3.13 3.13 3.13 3.13 3.13 ...
 - attr(*, "tdt_meta")=List of 6
  ..$ temp_mean     : num 36.9
  ..$ duration_unit : chr "minutes"
  ..$ random_effects: NULL
  ..$ response_type : chr "count"
  ..$ response_var  : NULL
  ..$ proportion_eps: NULL
```

6.2 Joint 4PL with species on CT_{max} and z

```
# Species effects on z and CTmax with a temperature
# slope on k, but we constrain the asymptotes to be constant (up = ~ 1, low = ~ 1).
# Letting up/low vary by species x temperature overfits here and there is littl evidence up
# and low vary dramatically with temperature.
 wf_aphid <- fit_4pl(aphid_std,
                     ctmax = ~ 0 + species,
                         z = ~ 0 + species,
                         k = ~ temp_c,
```

```
                       up = ~ 1,
                      low = ~ 1,
                    t_ref = 60, silent = 2, refresh = 0,
                    file_refit = "on_change",
                    file       = file.path(models_dir, "aphid_species"))
summary(wf_aphid)
```

```
 Family: beta_binomial
  Links: mu = identity
Formula: n_surv | trials(n_total) ~ low + (up - low)/(1 + exp(exp(logk) * (logd - mid)))
         low ~ (0.001 + inv_logit(lowraw) * 0.498)
         up ~ (0.501 + inv_logit(upraw) * 0.498)
         mid ~ (1.77815 - (temp_c - CTmaxdev)/exp(logz))
         lowraw ~ 1
         upraw ~ 1
         logk ~ temp_c
         CTmaxdev ~ 0 + species
         logz ~ 0 + species
   Data: data (Number of observations: 436)
  Draws: 4 chains, each with iter = 4000; warmup = 2000; thin = 1;
         total post-warmup draws = 8000

Regression Coefficients:
                          Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS
lowraw_Intercept             -3.30      0.60    -4.68    -2.34 1.00     5286
upraw_Intercept               1.87      0.42     1.15     2.82 1.00     4216
logk_Intercept                2.35      0.08     2.20     2.52 1.00     4340
logk_temp_c                   0.15      0.03     0.08     0.22 1.00     5229
CTmaxdev_speciesM_dirhodum   -1.70      0.10    -1.91    -1.51 1.00     3808
CTmaxdev_speciesS_avenae     -0.36      0.05    -0.47    -0.27 1.00     4614
CTmaxdev_speciesR_padi        0.27      0.05     0.17     0.38 1.00     5533
logz_speciesM_dirhodum        1.56      0.03     1.51     1.62 1.00     4235
logz_speciesS_avenae          1.29      0.02     1.24     1.33 1.00     5250
logz_speciesR_padi            1.38      0.04     1.31     1.45 1.00     5912
                          Tail_ESS
lowraw_Intercept              3654
upraw_Intercept               4803
logk_Intercept                5492
logk_temp_c                   5199
CTmaxdev_speciesM_dirhodum    4482
CTmaxdev_speciesS_avenae      6030
CTmaxdev_speciesR_padi        5490
logz_speciesM_dirhodum        4512
logz_speciesS_avenae          5883
logz_speciesR_padi            5445

Further Distributional Parameters:
```

Table S12: Per-species thermal-death-time parameters for the three cereal aphids, read from the joint 4PL posterior (grouped by species): thermal sensitivity z, the 1-hour heat tolerance $CT_{max,1h}$, and the critical temperature threshold T_{crit}. Cells are the posterior median with 95% credible interval.

Species	z (°C)	CTmax 1h (°C)	Tcrit (°C)
M. dirhodum	4.8 [4.5, 5]	35.2 [35, 35.4]	23.2 [20.7, 25.6]
R. padi	4 [3.7, 4.3]	37.1 [37, 37.3]	27.3 [25.1, 29.3]
S. avenae	3.6 [3.5, 3.8]	36.5 [36.4, 36.6]	27.4 [25.6, 29.2]

```
    Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
phi     7.84      1.14     5.93    10.39 1.00     8548     5522

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

Overall, the model looks good and explains 69.5% of variation [95% CI: 67.1 to 71.5].

6.3 Per-species thermal limits

```
aphid_tls <- tls(wf_aphid, by = "species", lethal = TRUE)
aphid_est <- get_tls_est(aphid_tls)

# One row per species: median [95% CI] for each quantity.
aphid_tls_tbl <- aphid_est |>
  dplyr::mutate(
    value    = format_interval(median, lower, upper, digits = 1),
    quantity = factor(quantity, levels = c("z", "CTmax", "Tcrit"),
                      labels = c("z (°C)", "CTmax 1h (°C)", "Tcrit (°C)")),
    species  = dplyr::recode(species, M_dirhodum = "M. dirhodum",
                             R_padi = "R. padi", S_avenae = "S. avenae")) |>
  dplyr::select(species, quantity, value) |>
  tidyr::pivot_wider(names_from = quantity, values_from = value) |>
  dplyr::rename(Species = species)
tinytable::tt(aphid_tls_tbl)
```

Our per-species estimates fall inside the ranges Li *et al.* (2023) report (their Table 1: z 3.2–4.7 °C, $CT_{max_{1hr}}$ 35.2–37.3 °C): here z spans 3.6–4.8 °C and $CT_{max_{1hr}}$ spans 35.2–37.1 °C (Table S12). The joint fit also reproduces their species *ordering* — *M. dirhodum* is the least heat-sensitive (highest z, 4.8 °C) and *R. padi* the most heat-tolerant (highest $CT_{max_{1hr}}$, 37.1 °C) — so the model recovers both the magnitudes and the ranking from a single posterior, without per-temperature regressions.

```r
psc_aphid <- predict_survival_curves(wf_aphid)
# Show the full fitted curves (clip_to_observed = FALSE): each species' complete
# sigmoid across the shared duration range, rather than capping each temperature
# at its longest tested exposure.
plot_survival_curves(psc_aphid, observed = aphid_std, log_time = TRUE,
                     clip_to_observed = FALSE)
```

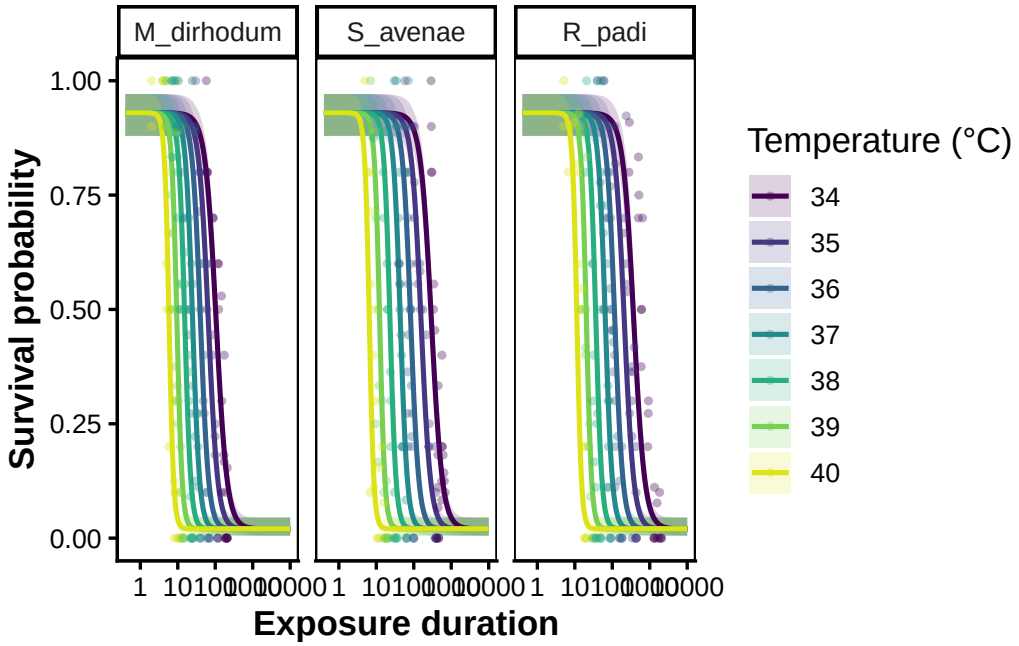


Figure S13: Aphid survival surfaces. Fitted joint-4PL survival probability against exposure duration for the three species (posterior median and 95% credible band), with the observed cell proportions overlaid.

We can also visualise the TDT curves.

```r
tdt_aphid <- derive_tdt_curve(wf_aphid, by = "species", temp_grid = seq(34, 40, by = 1))
plot_tdt_curve(tdt_aphid)
```

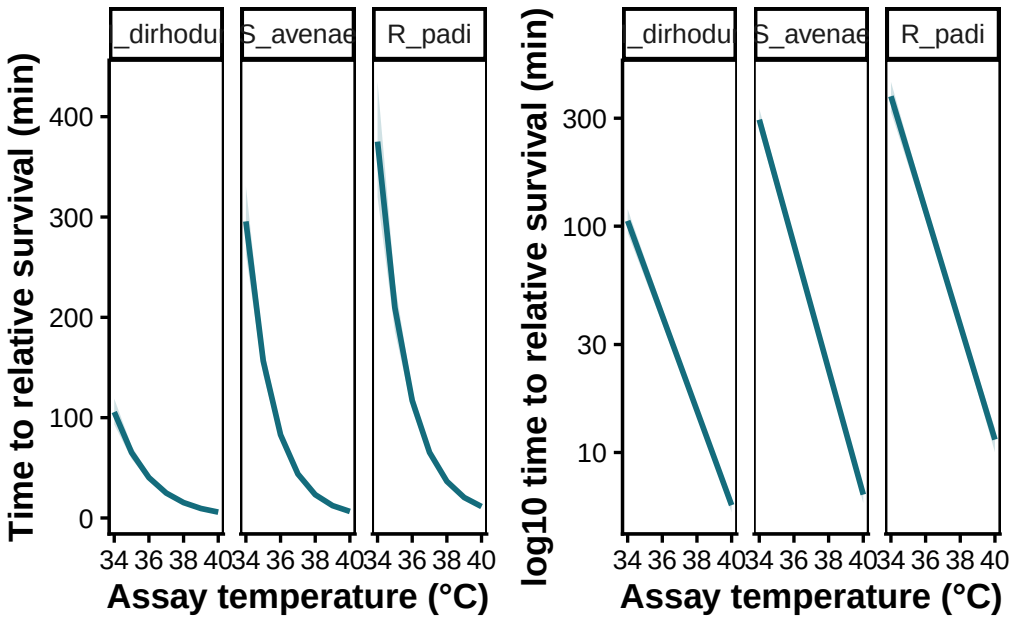


Figure S14: Thermal-death-time curves for the three cereal aphids, read from the joint 4PL posterior across assay temperatures (34–40 °C). Each panel gives the exposure duration needed to reach the relative survival threshold, on a $\log_{10}$ time axis (posterior median with 95% credible band). The slope of each line is $-1/z$, so a shallower curve means lower thermal sensitivity.

However, with our model, we can now go one step further! Li *et al.* (2023) also measure across different ages. We can ask whether the heat sensitivity of each species varies across age too, estimating how sensitivity and tolerance change across age for each species. Lets fit that model:

```
## First, we need to go back. to the raw data and just include all ages, not just age 6.
aphid_raw2 <- aphid_tdt |>
  dplyr::filter(branch == "heat") |>
  dplyr::mutate(species = droplevels(species))

# Standardize again
aphid_stn2 <- standardize_data(aphid_raw2, temp = "temp", duration = "duration_min",
                               n_total = "n_total", n_surv = "n_surv",
                               duration_unit = "minutes")   # keeps `species`

# As with the main species model, we constrain the asymptotes to be constant
# to avoid overfitting. Species x age
# enters the midpoint (so z and CTmax vary by species and age); k keeps a
# temperature slope.
wf_aphid_age <- fit_4pl(aphid_stn2,
                        ctmax = ~ 1 + species*age,
                            z = ~ 1 + species*age,
                            k = ~ temp_c,
                           up = ~ 1,
```

```
                      low = ~ 1,
                    t_ref = 60, silent = 2, refresh = 0,
                    file_refit = "on_change",
                    file       = file.path(models_dir, "aphid_species_age"))
summary(wf_aphid_age)
```

```
 Family: beta_binomial
  Links: mu = identity
Formula: n_surv | trials(n_total) ~ low + (up - low)/(1 + exp(exp(logk) * (logd - mid)))
         low ~ (0.001 + inv_logit(lowraw) * 0.498)
         up ~ (0.501 + inv_logit(upraw) * 0.498)
         mid ~ (1.77815 - (temp_c - CTmaxdev)/exp(logz))
         lowraw ~ 1
         upraw ~ 1
         logk ~ temp_c
         CTmaxdev ~ 1 + species * age
         logz ~ 1 + species * age
   Data: data (Number of observations: 1314)
  Draws: 4 chains, each with iter = 4000; warmup = 2000; thin = 1;
         total post-warmup draws = 8000

Regression Coefficients:
                              Estimate Est.Error l-95% CI u-95% CI Rhat
lowraw_Intercept                 -3.35      0.57    -4.66    -2.47 1.00
upraw_Intercept                   1.72      0.25     1.29     2.25 1.00
logk_Intercept                    2.29      0.06     2.19     2.41 1.00
logk_temp_c                       0.10      0.02     0.06     0.14 1.00
CTmaxdev_Intercept               -1.79      0.08    -1.95    -1.63 1.00
CTmaxdev_speciesS_avenae          0.90      0.11     0.68     1.11 1.00
CTmaxdev_speciesR_padi            1.58      0.09     1.41     1.76 1.00
CTmaxdev_age6                     0.10      0.12    -0.13     0.33 1.00
CTmaxdev_age12                    0.89      0.09     0.72     1.07 1.00
CTmaxdev_speciesS_avenae:age6     0.43      0.14     0.15     0.71 1.00
CTmaxdev_speciesR_padi:age6       0.38      0.13     0.13     0.64 1.00
CTmaxdev_speciesS_avenae:age12   -0.32      0.13    -0.56    -0.07 1.00
CTmaxdev_speciesR_padi:age12     -0.66      0.12    -0.90    -0.43 1.00
logz_Intercept                    1.57      0.03     1.51     1.63 1.00
logz_speciesS_avenae             -0.14      0.04    -0.22    -0.06 1.00
logz_speciesR_padi               -0.35      0.04    -0.42    -0.28 1.00
logz_age6                        -0.01      0.04    -0.09     0.06 1.00
logz_age12                       -0.18      0.03    -0.25    -0.12 1.00
logz_speciesS_avenae:age6        -0.15      0.05    -0.25    -0.04 1.00
logz_speciesR_padi:age6           0.14      0.05     0.04     0.24 1.00
logz_speciesS_avenae:age12       -0.06      0.05    -0.17     0.04 1.00
logz_speciesR_padi:age12          0.32      0.05     0.23     0.42 1.00
                              Bulk_ESS Tail_ESS
lowraw_Intercept                  6658     4922
```

```
upraw_Intercept                     7017     5325
logk_Intercept                      5953     5245
logk_temp_c                         9544     5879
CTmaxdev_Intercept                  1889     2939
CTmaxdev_speciesS_avenae            2171     3135
CTmaxdev_speciesR_padi              1883     3377
CTmaxdev_age6                       2029     3122
CTmaxdev_age12                      1918     3081
CTmaxdev_speciesS_avenae:age6       2203     3286
CTmaxdev_speciesR_padi:age6         2174     3762
CTmaxdev_speciesS_avenae:age12      2326     3645
CTmaxdev_speciesR_padi:age12        2397     4137
logz_Intercept                      1805     3042
logz_speciesS_avenae                2248     3298
logz_speciesR_padi                  2073     3711
logz_age6                           2041     3367
logz_age12                          2007     3226
logz_speciesS_avenae:age6           2351     3767
logz_speciesR_padi:age6             2732     4767
logz_speciesS_avenae:age12          2881     3955
logz_speciesR_padi:age12            2853     4592

Further Distributional Parameters:
    Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
phi     7.35      0.60     6.26     8.63 1.00     9633     5504

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

There's a lot going on here, but what's important for us is to pay attention to the `ctmaxdev` and `logz` in the model because this shows us that tolerance and sensitivity vary across age differently across species, which is pretty cool! Lets have a look:

```r
aphid_age_tls <- tls(wf_aphid_age, by = c("species", "age"), lethal = TRUE)
aphid_age_est <- get_tls_est(aphid_age_tls)

# z and CTmax for every species x age cell: median [95% CI].
aphid_age_tbl <- aphid_age_est |>
  dplyr::filter(quantity %in% c("z", "CTmax")) |>
  dplyr::mutate(
    value    = format_interval(median, lower, upper, digits = 1),
    quantity = factor(quantity, levels = c("z", "CTmax"),
                      labels = c("z (°C)", "CTmax 1h (°C)")),
    species  = dplyr::recode(species, M_dirhodum = "M. dirhodum",
                             R_padi = "R. padi", S_avenae = "S. avenae"),
    age      = factor(age, levels = c("2", "6", "12"))) |>
```

Table S13: Per-species, per-age thermal-death-time parameters for the three cereal aphids from the age-extended joint 4PL (each species × age cell via `tls()` and `get_tls_est()`): thermal sensitivity z and the 1-hour heat tolerance $CT_{max,1h}$. Cells are the posterior median with 95% credible interval.

Species	Age (days)	z (°C)	CTmax 1h (°C)
M. dirhodum	2	4.8 [4.5, 5.1]	35.1 [34.9, 35.2]
M. dirhodum	6	4.7 [4.5, 5]	35.2 [35, 35.4]
M. dirhodum	12	4 [3.9, 4.2]	36 [35.9, 36.1]
S. avenae	2	4.2 [4, 4.4]	36 [35.8, 36.1]
S. avenae	6	3.6 [3.4, 3.7]	36.5 [36.4, 36.6]
S. avenae	12	3.3 [3.1, 3.5]	36.6 [36.5, 36.7]
R. padi	2	3.4 [3.2, 3.6]	36.7 [36.6, 36.8]
R. padi	6	3.8 [3.6, 4.1]	37.2 [37.1, 37.3]
R. padi	12	3.9 [3.7, 4.2]	36.9 [36.8, 37]

```
  dplyr::select(species, age, quantity, value) |>
  tidyr::pivot_wider(names_from = quantity, values_from = value) |>
  dplyr::arrange(species, age) |>
  dplyr::rename(Species = species, `Age (days)` = age)
tinytable::tt(aphid_age_tbl)
```

Two patterns stand out (Table S13). **Heat tolerance rises with age in all three species**: the 1-hour CT_{max} increases from the youngest to the oldest aphids — from 35.1 to 36 °C in *M. dirhodum* and 36 to 36.6 °C in *S. avenae*, with a smaller increase in *R. padi* (which peaks at 6 days). The species ranking holds at every age — *R. padi* tolerates the highest temperatures and *M. dirhodum* the lowest.

Sensitivity, though, shifts with age in opposite directions across species. A smaller z means more sensitivity — less warming is needed to reduce survival time tenfold. *M. dirhodum* and *S. avenae* both grow more sensitive with age (z falling from 4.8 to 4 and from 4.2 to 3.3 °C), whereas *R. padi* grows *less* sensitive (z rising from 3.4 to 3.9 °C). Tolerance and sensitivity therefore do not track one another, and the age effect on sensitivity is itself species-specific.

We recovered similar findings to Li *et al.* (2023) because, in essence, their data generally follow assumptions met by the two-stage analysis (our simulations show this should be the case - Section 3.0.1.1).

However, the R^2 is inflated in their models (Li *et al.* (2023) report R^2 = 0.97-0.996) because much of the variation in survival outcomes is disposed of during stage-1. Our 4PL model provides much more realistic R^2 (again, 0.7 [95% CI: 0.7 to 0.7]).

6.4 Heat injury across species under a field temperature trace

With the per-species TDT surface in hand, `predict_heat_injury()` accumulates damage along an hourly field temperature trace and converts it to predicted survival. We use hourly 2 m air temperature for Wuhan (the hottest of Li et al.'s three field sites) from May–June 2016, re-sourced from the Open-Meteo ERA5 archive and shipped with the package (built by `data-raw/fetch_aphid_temp_trace.R`). We restrict the window to May–June so the injury scale stays readable — over the full May–August record the most heat-vulnerable species (*M. dirhodum*, the lowest T_{crit}) accrues injury into the tens of thousands of percent of an LT_{50} dose. Using `by = "species"` projects all three species at once, demonstrating multi-group heat injury.

```
# Hourly (time, temp) trace per city ships in inst/extdata; take Wuhan, trimmed
# to May-June (over the full May-Aug record the most vulnerable species runs to
# ~tens of thousands of % of an LT50 dose, swamping the axis -- see prose).
aphid_trace <- readr::read_csv(
  here::here("inst", "extdata", "data_temp_trace_aphid_summer2016.csv"),
  show_col_types = FALSE) |>
  dplyr::filter(city == "Wuhan",
                as.integer(format(datetime, "%m")) <= 6) |>   # May-June only
  dplyr::transmute(time = time_h, temp = temp_c)        # hours, degrees C

aphid_hi <- predict_heat_injury(aphid_trace, wf_aphid, target_surv = "relative",
                                trace_unit = "hours", by = "species")
plot_heat_injury(aphid_hi)
```

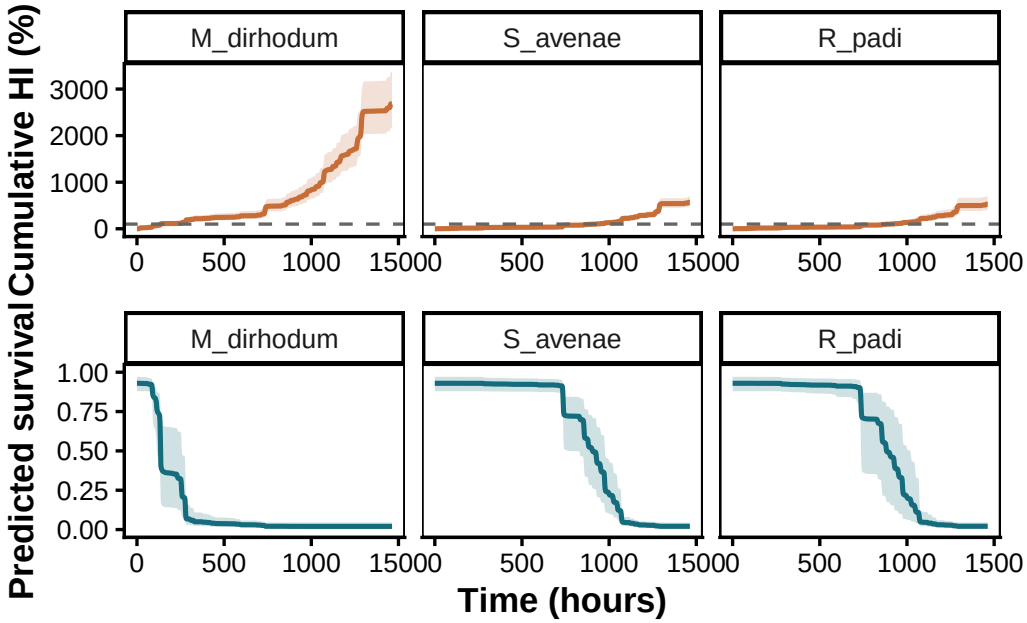


Figure S15: Multi-group heat-injury projection. Cumulative heat injury (100% = one LT50 dose) and predicted survival for the three aphid species under the unmodified Wuhan May–June 2016 hourly temperature trace, from a single `predict_heat_injury(by = "species")` call.

The accumulated injury and end-of-window survival differ across species in line with their thermal limits, and the same single call would map the south-to-north climate gradient (Wuhan hottest, Beijing coolest) onto a corresponding gradient in projected survival — the link Li *et al.* (2023) draw between thermal tolerance and field abundance.

7 Case Study 3: Snow gum leaf PSII — light vs dark recovery

Arnold *et al.* (2026) developed a thermal-load-sensitivity (TLS) protocol for photosystem II (PSII) and asked, among other things, whether the light environment a leaf experiences *after* heat exposure affects measured heat tolerance. Here we take the snow gum (*Eucalyptus pauciflora*) arm of their Experiment 1: excised leaf sections from six mature trees were heated across a grid of assay temperatures (30–56 °C) and exposure durations (5–120 min), then held either in **darkness** or under **90 min of moderate light** before F_v/F_m was re-measured 16–24 h later. The response — retained PSII function, post-exposure F_v/F_m divided by its pre-exposure value — is a **continuous proportion in** $[0, 1]$ **with no count denominator**, so the 4PL uses a **Beta** likelihood rather than the beta-binomial of the count case studies.

We let both CT_{max} and z depend on the recovery condition inside one joint 4PL. Arnold *et al.* (2026) report that post-heat light *lowers* apparent heat tolerance while leaving thermal sensitivity broadly unchanged, so the joint model should place the Light CT_{max} below the Dark one with similar z.

7.1 Data and standardisation

```
# Bundled Arnold et al. (2026) snow gum PSII data (Experiment 1, light vs dark
# recovery). Two raw rows with post/pre Fv/Fm marginally above 1 (measurement
# noise) are already excluded upstream, leaving 394 rows.
data(snowgum_psii)

# `proportion =` (not n_total/n_surv) flags a continuous response: standardize_data()
# records response_type = "proportion" and clamps to (0.001, 0.999) for the Beta
# density. `recovery` (Dark/Light) and `plant` (six trees) pass through as the
# moderator and the random-effect grouping.
leaf_std <- standardize_data(snowgum_psii, temp = "Temp", duration = "Time",
                             proportion = "fvfm_prop", duration_unit = "minutes")
str(leaf_std)
```

```
tibble [394 x 13] (S3: tbl_df/tbl/data.frame)
 $ Temp        : num [1:394] 30 30 30 30 30 30 30 30 30 30 ...
 $ Time        : num [1:394] 15 15 15 15 15 15 15 15 15 15 ...
 $ recovery    : Factor w/ 2 levels "Dark","Light": 1 1 1 1 1 1 2 2 2 2 ...
 $ plant       : Factor w/ 6 levels "1","2","3","4",..: 1 2 3 4 5 6 1 2 3 4 ...
 $ meas_day    : Factor w/ 2 levels "1","2": 2 2 2 2 2 2 2 2 2 2 ...
 $ initial_fvfm: num [1:394] 0.862 0.837 0.851 0.831 0.835 0.842 0.859 0.851 0.856 0.85 ...
 $ final_fvfm  : num [1:394] 0.794 0.79 0.807 0.781 0.805 0.809 0.79 0.767 0.798 0.775 ...
```

Table S14: Summary of the snow gum leaf PSII dataset after standardisation. Temperature centre is the grand mean used to mean-centre the temperature covariate.

n observations	n trees (plant)	Recovery conditions	Temperature range (°C)	Duration range (min)	Temp
394	6	Dark, Light	30–56	5–120	43

```
$ fvfm_prop   : num [1:394] 0.921 0.944 0.948 0.94 0.964 ...
$ temp        : num [1:394] 30 30 30 30 30 30 30 30 30 30 ...
$ duration    : num [1:394] 15 15 15 15 15 15 15 15 15 15 ...
$ logd        : num [1:394] 1.18 1.18 1.18 1.18 1.18 ...
$ survival    : num [1:394] 0.921 0.944 0.948 0.94 0.964 ...
$ temp_c      : num [1:394] -12.6 -12.6 -12.6 -12.6 -12.6 ...
- attr(*, "tdt_meta")=List of 6
 ..$ temp_mean     : num 42.6
 ..$ duration_unit : chr "minutes"
 ..$ random_effects: NULL
 ..$ response_type : chr "proportion"
 ..$ response_var  : chr "survival"
 ..$ proportion_eps: num 0.001
```

Response model and zeros. Unlike the count-based case studies, the leaf response is a continuous proportion with no denominator, so we use a Beta likelihood. The Beta density is undefined at exactly 0 or 1, but 89 of 394 observations (23%) are zeros — complete loss of measurable PSII function at the hottest, longest exposures. `standardize_data()` clamps these to 0.001 so a single Beta model covers the whole surface. A zero-inflated Beta that separately modelled complete shutdown would represent these structural zeros more faithfully, but we keep this simple for now and it's unlikely we would see much difference between the two models in any case.

```
leaf_summary <- tibble::tibble(
  `n observations`            = nrow(leaf_std),
  `n trees (plant)`           = nlevels(leaf_std$plant),
  `Recovery conditions`       = paste(levels(leaf_std$recovery), collapse = ", "),
  `Temperature range (°C)`    = paste(range(leaf_std$temp), collapse = "–"),
  `Duration range (min)`      = paste(round(range(leaf_std$duration), 0), collapse = "–"),
  `Temperature centre (°C)`   = round(attr(leaf_std, "tdt_meta")$temp_mean, 1)
)
tinytable::tt(leaf_summary, digits = 2, escape = TRUE)
```

7.2 Joint 4PL with recovery on CT_{max} and z

We fit CT_{max} and z directly as functions of `recovery` (`~ 0 + recovery`), so each condition gets its own coefficient. A random intercept of `plant` absorbs among-tree variation in the threshold. We ignored `meas_day` because it has only two levels, too few to estimate a variance.

```r
# Beta family is auto-picked from response_type; named here for clarity. The
# clamped structural zeros sit on the Beta boundary and induce a small number of
# divergent transitions (exact count + rate reported below the diagnostics
# table); adapt_delta is raised to keep them low. The posterior is otherwise
# well-mixed (see the diagnostics table).
wf_leaf <- fit_4pl(
  leaf_std,
  ctmax  = ~ 0 + recovery + (1 | plant),
  z      = ~ 0 + recovery,
  family = brms::Beta(link = "identity"),
  t_ref  = 60,
  iter = 6000, warmup = 3000, seed = 123,
  control = list(adapt_delta = 0.999, max_treedepth = 15),
  file_refit = "on_change",
  file = file.path(models_dir, "fit_leaf_recovery_4pl")
)
summary(wf_leaf)
```

```
 Family: beta
  Links: mu = identity
Formula: survival ~ low + (up - low)/(1 + exp(exp(logk) * (logd - mid)))
         low ~ (0.001 + inv_logit(lowraw) * 0.498)
         up ~ (0.501 + inv_logit(upraw) * 0.498)
         mid ~ (1.77815 - (temp_c - CTmaxdev)/exp(logz))
         lowraw ~ 0 + recovery + temp_c:recovery
         upraw ~ 0 + recovery + temp_c:recovery
         logk ~ 0 + recovery + temp_c:recovery
         CTmaxdev ~ 0 + recovery + (1 | plant)
         logz ~ 0 + recovery
   Data: data (Number of observations: 394)
  Draws: 4 chains, each with iter = 6000; warmup = 3000; thin = 1;
         total post-warmup draws = 12000

Multilevel Hyperparameters:
~plant (Number of levels: 6)
                      Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS
sd(CTmaxdev_Intercept)     0.55      0.25     0.19     1.16 1.00     2174
                      Tail_ESS
sd(CTmaxdev_Intercept)     5581

Regression Coefficients:
                           Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS
lowraw_recoveryDark           -3.48      0.98    -5.36    -1.50 1.00    10866
lowraw_recoveryLight          -1.93      0.52    -2.96    -0.97 1.00     2359
lowraw_recoveryDark:temp_c    -0.38      0.33    -1.13     0.13 1.00     5600
lowraw_recoveryLight:temp_c   -0.11      0.07    -0.24     0.01 1.00     2364
upraw_recoveryDark             1.26      0.16     0.96     1.58 1.00     7225
```

Table S15: Sampling diagnostics for the leaf 4PL fit. Healthy targets: Rhat < 1.01, high ESS. A small residual divergence count comes from the clamped structural zeros at the Beta boundary.

rhat_max	ess_bulk_min	ess_tail_min	divergences	treedepth_max	treedepth_hits	bfmi_min	rhat_
1	2110	2293	135	15	0	0.835	TRU

```
upraw_recoveryLight             1.17      0.17     0.85     1.53 1.00     3834
upraw_recoveryDark:temp_c      -0.02      0.02    -0.06     0.03 1.00     7890
upraw_recoveryLight:temp_c     -0.01      0.02    -0.06     0.03 1.00     4944
logk_recoveryDark               1.92      0.21     1.51     2.32 1.00     5385
logk_recoveryLight              3.06      0.73     1.63     4.36 1.00     2110
logk_recoveryDark:temp_c       -0.10      0.03    -0.14    -0.04 1.00     4509
logk_recoveryLight:temp_c      -0.11      0.11    -0.27     0.14 1.00     2234
CTmaxdev_recoveryDark           3.09      0.30     2.46     3.68 1.00     4000
CTmaxdev_recoveryLight          1.40      0.33     0.82     2.17 1.00     2274
logz_recoveryDark               1.38      0.07     1.24     1.51 1.00     9124
logz_recoveryLight              1.36      0.07     1.18     1.48 1.00     4036
                            Tail_ESS
lowraw_recoveryDark             9318
lowraw_recoveryLight            6012
lowraw_recoveryDark:temp_c      5160
lowraw_recoveryLight:temp_c     6629
upraw_recoveryDark              7599
upraw_recoveryLight             4402
upraw_recoveryDark:temp_c       7559
upraw_recoveryLight:temp_c      4540
logk_recoveryDark               4972
logk_recoveryLight              2850
logk_recoveryDark:temp_c        3275
logk_recoveryLight:temp_c       2774
CTmaxdev_recoveryDark           5572
CTmaxdev_recoveryLight          2293
logz_recoveryDark               7273
logz_recoveryLight              3519

Further Distributional Parameters:
    Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
phi     9.43      0.76     8.00    11.02 1.00     9195     8583

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
tinytable::tt(diagnose_tdt_fit(wf_leaf), digits = 3, escape = TRUE)
```

The clamped structural zeros leave 135 divergent transitions out of 1.2×10^4 post-warmup draws

Table S16: Per-condition TDT summaries for snow gum leaf PSII (relative midpoint threshold), posterior medians with 95% credible intervals. z is the temperature slope on the midpoint ($z = -1/\beta_1$); $CT_{max_{1hr}}$ is the temperature at which the retained-function threshold is reached after 1 h.

Recovery	Quantity	Median	Lower	Upper
Dark	z (°C)	3.98	3.46	4.54
Dark	CTmax_1hr (°C)	45.74	45.10	46.32
Light	z (°C)	3.94	3.26	4.39
Light	CTmax_1hr (°C)	44.00	43.46	44.81

(1.1%) but the z and CT_{max} estimates below are insensitive to them in any case.

7.3 Per-condition thermal limits

`tls(by = "recovery")` derives z and $CT_{max_{1hr}}$ for each condition from the shared posterior, so each quantity carries full uncertainty. Loss of PSII function is sublethal, so `lethal = FALSE` and no T_{crit} is returned.

```
leaf_tls <- tls(wf_leaf, by = "recovery", lethal = FALSE)
leaf_tls$summary
```

```
# A tibble: 4 x 5
  recovery quantity median lower upper
  <fct>    <chr>     <dbl> <dbl> <dbl>
1 Dark     z          3.98  3.46  4.54
2 Dark     CTmax     45.7  45.1  46.3
3 Light    z          3.94  3.26  4.39
4 Light    CTmax     44.0  43.5  44.8
```

```
leaf_tls_tab <- leaf_tls$summary |>
  dplyr::mutate(quantity = dplyr::recode(quantity,
                                         z = "z (°C)", CTmax = "CTmax_1hr (°C)")) |>
  dplyr::transmute(`Recovery` = recovery, Quantity = quantity,
                   Median = round(median, 2),
                   Lower = round(lower, 2), Upper = round(upper, 2))
tinytable::tt(leaf_tls_tab, escape = TRUE)
```

The joint fit reproduces the direction Arnold *et al.* (2026) report: 90 min of post-heat light lowers $CT_{max_{1hr}}$ to 44 °C, below the dark-recovery 45.7 °C, while thermal sensitivity is essentially unchanged (z 3.9 °C under light vs 4 °C in the dark). Post-heat light therefore reduces measured tolerance limit without altering how steeply tolerance time declines with temperature — matching their descriptive conclusion that light following heat alters tolerance but not sensitivity.

7.4 Retained PSII function curves with observed overlay

`predict_survival_curves()` evaluates the fitted 4PL on a grid of assay temperatures and durations overlaying the observed and predicted proportions.

```
psc_leaf <- predict_survival_curves(wf_leaf)
# clip_to_observed = FALSE: full fitted curves (complete sigmoids) for both
# recovery treatments, not truncated at the observed durations.
plot_survival_curves(psc_leaf, observed = leaf_std, log_time = TRUE,
                     clip_to_observed = FALSE) +
  ggplot2::labs(x = "Exposure duration (minutes)",
                y = "Retained PSII function")
```

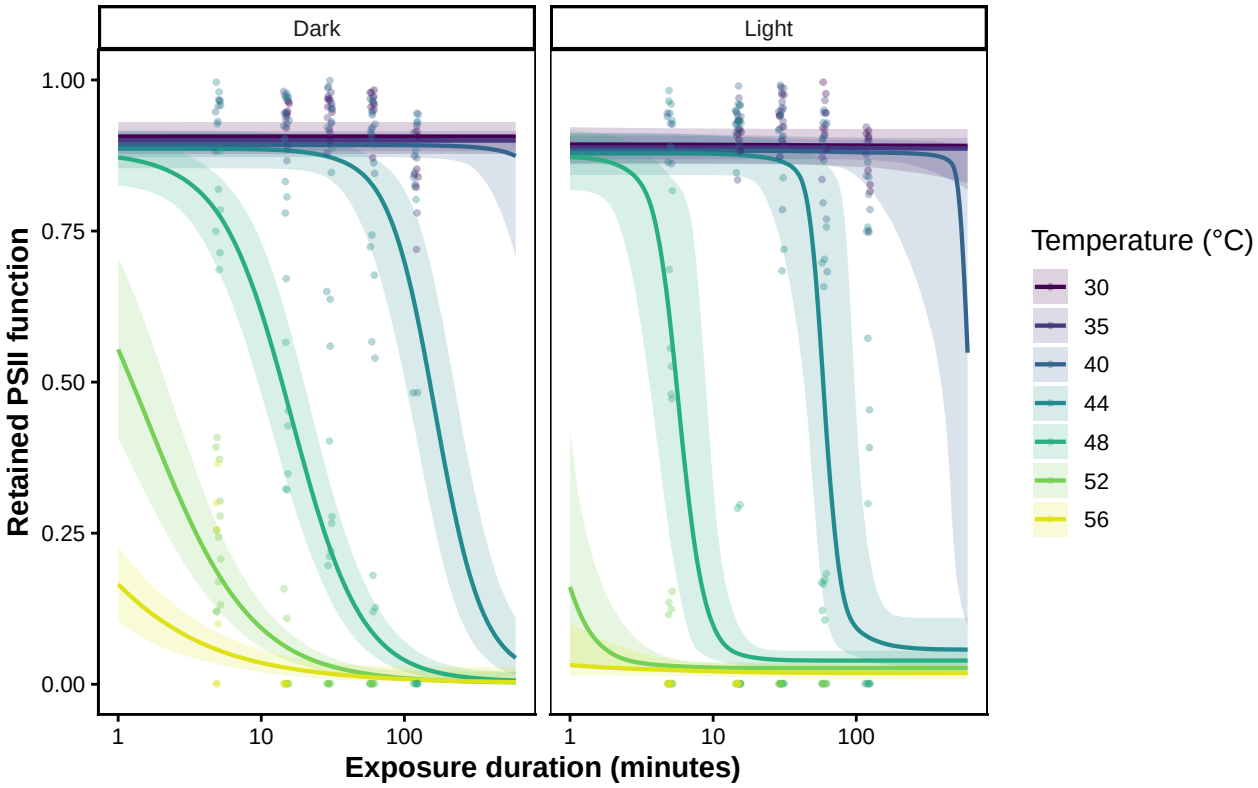


Figure S16: Fitted retained-PSII-function surfaces for snow gum under dark and light post-heat recovery (posterior median and 95% credible band), with observed per-replicate proportions overlaid. The light-recovery curves cross the midpoint at shorter durations, the visual signature of the lower CT_{max}.

7.5 Model fit: Bayesian R^2

Loss of PSII function is described by a single joint 4PL (Beta likelihood), so there is one model to report (Table S17). `bayes_R2_tls()` wraps `brms::bayes_R2()` to return a tidy posterior median with its 95% credible interval for R^2.

Table S17: Bayesian R^2 for the snow gum leaf PSII 4PL fit, from `bayes_R2_tls()` (a tidy wrapper around `brms::bayes_R2()`). Posterior median with 95% credible interval.

Model	Bayesian R^2
Leaf PSII (4PL, Beta)	0.935 [0.93, 0.94]

```
bayes_R2_tls(wf_leaf) |>
  dplyr::transmute(
    Model         = "Leaf PSII (4PL, Beta)",
    `Bayesian R²` = format_interval(estimate, lower, upper, digits = 3)
  ) |>
  tinytable::tt(escape = TRUE)
```

8 Case Study 4: Vinegar fly (*Drosophila suzukii*) TDT across sexes

We'll now turn our attention to a paper by Ørsted *et al.* (2024), who measured TDTs for productivity (offspring number; a measure of fertility), time to coma (sublethal) and mortality using *Drosophila suzukii*.

We'll first load the data and start with the mortality data:

8.0.0.1 Load the data

```
# Load the Drosophila data and summarise mortality counts by sex.
data(dsuzukii)

# Count deaths in each temperature-by-time-by-sex group.
mort_dros <- dsuzukii |>
             dplyr::group_by(temp, time, sex) |>
             dplyr::summarise(n_total = dplyr::n(), n_dead = sum(dead), .groups = "drop") |>
             arrange(temp, time, sex)
```

8.0.0.2 Fitting the 4PL model

Now that we have the data set up we'll fit the 4PL model. Just to showcase how we can use the posterior to make comparisons we'll first model the sexes separately.

```
# Fit separate mortality 4PL models for female and male Drosophila.
# Split mortality counts by sex.
mort_dros_F <- dplyr::filter(mort_dros, sex == "F")
mort_dros_M <- dplyr::filter(mort_dros, sex == "M")

# Standardise each sex's counts for the modelling schema.
```

```r
std_mort_F <- standardize_data(mort_dros_F, temp = "temp", duration = "time",
                               n_total = "n_total", n_dead = "n_dead",
                               duration_unit = "minutes")
std_mort_M <- standardize_data(mort_dros_M, temp = "temp", duration = "time",
                               n_total = "n_total", n_dead = "n_dead",
                               duration_unit = "minutes")

# Fit one 4PL per sex.
wf_dros_mort_F <- fit_4pl(
  std_mort_F,
  chains  = 4, iter = 8000, warmup = 4000, cores = 4, seed = 123,
  control = list(adapt_delta = 0.99, max_treedepth = 12),
  file    = file.path(models_dir, "fit_dsuzukii_mortality_4pl_F")
)

wf_dros_mort_M <- fit_4pl(
  std_mort_M,
  chains  = 4, iter = 8000, warmup = 4000, cores = 4, seed = 123,
  control = list(adapt_delta = 0.99, max_treedepth = 12),
  file    = file.path(models_dir, "fit_dsuzukii_mortality_4pl_M")
)

# Show brms summaries and diagnostics for each fit.
summary(wf_dros_mort_F)
```

```
 Family: beta_binomial
  Links: mu = identity
Formula: n_surv | trials(n_total) ~ (0.001 + inv_logit(lowraw) * 0.498) + ((0.501 + inv_logit(u
         lowraw ~ temp_c
         upraw ~ temp_c
         logk ~ temp_c
         mid ~ temp_c
   Data: data (Number of observations: 45)
  Draws: 4 chains, each with iter = 8000; warmup = 4000; thin = 1;
         total post-warmup draws = 16000

Regression Coefficients:
                Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
lowraw_Intercept    -4.08      0.78    -5.71    -2.66 1.00    12930    11013
lowraw_temp_c        0.08      0.40    -0.70     0.86 1.00    14664    11307
upraw_Intercept      0.86      0.32     0.26     1.52 1.00    10127     9695
upraw_temp_c        -0.13      0.20    -0.54     0.26 1.00    11649    11209
logk_Intercept       2.88      0.21     2.47     3.32 1.00    10048    11037
logk_temp_c          0.05      0.13    -0.21     0.29 1.00    11240    10922
mid_Intercept        2.10      0.02     2.06     2.13 1.00     9791     9418
mid_temp_c          -0.33      0.01    -0.35    -0.31 1.00    10708    10260
```

```
Further Distributional Parameters:
    Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
phi    21.20      9.95     8.32    46.61 1.00    13520    11797

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
summary(wf_dros_mort_M)
```

```
 Family: beta_binomial
  Links: mu = identity
Formula: n_surv | trials(n_total) ~ (0.001 + inv_logit(lowraw) * 0.498) + ((0.501 + inv_logit(u
         lowraw ~ temp_c
         upraw ~ temp_c
         logk ~ temp_c
         mid ~ temp_c
   Data: data (Number of observations: 49)
  Draws: 4 chains, each with iter = 8000; warmup = 4000; thin = 1;
         total post-warmup draws = 16000

Regression Coefficients:
                Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
lowraw_Intercept    -3.59      0.85    -5.36    -2.03 1.00    13433    10665
lowraw_temp_c        0.17      0.53    -0.85     1.18 1.00    12294    11927
upraw_Intercept      0.94      0.26     0.44     1.47 1.00    12223    11112
upraw_temp_c        -0.25      0.16    -0.58     0.05 1.00    13551    10456
logk_Intercept       2.82      0.22     2.40     3.26 1.00    11760    10646
logk_temp_c          0.01      0.13    -0.23     0.27 1.00    12607    10667
mid_Intercept        2.08      0.02     2.04     2.11 1.00    11907    10407
mid_temp_c          -0.31      0.01    -0.34    -0.29 1.00    11758    10919

Further Distributional Parameters:
    Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
phi    17.09      7.88     7.28    37.10 1.00    13420    10731

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

The per-sex Bayesian R^2 for the two mortality fits:

```
# Bayesian R2 for each mortality fit, straight from bayes_R2_tls().
dplyr::bind_rows(
  Female = bayes_R2_tls(wf_dros_mort_F),
  Male   = bayes_R2_tls(wf_dros_mort_M),
```

Table S18: Bayesian R^2 for the per-sex *D. suzukii* mortality 4PL fits, via `bayes_R2_tls()` (a tidy wrapper around `brms::bayes_R2()`). Posterior median with 95% credible interval.

Model	Bayesian R^2
Female	0.896 [0.86, 0.915]
Male	0.785 [0.729, 0.82]

```
  .id = "Model"
) |>
  dplyr::transmute(
    Model,
    `Bayesian R²` = format_interval(estimate, lower, upper, digits = 3)
  ) |>
  tinytable::tt(escape = TRUE)
```

We can see each model explains quite a bit of variance but not the >90% reported in the paper. That's because all the variance is retained in the joint fit. Regardless, its high! Note that the models have converged well based on the Rhat, but we can also look at the MCMC chains if we need to as per below:

8.0.0.3 Extracting *z* and $CT_{max_{1hr}}$ for males and females

We can extract the TDT parameters now as we have done in other case studies:

```
# Compute the z, CTmax and Tcrit from the separate Drosophila mortality fits.
# Use the absolute LT50 threshold and the 4-hour reference exposure.
  tdt_dros_m <- tls(wf_dros_mort_M, target_surv = "absolute", t_ref = 60*4, lethal = TRUE)
  tdt_dros_f <- tls(wf_dros_mort_F, target_surv = "absolute", t_ref = 60*4, lethal = TRUE)

# We'll use the get_tls_est() function. This allows us to extract the full posterior or the su
# Combine the per-sex tidy-long summaries (quantity / median / lower / upper)
# into one table, tagging each block with its Sex. This `dros_sex_sum` object
# feeds both the per-sex TDT table below and the manuscript Figure 5 summary.
  dros_sex_sum <- dplyr::bind_rows(
    Female = get_tls_est(tdt_dros_f, "summary"),
    Male   = get_tls_est(tdt_dros_m, "summary"),
    .id    = "Sex"
  )
```

```
# Format the separate-fit Drosophila sex TDT table from the combined per-sex
# summaries (same pivot idiom as the joint-fit tables above).
dros_sex_tdt <- dros_sex_sum |>
  dplyr::transmute(
    Sex,
    quantity = factor(quantity,
```

Table S19: Thermal-death-time parameters for *D. suzukii* mortality, estimated from a separate joint Bayesian 4PL fit per sex (absolute LT_{50} threshold, reference exposure $t_{ref} = 4$ h). Cells are the posterior median with the 95% credible interval in brackets. T_{crit} uses the rate-multiplier definition and is available because the endpoint is lethal.

Sex	z (°C)	CTmax (4 h, °C)	T_crit (°C)
Female	3.04 [2.84, 3.26]	35.13 [34.99, 35.25]	29.34 [27.75, 30.87]
Male	3.16 [2.93, 3.43]	35.16 [35.02, 35.26]	29.16 [27.44, 30.75]

```
                      levels = c("z", "CTmax", "Tcrit"),
                      labels = c("z (°C)", "CTmax (4 h, °C)", "T_crit (°C)")),
    cell     = format_interval(median, lower, upper)
  ) |>
  dplyr::arrange(Sex) |>
  tidyr::pivot_wider(names_from = quantity, values_from = cell)

tinytable::tt(dros_sex_tdt, escape = TRUE)
# Absolute CTmax_4hr medians per sex, for inline reporting (no magic numbers).
ctmax4_f <- round(dros_sex_sum$median[dros_sex_sum$Sex == "Female" &
                                      dros_sex_sum$quantity == "CTmax"], 1)
ctmax4_m <- round(dros_sex_sum$median[dros_sex_sum$Sex == "Male" &
                                      dros_sex_sum$quantity == "CTmax"], 1)
```

The per-sex TDT parameters are summarised in Table S19. We see that our z values (their Table 1: 3.28 for males, 3.03 for females) as well as $CT_{max_{4hr}}$ (35.1 °C for females and 35.2 °C for males, at the absolute threshold) pretty much match Ørsted *et al.* (2024) quite well for mortality.

We could ask whether males and females are significantly different or not from each other by simply comparing the posterior distributions. Since we have the full posterior draws, this is easy to do:

```
# Compare female and male posterior z distributions directly.
# get_tls_est(..., "draws", "z") returns tidy-long z draws (quantity/.draw/value).
  z_draws_m  <- get_tls_est(tdt_dros_m, "draws", "z")
  z_draws_f  <- get_tls_est(tdt_dros_f, "draws", "z")

# Form the female-minus-male contrast on the per-draw values.
  diff_sex_z <-  z_draws_f$value - z_draws_m$value

# pMCMC is twice the smaller posterior tail probability.
  pmcmc_sex_z <- 2 * min(mean(diff_sex_z > 0), mean(diff_sex_z < 0))
```

The median difference in z between males and females is -0.12 °C (95% CI: -0.45 to 0.2 °C; pMCMC = 0.45). The interval spans zero, so the sexes' thermal sensitivities are not clearly distinguishable.

8.0.0.4 Fitting a model to compare sexes directly - indirect method

We can also fit the sex data in a single model, which is a more powerful approach. To do that, we need to add **sex** as a categorical predictor. We'll first use the indirect method by fiting to the midpoint paremeter.

```
# Fit a single Drosophila mortality model with sex on the midpoint.
# Prepare one dataset with both sexes and default priors.
std_sex    <- standardize_data(mort_dros, temp = "temp", duration = "time",
                               n_total = "n_total", n_dead = "n_dead",
                               duration_unit = "minutes")
priors_sex <- make_4pl_priors(std_sex)

# Put sex on `mid` so z and CTmax can differ by sex. Note here that we are assuming temperature
joint_sex_fit <- fit_4pl(std_sex, mid = ~ sex * temp_c, prior = priors_sex,
                         t_ref = 60 * 4,
                         chains = 4, iter = 3000, warmup = 1000, cores = 4, seed = 123,
                         control = list(adapt_delta = 0.95),
                         file = file.path(models_dir, "fit_dsuzukii_mortality_sex_joint"),
                         file_refit = "on_change")
summary(joint_sex_fit)
```

```
 Family: beta_binomial
  Links: mu = identity
Formula: n_surv | trials(n_total) ~ (0.001 + inv_logit(lowraw) * 0.498) + ((0.501 + inv_logit(u
         lowraw ~ temp_c
         upraw ~ temp_c
         logk ~ temp_c
         mid ~ sex * temp_c
   Data: std_sex (Number of observations: 94)
  Draws: 4 chains, each with iter = 3000; warmup = 1000; thin = 1;
         total post-warmup draws = 8000

Regression Coefficients:
                Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
lowraw_Intercept    -4.11      0.74    -5.69    -2.76 1.00     8455     5001
lowraw_temp_c        0.16      0.42    -0.67     0.98 1.00     9237     5819
upraw_Intercept      0.80      0.19     0.43     1.18 1.00     8749     5724
upraw_temp_c        -0.20      0.12    -0.45     0.05 1.00    10011     6401
logk_Intercept       2.91      0.15     2.63     3.21 1.00     7502     6173
logk_temp_c          0.04      0.09    -0.14     0.23 1.00     8361     5396
mid_Intercept        2.08      0.01     2.05     2.11 1.00     6820     5212
mid_sexM             0.02      0.02    -0.01     0.06 1.00     7899     5721
mid_temp_c          -0.33      0.01    -0.35    -0.31 1.00     7308     5569
mid_sexM:temp_c      0.01      0.01    -0.01     0.04 1.00     7953     5072

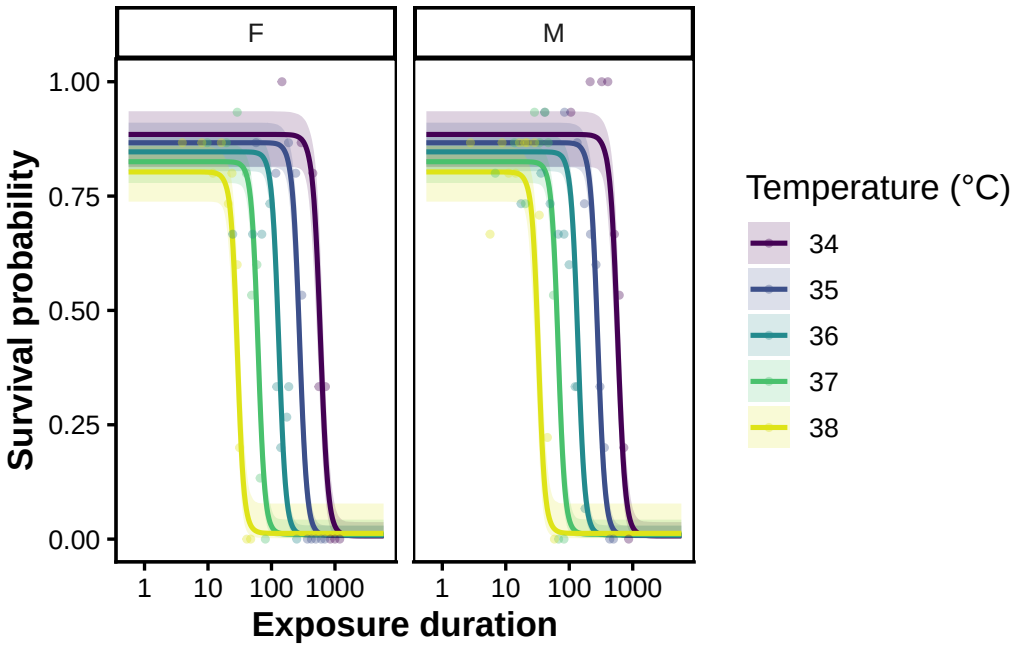
Further Distributional Parameters:
    Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
```

```
phi     18.70       6.77      9.67     35.63 1.00      7984      5226

Draws were sampled using sampling(NUTS). For each parameter, Bulk_ESS
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

We can also plot the model predictions and data out:

```
# Posterior survival curves for each sex with the observed proportions
# overlaid. predict_survival_curves(by = "sex") builds the per-sex
# temperature x duration grid; plot_survival_curves() draws the posterior
# median and 95% band and adds the observed survival points. clip_to_observed =
# FALSE shows the full fitted curves (complete sigmoids), not truncated at the
# observed durations.
psc_sex <- predict_survival_curves(joint_sex_fit, by = "sex")
plot_survival_curves(psc_sex, observed = std_sex, log_time = TRUE,
                     clip_to_observed = FALSE)
```



We can see here that the model is estimating a survival reduction at high temperatures but there doesn't appear to be much difference between the sexes. Of course, we haven't fully tested this on all parameters, but we could! Either way, we can see from the model output that the interaction `mid_sexM:temp_c` spans zero, supporting little evidence for sex differences in z.

We can now do some manual manipulation to show how to recover the same z and $CT_{max_{4hr}}$ that `tls()` and the individual per-sex models return. We do this in two steps: **Step 1** reads the *relative* values off the midpoint line ($z = -1/\text{slope}$), and **Step 2** computes the *absolute* values (adding the asymmetry correction so the line follows the full-4PL LT_{50}). Rather than printing each value, we collect them into one table (Table S20).

```
# Recover z and CTmax (4 h) by hand from the joint sex model's posterior - both
# the relative (midpoint) and absolute (full-4PL LT50) thresholds - to compare
# against the one-call tls() summaries below.
post  <- as_draws_df(get_brmsfit(joint_sex_fit))   # workflow -> underlying brmsfit
t_ref <- 60 * 4
T_bar <- mean(std_sex$temp - std_sex$temp_c)        # centring temperature

# Females are the reference; males add the sex-by-temperature interaction.
slope_F <- post$b_mid_temp_c
slope_M <- post$b_mid_temp_c + post[["b_mid_sexM:temp_c"]]
int_F   <- post$b_mid_Intercept
int_M   <- post$b_mid_Intercept + post$b_mid_sexM

# Step 1 -- RELATIVE: z is the inverse midpoint-temperature slope; CTmax solves
# the midpoint line for t_ref.
z_F     <- -1 / slope_F
z_M     <- -1 / slope_M
ctmax_F <- T_bar + (log10(t_ref) - int_F) / slope_F
ctmax_M <- T_bar + (log10(t_ref) - int_M) / slope_M

# Step 2 -- ABSOLUTE: follow the corrected full-4PL LT50 curve (the midpoint line
# plus the asymmetry correction), then read z and CTmax off that curve.
lt50 <- function(tc, slope, intercept) {
  low <- 0.001 + plogis(post$b_lowraw_Intercept + post$b_lowraw_temp_c * tc) * 0.498
  up  <- 0.501 + plogis(post$b_upraw_Intercept  + post$b_upraw_temp_c  * tc) * 0.498
  k   <- exp(post$b_logk_Intercept + post$b_logk_temp_c * tc)
  (intercept + slope * tc) + log((up - 0.5) / (0.5 - low)) / k   # natural log
}
slope_abs   <- function(slope, intercept) lt50(0.5, slope, intercept) - lt50(-0.5, slope, inte
z_F_abs     <- -1 / slope_abs(slope_F, int_F)
z_M_abs     <- -1 / slope_abs(slope_M, int_M)
ctmax_F_abs <- T_bar + (log10(t_ref) - lt50(0, slope_F, int_F)) / slope_abs(slope_F, int_F)
ctmax_M_abs <- T_bar + (log10(t_ref) - lt50(0, slope_M, int_M)) / slope_abs(slope_M, int_M)
```

```
# median [2.5%, 97.5%] from a vector of posterior draws.
ci <- function(x) { q <- unname(stats::quantile(x, c(0.5, 0.025, 0.975)))
                    format_interval(q[1], q[2], q[3]) }

byhand_tbl <- tibble::tribble(
  ~Sex,     ~`z, relative (°C)`, ~`z, absolute (°C)`, ~`CTmax, relative (4 h, °C)`, ~`CTmax, al
  "Female", ci(z_F),             ci(z_F_abs),         ci(ctmax_F),                  ci(ctmax_F_
  "Male",   ci(z_M),             ci(z_M_abs),         ci(ctmax_M),                  ci(ctmax_M_
)
tinytable::tt(byhand_tbl, escape = TRUE)
```

Everything above – z, $CT_{max_{4hr}}$ and T_{crit}, at either the relative or the absolute threshold — can be calculated using a single `tls()` call on any custom `brms` 4PL. It evaluates each sub-parameter at

Table S20: By-hand thermal-death-time parameters for *D. suzukii* mortality, derived directly from the joint sex model's posterior draws (`joint_sex_fit`): relative (midpoint) and absolute (LT_{50}) z and $CT_{max_{4hr}}$ for each sex. Cells are the posterior median with the 95% credible interval in brackets. These reproduce the one-call `tls()` summaries below.

Sex	z, relative (°C)	z, absolute (°C)	CTmax, relative (4 h, °C)	CTmax, absolute (4 h, °C)
Female	3.06 [2.88, 3.26]	3.04 [2.86, 3.24]	35.19 [35.07, 35.29]	35.14 [35.02, 35.24]
Male	3.21 [3.01, 3.42]	3.18 [2.98, 3.4]	35.22 [35.11, 35.32]	35.17 [35.06, 35.26]

the requested moderator levels with `posterior_linpred()` (just like you would predict with any brms object). Moderators on *any* 4PL parameter (and random effects) are handled automatically. We show both thresholds below.

```
# Extract relative TLS summaries by sex from the joint Drosophila model.
set.seed(123)
# One call bundles z, CTmax and T_crit for every level of `by` into a tidy,
# long summary (one row per sex x quantity). target_surv = "relative" uses the
# midpoint (low+up)/2 threshold, matching the Step-1 by-hand calculation.
tls_sex_summary <- tls(joint_sex_fit, by = "sex", target_surv = "relative",
                       lethal = TRUE, t_ref = 60 * 4)$summary
```

```
# Format the Drosophila sex joint table.
tls_sex_table <- tls_sex_summary |>
  dplyr::transmute(
    Sex      = dplyr::recode(as.character(sex), F = "Female", M = "Male"),
    quantity = factor(quantity,
                      levels = c("z", "CTmax", "Tcrit"),
                      labels = c("z (°C)", "CTmax (4 h, °C)", "T_crit (°C)")),
    cell     = format_interval(median, lower, upper)
  ) |>
  dplyr::arrange(Sex) |>                        # Female before Male, matching the table above
  tidyr::pivot_wider(names_from = quantity, values_from = cell)

tinytable::tt(tls_sex_table, escape = TRUE)
```

Switching to the **absolute** LT_{50} threshold — matching the Step-2 by-hand calculation — needs only `target_surv = "absolute"`:

```
# Extract absolute TLS summaries by sex from the joint Drosophila model.
set.seed(123)
# target_surv = "absolute" reads the curve at the 0.5 absolute-survival crossing
# (adding the asymmetry correction), matching the Step-2 by-hand calculation.
tls_sex_summary_abs <- tls(joint_sex_fit, by = "sex", target_surv = "absolute",
                           lethal = TRUE, t_ref = 60 * 4)$summary
```

Table S21: Thermal-death-time parameters for *D. suzukii* mortality from the single joint 4PL model with sex on the midpoint (`joint_sex_fit`), bundled by one `tls()` call at the **relative** midpoint threshold (`target_surv = "relative"`, reference exposure $t_{ref} = 4$ h), matching the Step-1 by-hand relative calculation. Cells are the posterior median with the 95% credible interval in brackets. Compare with the separate-fit estimates in the table above.

Sex	z (°C)	CTmax (4 h, °C)	T_crit (°C)
Female	3.06 [2.88, 3.26]	35.19 [35.07, 35.29]	29.36 [27.81, 30.88]
Male	3.21 [3.01, 3.42]	35.22 [35.11, 35.32]	29.14 [27.46, 30.73]

Table S22: As Table S21 but at the **absolute** LT_{50} threshold (`target_surv = "absolute"`), matching the Step-2 by-hand absolute calculation. Cells are the posterior median with the 95% credible interval in brackets.

Sex	z (°C)	CTmax (4 h, °C)	T_crit (°C)
Female	3.04 [2.86, 3.24]	35.14 [35.02, 35.24]	29.35 [27.81, 30.86]
Male	3.18 [2.98, 3.4]	35.17 [35.06, 35.26]	29.12 [27.45, 30.7]

```
# Format the Drosophila sex joint absolute table.
tls_sex_table_abs <- tls_sex_summary_abs |>
  dplyr::transmute(
    Sex      = dplyr::recode(as.character(sex), F = "Female", M = "Male"),
    quantity = factor(quantity,
                      levels = c("z", "CTmax", "Tcrit"),
                      labels = c("z (°C)", "CTmax (4 h, °C)", "T_crit (°C)")),
    cell     = format_interval(median, lower, upper)
  ) |>
  dplyr::arrange(Sex) |>
  tidyr::pivot_wider(names_from = quantity, values_from = cell)

tinytable::tt(tls_sex_table_abs, escape = TRUE)
```

The two `tls()` calls reproduce the by-hand calculations: the relative-threshold summary (Table S21) matches the relative columns of Table S20, and the absolute-threshold summary (Table S22) matches the absolute columns. Relative and absolute estimates barely differ here because the fitted asymptotes change little with temperature, so the asymmetry correction is nearly constant; `tls()` additionally returns T_{crit}, which is similar under both thresholds.

8.0.0.5 Fitting a model to compare z and $Ct_{max_{1hr}}$ between sexes directly - direct method

Now, lets refit this model in the **direct** form and estimate z and $CT_{max_{4hr}}$ for each of the sexes.

```r
# Now, lets parameterise the 4PL in 'direct mode'. Recall that the models above just have temp_
joint_sex_direct <- fit_4pl(std_sex,
                            ctmax = ~ 0 + sex,
                                z = ~ 0 + sex,
                              low = ~ temp_c,
                                k = ~ temp_c,
                               up = ~ temp_c,
                            t_ref = 60 * 4,
                           chains = 4, iter = 3000, warmup = 1000, cores = 4, seed = 123,
                           control = list(adapt_delta = 0.95),
                           file = file.path(models_dir, "fit_dsuzukii_mortality_sex_direct"),
                           file_refit = "on_change")
summary(joint_sex_direct)
```

```
 Family: beta_binomial
  Links: mu = identity
Formula: n_surv | trials(n_total) ~ low + (up - low)/(1 + exp(exp(logk) * (logd - mid)))
         low ~ (0.001 + inv_logit(lowraw) * 0.498)
         up ~ (0.501 + inv_logit(upraw) * 0.498)
         mid ~ (2.38021 - (temp_c - CTmaxdev)/exp(logz))
         lowraw ~ temp_c
         upraw ~ temp_c
         logk ~ temp_c
         CTmaxdev ~ 0 + sex
         logz ~ 0 + sex
   Data: data (Number of observations: 94)
  Draws: 4 chains, each with iter = 3000; warmup = 1000; thin = 1;
         total post-warmup draws = 8000

Regression Coefficients:
                Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
lowraw_Intercept    -4.12      0.74    -5.63    -2.77 1.00     7216     5530
lowraw_temp_c        0.17      0.42    -0.65     0.98 1.00     8416     5537
upraw_Intercept      0.80      0.19     0.44     1.20 1.00     7204     6353
upraw_temp_c        -0.20      0.12    -0.45     0.03 1.00     8028     6254
logk_Intercept       2.90      0.15     2.63     3.21 1.00     7427     5387
logk_temp_c          0.05      0.09    -0.13     0.22 1.00     7223     6360
CTmaxdev_sexF       -0.92      0.06    -1.04    -0.82 1.00     6441     5499
CTmaxdev_sexM       -0.89      0.05    -1.00    -0.79 1.00     6777     5843
logz_sexF            1.12      0.03     1.06     1.18 1.00     6568     5784
logz_sexM            1.17      0.03     1.10     1.23 1.00     7412     5908

Further Distributional Parameters:
    Estimate Est.Error l-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
phi    18.68      6.70     9.55    35.68 1.00     7183     5257

Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
```

```
and Tail_ESS are effective sample size measures, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

```
# Extract per-sex CTmax and z estimates from the direct model.
 direct_yls_dros <- tls(joint_sex_direct, by = "sex", target_surv = "absolute", t_ref = 60 * 4)
direct_joint_sex <- get_tls_est(direct_yls_dros)
```

Now, we can compare the different model parameterisations with Ørsted *et al.* (2024).

```
# Pull per-sex z and CTmax from each fit into one long frame. The summaries are
# all tidy-long (quantity / median / lower / upper); the separate fits carry a
# "Sex" column (Female/Male), the joint fits an "sex" column (F/M).
.cmp_rows <- function(s, approach) {
  sx <- if ("Sex" %in% names(s)) s$Sex else dplyr::recode(as.character(s$sex), F = "Female", M
  tibble::tibble(Approach = approach, Sex = sx, quantity = s$quantity,
                 median = s$median, lower = s$lower, upper = s$upper) |>
    dplyr::filter(quantity %in% c("z", "CTmax"))
}

cmp_models <- dplyr::bind_rows(
    .cmp_rows(dros_sex_sum,            "Separate per-sex fits"),
    .cmp_rows(tls_sex_summary_abs,     "Joint 4PL, sex on midpoint"),
    .cmp_rows(direct_yls_dros$summary, "Joint 4PL, sex on CTmax / z (direct)")
  ) |>
  dplyr::transmute(
    Approach,
    col  = factor(ifelse(quantity == "z",
                         paste0("z, ", Sex, " (°C)"),
                         paste0("CTmax, ", Sex, " (4 h, °C)")),
                  levels = c("z, Female (°C)", "z, Male (°C)",
                             "CTmax, Female (4 h, °C)", "CTmax, Male (4 h, °C)")),
    cell = format_interval(median, lower, upper)
  ) |>
  tidyr::pivot_wider(names_from = col, values_from = cell)

# Ørsted et al. (2024) reported point estimates (their Table 1); no intervals.
cmp_orsted <- tibble::tibble(
  Approach                  = "Ørsted et al. (2024), reported",
  `z, Female (°C)`          = "3.03",
  `z, Male (°C)`            = "3.28",
  `CTmax, Female (4 h, °C)` = "35.2",
  `CTmax, Male (4 h, °C)`   = "35.2"
)

tinytable::tt(dplyr::bind_rows(cmp_models, cmp_orsted), escape = TRUE)
```

All three parameterisations give effectively the same per-sex z and $CT_{max_{4hr}}$ — whether each sex

Table S23: Per-sex thermal sensitivity (z) and critical limit ($CT_{max_{4hr}}$, absolute LT_{50} threshold) for *D. suzukii* mortality across the three model parameterisations — separate per-sex 4PL fits, one joint 4PL with `sex` on the midpoint, and one joint 4PL with `sex` placed directly on CT_{max} and z — alongside the values reported by Ørsted *et al.* (2024). Model cells are the posterior median with the 95% credible interval; Ørsted et al. report point estimates (T_{crit} is omitted as they do not report it).

Approach	z, Female (°C)	CTmax, Female (4 h, °C)	z, Male (°C)	CTmax
Separate per-sex fits	3.04 [2.84, 3.26]	35.13 [34.99, 35.25]	3.16 [2.93, 3.43]	35.16 [3
Joint 4PL, sex on midpoint	3.04 [2.86, 3.24]	35.14 [35.02, 35.24]	3.18 [2.98, 3.4]	35.17 [3
Joint 4PL, sex on CTmax / z (direct)	3.04 [2.87, 3.24]	35.13 [35.01, 35.24]	3.19 [2.99, 3.41]	35.17 [3
Ørsted et al. (2024), reported	3.03	35.2	3.28	35.2

is fit separately, or compared in one joint model with `sex` on the midpoint or placed directly on CT_{max} and z — and all sit close to the values reported by Ørsted *et al.* (2024).

9 Case Study 4.2: Vinegar fly (*Drosophila suzukii*): Analysing sublethal measures

The *Drosophila* dataset also includes two sublethal endpoints: heat-coma time and reproductive output.

Heat-coma time can be analysed in two equivalent ways. The classical route fits a Bayesian hierarchical regression of $\log_{10}$(time to coma) on temperature and derives $z = -1/\beta_1$ and $CT_{max_{1hr}} = \bar{T} + (\log_{10} 60 - \beta_0)/\beta_1$ from the fitted coefficients. The joint route reformulates the same observations as proportion-awake counts and fits the 4PL, recovering the same quantities from the posterior. Below, we fit both models with `sex` as a moderator so one model estimates female- and male-specific parameters.

9.0.0.0.1 Knockdown: time to coma

Each fly was exposed for a fixed duration, scored as awake or in heat coma, and assigned a coma time if it lost the righting response during the exposure. Flies still awake at the end of the exposure had not yet reached the endpoint, so their coma times are **right-censored** at the exposure duration. Overall, 41% of flies are censored.

Censoring potentially matters because dropping censored flies would select for individuals that entered coma early and potentially bias the z estimate. We retain all flies in two complementary analyses. The **linear** model fits a right-censored log-time regression, treating the exposure duration of censored flies as a lower bound on their unobserved coma time. The **4PL** model instead analyses the binary awake/coma outcome as proportion-awake counts across exposure durations, matching the `awake ~ time` dose-response used by Ørsted *et al.* (2024).

```
# Prepare right-censored Drosophila heat-coma data for modelling.
# Use exposure duration as the right-censoring bound for awake flies.
coma_tbar <- mean(dsuzukii$temp)
coma_dat <- dsuzukii |>
  dplyr::mutate(
    in_coma    = !is.na(t_coma),
    coma_min   = ifelse(in_coma, t_coma, time),  # observed coma time, else exposure time
    log10_coma = log10(coma_min),
    cens       = ifelse(in_coma, "none", "right"),
    temp_c     = temp - coma_tbar,
    # the assay block shared by flies measured together (one temperature x
    # exposure-level x sex group); used as a random intercept below
    block      = interaction(temp, lvl, sex, drop = TRUE)
  )
```

The classical sublethal TDT model is a hierarchical log-time regression, with a censoring term and a `sex` interaction so females and males have separate slopes (z) and intercepts (CT_{max}). The `(1 | block)` random intercept absorbs variation shared by the flies assayed together in one temperature × exposure-level × sex block, to control for non-independence within a block:

```
# Fit the censored log-time heat-coma model by sex.
priors_coma_lin <- c(
  brms::prior(normal(2, 1.5), class = "Intercept"),
  brms::prior(normal(-1, 1),  class = "b"),
  brms::prior(exponential(2), class = "sd"),
  brms::prior(exponential(2), class = "sigma")
)

fit_coma_lin <- brms::brm(
  brms::bf(log10_coma | cens(cens) ~ temp_c * sex + (1 | block)),
  data       = coma_dat,
  prior      = priors_coma_lin,
  chains     = 4, iter = 6000, warmup = 3000, cores = 4, seed = 123,
  control    = list(adapt_delta = 0.95),
  backend    = "cmdstanr", silent = 2, refresh = 0,
  file       = file.path(models_dir, "fit_dsuzukii_coma_linear"),
  file_refit = "on_change"
)
```

```
# Derive z and CTmax from the censored heat-coma model.
# Females are the reference; males add the sex interaction.
post_coma_lin <- posterior::as_draws_df(fit_coma_lin)
slope_F_c <- post_coma_lin$b_temp_c
slope_M_c <- post_coma_lin$b_temp_c + post_coma_lin[["b_temp_c:sexM"]]
int_F_c   <- post_coma_lin$b_Intercept
int_M_c   <- post_coma_lin$b_Intercept + post_coma_lin$b_sexM
```

```
z_coma_lin_F  <- -1 / slope_F_c
z_coma_lin_M  <- -1 / slope_M_c
# The fitted model uses temp_c = temp - coma_tbar. The CTmax equation solves
# for temp_c, so add coma_tbar to express the threshold in degrees C.
ct_coma_lin_F <- coma_tbar + (log10(60) - int_F_c) / slope_F_c
ct_coma_lin_M <- coma_tbar + (log10(60) - int_M_c) / slope_M_c
```

Now, we could also think about these data differently and retain the use of the 4PL model. To take this path we reformulate the same data as proportion-awake counts for each temperature-by-exposure-by-sex block. We fit one joint model with `sex` on CT_{max} and z — the same structure as the mortality `joint_sex_fit` above, but here we write the `brms` model out **by hand** (rather than via `fit_4pl(ctmax = ... , z = ...)`) to show that `tls()` derives the per-sex quantities from *any* `brms` 4PL, not only from `fit_4pl()` workflows:

```
# Convert heat-coma observations into proportion-awake counts for the 4PL.
coma_panel <- dsuzukii |>
  dplyr::group_by(temp, lvl, sex) |>
  dplyr::summarise(n_total  = dplyr::n(),
                   n_awake  = sum(is.na(t_coma)),   # awake = not (yet) in coma
                   duration = dplyr::first(time),
                   .groups  = "drop") |>
  dplyr::filter(duration > 0)

# Standardise the data just to keep naming conventions consistent and t_ref
std_coma        <- standardize_data(coma_panel, temp = "temp", duration = "duration",
                                    n_total = "n_total", n_surv = "n_awake",
                                    duration_unit = "minutes")
# We need to set up the priors based on the formulas we plan to use. The `make_4pl_priors()` ca
priors_coma_4pl <- make_4pl_priors(std_coma, ctmax = ~ sex, z = ~ sex,
                                   low = ~ temp_c, up = ~ temp_c, k = ~ temp_c)
```

```
# Build the same sex-on-CTmax/z (direct) 4PL BY HAND with brms::bf() (rather
# than fit_4pl(ctmax = ..., z = ...)) to show tls() works on ANY
# brms 4PL. mid is written as a derived term via nlf(), so tls() still reads it.
coma_4pl_bf <- brms::bf(
  n_surv | trials(n_total) ~ low + (up - low) / (1 + exp(exp(logk) * (logd - mid))),  # Main 4
  brms::nlf(low ~ 0.001 + inv_logit(lowraw) * 0.498),                                 # Low par
  brms::nlf(up  ~ 0.501 + inv_logit(upraw)  * 0.498),                                 # Up para
  brms::nlf(mid ~ log10(60) - (temp_c - CTmaxdev) / exp(logz)),                       # 1 h re
        lowraw ~ temp_c,
         upraw ~ temp_c,
          logk ~ temp_c,
      CTmaxdev ~ sex,
          logz ~ sex,                  # sex on CTmax & z => sex-specific values
  nl = TRUE, family = brms::beta_binomial(link = "identity")
)
```

```r
fit_coma_4pl <- brms::brm(
  coma_4pl_bf, data = std_coma, prior = priors_coma_4pl,
  chains  = 4, iter = 4000, warmup = 2000, cores = 4, seed = 123,
  control = list(adapt_delta = 0.95),
  backend = "cmdstanr", silent = 2, refresh = 0,
  file    = file.path(models_dir, "fit_dsuzukii_coma_4pl"),
  file_refit = "on_change"
)
```

```r
# Extract sex-specific heat-coma TLS summaries from the 4PL.
set.seed(123)
# Report z and 1-hour CTmax by sex. fit_coma_4pl is a raw brmsfit (no bayesTLS
# metadata), so pass temp_mean explicitly -- tls() needs the centring temperature.
coma_4pl_tls <- tls(fit_coma_4pl, by = "sex", target_surv = "relative",
                    t_ref = 60, time_multiplier = 1,
                    temp_mean = mean(std_coma$temp - std_coma$temp_c))
```

```r
# Function for extracting summaries for the estimates
ctls <- function(est, sex) {
  r <- get_tls_est(coma_4pl_tls, "summary", est)   # tidy-long summary for quantity estimate
  r <- r[r$sex == sex, ]
  format_interval(r$median, r$lower, r$upper)
}

# Format function
fmt <- function(x) format_interval(stats::median(x),
                                   stats::quantile(x, .025, names = FALSE),
                                   stats::quantile(x, .975, names = FALSE))
# Build table
coma_compare <- tibble::tribble(
  ~Method,                          ~`z (F)`,            ~`z (M)`,            ~`CTmax 1h (F)`,
  "Linear (censored log-time)",     fmt(z_coma_lin_F),   fmt(z_coma_lin_M),   fmt(ct_coma_lin_F
  "Joint 4PL (proportion awake)",   ctls("z","F"),       ctls("z","M"),       ctls("CTmax","F")
  "Ørsted et al. (2024), drc",      "2.23",              "2.33",              "36.36",
)

tinytable::tt(coma_compare, escape = TRUE)
```

Both Bayesian routes agree and recover similar estimates to Ørsted *et al.* (2024) within their credible intervals (Table S24). Heat-coma sensitivity is $z \approx$ 2.3-2.4 °C and $CT_{max_{1hr}} \approx$ 36.3-36.5 °C for both sexes, with little evidence for sex differences.

9.0.0.0.2 Productivity: reproductive output

Reproductive output has two parts: *whether* a fly reproduced at all, and *how many* offspring it produced if it did. Such data are zero-inflated — many flies produce nothing — which a single curve

Table S24: Thermal sensitivity z and the 1-hour critical temperature $CT_{max_{1hr}}$ for *D. suzukii* heat coma, by sex, from the censored linear log-time regression and the joint 4PL on proportion-awake counts, compared with the two-stage drc estimates of Ørsted *et al.* (2024) (their static $CT_{max,1h}$). Cells are the posterior median with the 95% credible interval in brackets; the Ørsted values are point estimates.

Method	z (F)	z (M)	CTmax 1h (F)	CTmax 1h (M)
Linear (censored log-time)	2.34 [2.22, 2.48]	2.34 [2.23, 2.47]	36.47 [36.39, 36.55]	36.27 [36.19, 36.35]
Joint 4PL (proportion awake)	2.41 [2.25, 2.59]	2.4 [2.27, 2.57]	36.48 [36.36, 36.59]	36.27 [36.16, 36.37]
Ørsted et al. (2024), drc	2.23	2.33	36.36	36.31

handles poorly. A **hurdle model** deals with this by splitting the response into two pieces that are fit together: a yes/no part for whether a fly clears the "hurdle" of producing any offspring, and a count part for how many it produces once it has. Keeping the two pieces separate lets thermal stress act differently on *whether* a fly reproduces and on *how much* it produces.

The yes/no part connects directly to the rest of this paper: it is **just a simplified 4PL**. It describes the probability of reproducing as exposure lengthens with the same S-shaped logistic curve as the survival 4PL, only with the upper and lower asymptotes fixed at 1 and 0 (the 4PL with up = 1 and low = 0). So "whether a fly reproduces" is itself a thermal-tolerance curve, and we read its z and $CT_{max_{1hr}}$ from it exactly as we do for survival. The full 4PL simply restores the freedom for those asymptotes to differ from 0 and 1 — useful when, say, a fixed fraction of flies never reproduces no matter how brief the exposure. Given the asymptotes in the flies don't differ dramatically from 0/1 here, we will make this simplifying assumption. These same assumptions could be made in the 4PL models above. The count part (clutch size) is the genuinely new piece the hurdle adds.

As with the 4PL endpoints, productivity changes with exposure duration and temperature. Instead of fitting separate dose-response curves at each temperature and then regressing thresholds, we derive the thermal-death-time quantities directly from the single joint model using the duration and temperature coefficients for each hurdle component.

The count part — clutch size among the flies that did reproduce — can follow different distributions, so we try two (a Gamma and a log-normal) and keep whichever predicts the data better:

```
# Prepare Drosophila productivity data for the hurdle models.
prod_dat <- dsuzukii |>
  dplyr::mutate(logd = log10(time), temp_c = temp - mean(temp), sex = factor(sex))
```

```
# Fit and compare hurdle-Gamma and hurdle-lognormal productivity models.
# Allow both hurdle components to vary with exposure, temperature and sex.
prod_form <- brms::bf(prod ~ logd * temp_c * sex,   # clutch size | reproduced
                      hu   ~ logd * temp_c * sex)    # P(no offspring at all)

# Same formula and sampler settings for both fits; they differ only in the
# positive part's distribution (Gamma vs log-normal) and the cache file.
fit_prod_ga <- brms::brm(
  prod_form, data = prod_dat, family = brms::hurdle_gamma(),
```

Table S25: Leave-one-out predictive comparison of the two productivity models, which differ only in the positive (clutch-size) part. `loo_compare()` ranks them best-first; the ELPD difference is each model's expected log predictive density relative to the best (0), with the standard error of that difference. The Gamma positive part predicts the data better.

Positive part	ELPD difference	SE of difference
Gamma positive part	0	0.0
Log-normal positive part	-48	6.8

```
  chains = 4, iter = 3000, warmup = 1000, cores = 4, seed = 123,
  control = list(adapt_delta = 0.95), backend = "cmdstanr", silent = 2, refresh = 0,
  file = file.path(models_dir, "fit_dsuzukii_prod_hurdle_gamma"), file_refit = "on_change")

fit_prod_ln <- brms::brm(
  prod_form, data = prod_dat, family = brms::hurdle_lognormal(),
  chains = 4, iter = 3000, warmup = 1000, cores = 4, seed = 123,
  control = list(adapt_delta = 0.95), backend = "cmdstanr", silent = 2, refresh = 0,
  file = file.path(models_dir, "fit_dsuzukii_prod_hurdle_lognormal"), file_refit = "on_change")

# Compare the two by leave-one-out predictive fit.
fit_prod_ga <- brms::add_criterion(fit_prod_ga, "loo")
fit_prod_ln <- brms::add_criterion(fit_prod_ln, "loo")
prod_loo    <- brms::loo_compare(fit_prod_ga, fit_prod_ln)
```

The Gamma version predicts the data better than the log-normal — by 48 points on the leave-one-out predictive scale (7 standard errors) — so we use it (`fit_prod_ga`) from here on.

We read both reproductive summaries straight from the fitted model, one for each part of the hurdle. For *whether* a fly reproduces, we get the exposure time at which half the flies still reproduce and how that shifts with temperature — i.e. a thermal-tolerance curve. For *how many* offspring (among flies that did reproduce), we get how quickly clutch size falls as exposure lengthens. Both are computed from the fitted coefficients on every posterior draw, so they carry full uncertainty.

```
# Posterior draws of the hurdle-Gamma fit. The model has TWO sets of coefficients:
#   * b_hu_... -> the HURDLE part: a logistic model for P(zero offspring), i.e.
#                 whether a fly reproduces at all (the yes/no part).
#   * b_...    -> the GAMMA part: clutch size among flies that DID reproduce
#                 (the "how many" / count part).
# sex_coefs() below reads one set or the other (pre = "b_hu_" or "b_") to derive
# the TDT summaries analytically, draw by draw.
prod_post  <- posterior::as_draws_df(fit_prod_ga)
prod_tbar  <- mean(dsuzukii$temp)            # mean assay temperature (centring constant)
prod_temps <- sort(unique(dsuzukii$temp))
prod_tc    <- prod_temps - prod_tbar

# For one hurdle part (`pre = "b_"` for clutch size, `"b_hu_"` for the
```

```
# probability of zero offspring), the linear predictor for a given sex is
# I + T*temp_c + D*logd + DT*(logd*temp_c). sex_coefs() returns those four
# coefficients per posterior draw: females are the reference, and for males we
# add the matching `:sexM` interaction to each term.
sex_coefs <- function(post, pre, male) {

  # Pull one coefficient's posterior draws. Females are the reference (the base
  # term); for males, add the matching `:sexM` interaction to each term.
  base     <- function(term) post[[paste0(pre, term)]]
  male_add <- function(term) if (male) post[[paste0(pre, term)]] else 0

  # Return the four coefficients of the log-duration line, one value per draw.
  list(I  = base("Intercept")   + male_add("sexM"),
       T  = base("temp_c")      + male_add("temp_c:sexM"),
       D  = base("logd")        + male_add("logd:sexM"),
       DT = base("logd:temp_c") + male_add("logd:temp_c:sexM"))
}
```

Now, from the hurdle models posterior we can calculate z and $CT_{max_{1hr}}$ because we have a model that estimates a temperature, time and their interaction. These are all the ingredients we need. But, to understand how to calculate these values from a linear predictor we first need to recognise a few things.

First, we are, statistically speaking, fitting the following model (using the notation in the `sex_coefs()` function above):

$$\eta(\text{logd}, t) \;=\; I + T\,t + D\,\text{logd} + DT\,(\text{logd}\,t),$$

where

- η is the **linear predictor** — the quantity the model assembles from the coefficients below, which the logistic link then maps onto a probability,
- $\text{logd} = \log_{10}(\text{exposure duration in minutes})$ — the dose axis,
- $t = \tau - \bar{T}$ — temperature centred on the mean assay temperature $\bar{T}$ (so $t = 0$ at the mean, τ is the actual temperature),
- I, T, D, DT are the four posterior coefficients `sex_coefs()` returns: intercept, temperature effect, duration effect, and the duration × temperature interaction (`b_hu_Intercept`, `b_hu_temp_c`, `b_hu_logd`, `b_hu_logd:temp_c`, plus the matching `:sexM` terms for males).

However, this is the linear predictor. Remember, we are using a logistic link function so:

$$P(\text{reproduce}) = \text{logit}^{-1}(-\eta).$$

The natural halfway mark is the exposure at which half the flies still reproduce: $P(\text{reproduce}) = 0.5$. A logistic equals 0.5 **exactly when its input is zero**, so we set $\eta = 0$ and solve for logd, collecting the logd terms on one side:

$$I + T\,t + D\,\mathrm{logd} + DT\,(\mathrm{logd}\,t) = 0 \implies \mathrm{logd}\,(D + DT\,t) = -(I + T\,t) \implies \mathrm{logd}_{50}(t) = -\frac{I + T\,t}{D + DT\,t}.$$

That is the first formula: the $\log_{10}$ of the exposure time that halves reproduction, as a function of temperature.

z reflects **how many degrees results in a 10-fold change in tolerance time**. On the thermal-death-time plot ($\log_{10}$ time on the y-axis, temperature on the x-axis) the line slopes down, and z is defined so the line falls by **one** $\log_{10}$ unit — a factor of ten in time — every z degrees:

$$z(t) \;=\; -\frac{1}{\mathrm{dlogd}_{50}/\mathrm{d}t}.$$

(Because $t = \tau - \bar{T}$, a derivative in t equals a derivative in the absolute temperature τ, so z is per actual degree.)

Differentiate $\mathrm{logd}_{50}(t) = -(I + Tt)/(D + DT\,t)$ with the quotient rule. The key simplification: **the two terms that carry t cancel** in the numerator:

$$\frac{\mathrm{dlogd}_{50}}{\mathrm{d}t} = -\frac{T(D + DT\,t) - (I + Tt)\,DT}{(D + DT\,t)^2} = -\frac{\overbrace{TD + T\,DT\,t - I\,DT - T\,DT\,t}^{=T\,D - I\,DT}}{(D + DT\,t)^2} = -\frac{T\,D - I\,DT}{(D + DT\,t)^2}.$$

Take -1 over that. The minus from the derivative and the minus in $z = -1/(\cdot)$ **cancel**, leaving a positive expression:

$$\boxed{z(t) = \frac{(D + DT\,t)^2}{T\,D - I\,DT}}.$$

The above equation is the `z_at` object we use below. However, we make that calculation for every row of the posterior, which means that we get the full posterior distribution of z.

$CT_{max}(1\,\mathrm{h})$ is the temperature hot enough that a **one-hour exposure already drops reproduction to 50%**. So, we ask, at what temperature does logd_{50} equal $\log_{10}(60\ \mathrm{min})$? Set $\mathrm{logd}_{50}(t) = L_{\mathrm{ref}}$, where $L_{\mathrm{ref}} = \log_{10} 60$ is the **reference exposure** (one hour) expressed on the $\log_{10}$-minutes scale, and solve for t:

$$-\frac{I + Tt}{D + DT\,t} = L_{\mathrm{ref}} \implies -(I + Tt) = L_{\mathrm{ref}}(D + DT\,t) \implies -I - L_{\mathrm{ref}}D = t\,(T + L_{\mathrm{ref}}DT).$$

So the centred temperature is $t^\star = -(I + L_{\mathrm{ref}}D)/(T + L_{\mathrm{ref}}DT)$. Add the mean temperature back to land on the real scale:

$$\boxed{CT_{max} = \bar{T} - \frac{I + L_{\mathrm{ref}}D}{T + L_{\mathrm{ref}}DT}}, \qquad L_{\mathrm{ref}} = \log_{10} 60.$$

This is the code's `tbar - (cc$I + log10_tref*cc$D)/(cc$T + log10_tref*cc$DT)`. Again, remember we can make this calculation for every **row** of the joint posterior so we are getting a full posterior distribution of $CT_{max}(1\,\text{h})$.

```
# For a given sex and hurdle part the fitted linear predictor is a straight line
# in log-duration whose intercept and slope both shift with temperature:
#     I + T*t + (D + DT*t) * logd        (t = centred temperature).
# From this line we read the thermal summaries:
#   * INCIDENCE (hurdle part): there is a real 50% mark (half the flies still
#     reproducing), so a conventional z and CTmax exist --
#       - logd50(t) = -(I + T*t) / (D + DT*t)   : time at which P(reproduce) = 0.5
#       - z(t)      = -1 / (slope of logd50 in T): thermal sensitivity
#       - CTmax     : the temperature where logd50 = log10(60) (the 1-hour mark)
#     Both z(t) and CTmax have simple closed forms (logd50 is a rational function
#     of t), used below; z is averaged over the assay temperatures because the
#     interaction makes the line slightly curved.
#   * MAGNITUDE (Gamma part): clutch size has no 50% mark, so instead of z we
#     report b(T) = how fast clutch size falls as exposure lengthens.

hurd_tdt <- function(male = TRUE, post, tbar, tc, t_ref = 60) {

  # Transform so it's on same units, log10
  log10_tref <- log10(t_ref)

  # Extract the posterior distribution for the coefficients for the relevant sex
          cc <- sex_coefs(post, "b_hu_", male)  # whether-it-reproduces part

  # Calculate z over the assay range
     z_at    <- function(t) (cc$D + cc$DT * t)^2 / (cc$T * cc$D - cc$I * cc$DT)  # = -1 / slope
     z_draws <- vapply(tc, z_at, numeric(nrow(post)))  # draws x assay temperatures

  # Calculate CTmax 1hr: temperature where a 1-hour exposure drops reproduction to 50%
       ctmax <- tbar - (cc$I + log10_tref * cc$D) / (cc$T + log10_tref * cc$DT)

  # Return a list with z and ctmax
  list(    z  = rowMeans(z_draws),
       ctmax = ctmax)
}

prod_hurd_F <- hurd_tdt(FALSE, prod_post, prod_tbar, prod_tc)
prod_hurd_M <- hurd_tdt(TRUE,  prod_post, prod_tbar, prod_tc)

mag_b <- function(male = TRUE, post, tc) {

  # Extract the posterior distribution for the coefficients for the relevant sex
  cc <- sex_coefs(post, "b_", male)  # clutch-size (Gamma) part

  # Calculate b(T) at each assay temperature: how fast clutch size falls as
```

Table S26: Reproductive **incidence** thermal-death-time parameters for *D. suzukii* (whether a fly reproduces at all; the hurdle component of the hurdle-Gamma model), read analytically from the joint posterior, by sex, with the two-stage drc fecundity estimates of Ørsted *et al.* (2024) (static $CT_{max,1h}$) for comparison. z is the mean local z over the assay range. Cells are the posterior median with the 95% credible interval; Ørsted values are point estimates.

Source	z (F)	z (M)	CTmax 1h (F)	CTmax 1h (M)
Incidence (this study)	3.18 [2.9, 3.55]	2.88 [2.65, 3.17]	36.1 [35.9, 36.28]	36.48 [36.3, 36.66]
Ørsted et al. (2024), drc	3.27	3.46	36.27	36.66

```
  # exposure lengthens, b(T) = d log10(clutch) / d log10(exposure). The Gamma's
  # log link puts the log-duration slope (D + DT*t) on the natural-log scale, so
  # divide by log(10) to express it per log10-decade (factor-of-ten) of exposure.
  vapply(tc, function(t) (cc$D + cc$DT * t) / log(10), numeric(nrow(post)))
}

mag_b_F <- mag_b(FALSE, prod_post, prod_tc)
mag_b_M <- mag_b(TRUE,  prod_post, prod_tc)
```

```
# Format the Drosophila productivity incidence table.
fmt_q <- function(x) format_interval(stats::median(x, na.rm = TRUE),
                                     stats::quantile(x, .025, names = FALSE, na.rm = TRUE),
                                     stats::quantile(x, .975, names = FALSE, na.rm = TRUE))
prod_inc_tbl <- tibble::tribble(
  ~Source,                          ~`z (F)`,            ~`z (M)`,            ~`CTmax 1h (F)`,
  "Incidence (this study)",      fmt_q(prod_hurd_F$z), fmt_q(prod_hurd_M$z), fmt_q(prod_hurd_F$
  "Ørsted et al. (2024), drc",   "3.27",              "3.46",              "36.27",
)
tinytable::tt(prod_inc_tbl, escape = TRUE)
```

The *whether-it-reproduces* part behaves like a normal tolerance endpoint, because it has a natural halfway mark — the point where half the flies still reproduce. Its z is about 3.2 °C for females and 2.9 °C for males, with $CT_{max_{1hr}}$ from 36.1 to 36.5 °C — close to the fecundity values reported by Ørsted *et al.* (2024) (Table S26). The male and female intervals overlap, so we cannot confirm the male-greater-than-female z difference they report. (Exposure time and temperature interact, so the tolerance line bends slightly; we therefore report the average z across the tested temperatures.)

The *how-many-offspring* part answers a different question. Clutch size simply rises or falls — it has no 0–100% scale and so no natural halfway mark — so a conventional z does not apply. What we *can* read off is how steeply clutch size drops as exposure lengthens, $b(T) = \mathrm{d}\log_{10}(\text{clutch})/\mathrm{d}\log_{10}(\text{exposure})$, and this changes a lot with temperature (Table S27):

```
# Format the Drosophila productivity magnitude table.
prod_mag_tbl <- tibble::tibble(
  `Temp (°C)`      = prod_temps,
```

Table S27: Duration-sensitivity of the conditional clutch (offspring among flies that reproduced), $b(T) = \mathrm{d}\log_{10}(\text{clutch})/\mathrm{d}\log_{10}(\text{exposure})$, read from the hurdle-Gamma μ posterior at each assay temperature, by sex (posterior median). $P(\text{decline})$ is the posterior probability that $b(T) < 0$. The clutch is insensitive to exposure at 34 °C ($b \approx 0$) and increasingly reduced by it toward 38 °C.

Temp (°C)	b (F)	P(decline) (F)	b (M)	P(decline) (M)
34	0.02	0.45	-0.04	0.67
35	-0.14	0.90	-0.13	0.98
36	-0.30	1.00	-0.23	1.00
37	-0.46	1.00	-0.32	1.00
38	-0.62	1.00	-0.42	1.00

```
  `b (F)`           = round(apply(mag_b_F, 2, stats::median), 2),
  `P(decline) (F)` = round(apply(mag_b_F, 2, function(b) mean(b < 0)), 2),
  `b (M)`           = round(apply(mag_b_M, 2, stats::median), 2),
  `P(decline) (M)` = round(apply(mag_b_M, 2, function(b) mean(b < 0)), 2)
)
tinytable::tt(prod_mag_tbl, escape = TRUE)
```

Among flies that do reproduce, longer exposure barely changes clutch size at 34 °C ($b \approx 0$; the probability it declines is only 0.45 for females), but it cuts clutch size more and more at hotter temperatures, reaching $b \approx -0.62$ at 38 °C.

Splitting reproduction into these two parts shows something a single fecundity curve would blur together: heat makes flies less likely to reproduce at all, across every temperature we tested ($z \approx 3$ °C), while among the flies that still reproduce it shrinks the clutch mainly at the hotter end ($\gtrsim 36$ °C).

10 Extended Simulation Results

This section provides more detail on the simulation results from the various scenarios we explored to understand bias and coverage of z and CT_{max}. The main manuscript figures summarize bias estimates retrieved from the 4PL and two-stage approaches using two data-generating scenarios (shifting upper asymptote `up`, varying slope `k`).

Here we document the remaining scenarios we explored: the strict-equivalence baseline, likelihood misspecification alone, symmetric asymptote compression, and changes in experimental design structure and replication. We also present how shifts in upper asymptote and slope affects coverage, which was only partially reported in the main text.

For each scenario we also tabulate the root-mean-square error (RMSE, in °C) of z and $CT_{max,1h}$ — the typical magnitude of the estimation error. Because RMSE depends only on the point estimates, the Normal- and t-quantile two-stage CI variants share a value and are reported once each.

10.1 *Strict equivalence baseline*

For this simulation, the data-generating curve spans the full survival range (up = 0.999, low = 0.001), has a shared slope ($k = 8$), and has a midpoint that declines linearly with temperature (slope -0.15). We crossed this scenario with two replicate designs, $n_{\text{reps}} \in \{3, 5\}$, with $N_{\text{sim}} = 1000$ datasets per cell.

We included this scenario to check whether the estimators agree when the classical two-stage assumptions are satisfied. As expected, all methods recovered z and $CT_{max,1h}$ with little bias. The main difference was confidence/credible interval calibration: two-stage intervals based on Normal quantiles did not have nominal coverage, whereas the t-quantile delta-method intervals were close to the nominal 95% coverage. Here, it is important to recognise that we have ensured error from the two-stage pipeline is correctly propagated through the Delta method for comparison. However, in practice, error propagation is often ignored.

Of the 1,000 datasets per simulated scenario, models in the two-stage analytical approach failed to converge or resulted in extreme bias (i.e., $|z|$ bias above 20 °C or $|CT_{max,1h}|$ bias above 10 °C) for 0 binomial and 29–30 (2.9–3.0%) beta-binomial (glmmTMB) Stage-1 fits. In contrast, the joint 4PL failed in 0 fits.

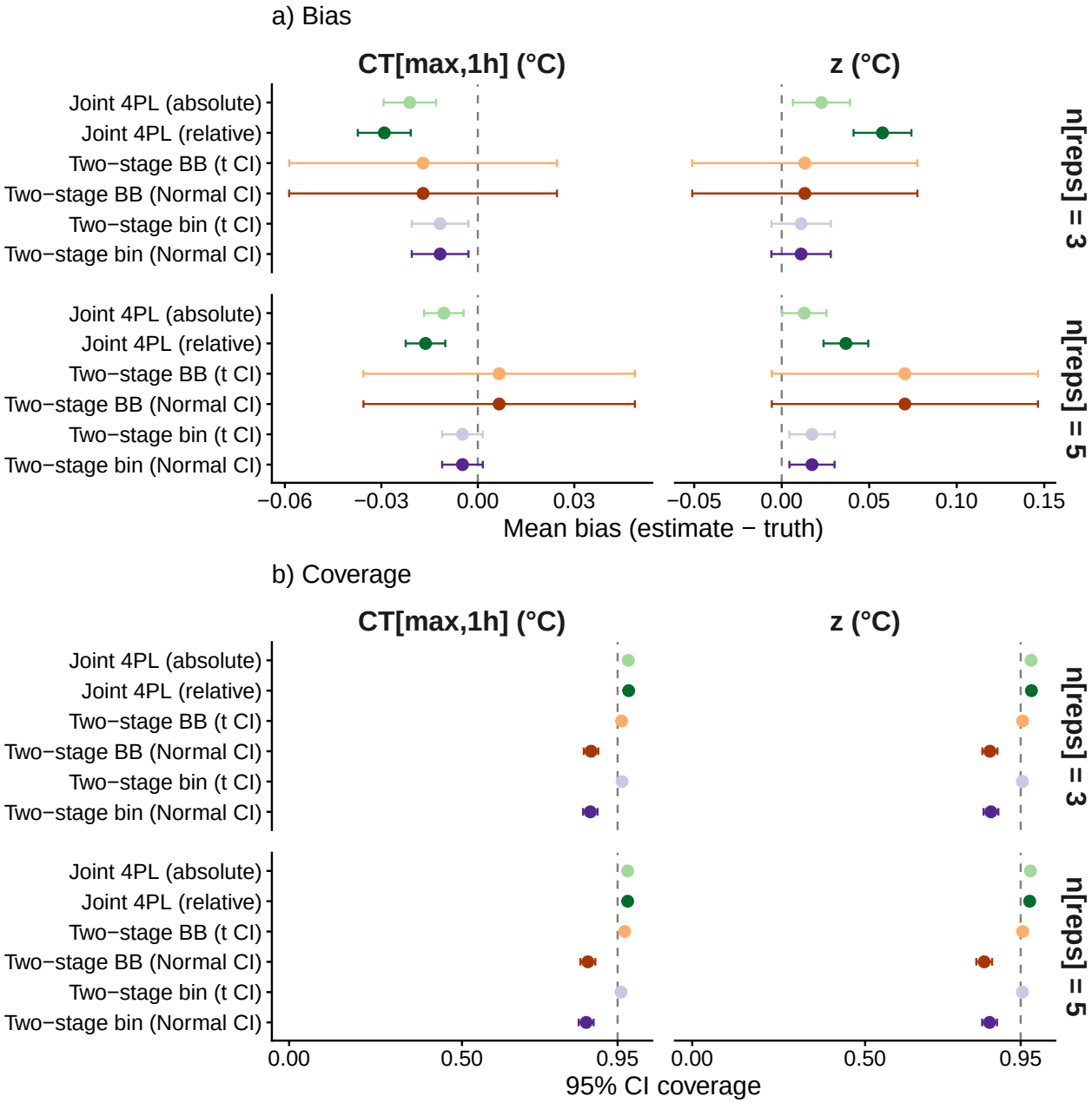


Figure S17: Bias and coverage at the strict-equivalence baseline (binomial data-generating process, asymptotes near 0/1, single slope k). Rows: $n_{reps} = 3$ (top) and $n_{reps} = 5$ (bottom). Columns: z (left) and $CT_{max,1h}$ (right). Dashed reference lines at zero bias and nominal 0.95 coverage.

```
# Format the RMSE table for simulation scenario 1.
.load_screened("scen1") |>
  dplyr::mutate(Cell = factor(sub(".*_n([0-9]+)$", "\\1", scenario),
                              levels = c("3", "5"))) |>
  .rmse_table(cell_name = "n_reps")
```

Table S28: RMSE (°C) of z and $CT_{max,1h}$ at the strict-equivalence baseline, by point estimator and level of replicates per treatment ($N_{sim} = 1000$ per cell). Lower is better; RMSE combines bias and precision on the parameter's own scale.

n_reps	Method	z (°C)	CTmax (°C)
3	Two-stage (binomial)	0.27	0.14
3	Two-stage (beta-binomial)	1.02	0.66
3	Joint 4PL (relative)	0.27	0.14
3	Joint 4PL (absolute)	0.26	0.13
5	Two-stage (binomial)	0.21	0.10
5	Two-stage (beta-binomial)	1.21	0.67
5	Joint 4PL (relative)	0.21	0.10
5	Joint 4PL (absolute)	0.21	0.10

10.2 *Likelihood misspecification alone*

This scenario keeps the same mean dose-response shape as the strict-equivalence baseline, but generates counts from a beta-binomial likelihood with overdispersion $\phi = 5$. We again used two levels of replication, $n_{\text{reps}} \in \{3, 5\}$, with $N_{\text{sim}} = 1000$ datasets per scenario.

We included this scenario to isolate the cost of likelihood misspecification: the mean structure is correct for all estimators, but the binomial two-stage variant ignores overdispersion at Stage 1. Point estimates remained largely unbiased, showing that overdispersion alone did not shift the target estimands. The effect was on uncertainty rather than location, and it was modest here: overdispersion alone barely separated the binomial and beta-binomial two-stage fits, in either point estimates or coverage. Both produced Normal-quantile intervals that under-covered and were brought close to nominal by the t-quantile correction, while the joint 4PL was well calibrated throughout.

Of the 1,000 datasets, models in the two-stage analytical approach failed to fit or resulted in extreme bias for 0 binomial and 2–7 (0.2–0.7%) beta-binomial (glmmTMB) Stage-1 fits, versus 0 for the joint 4PL, across the simulations.

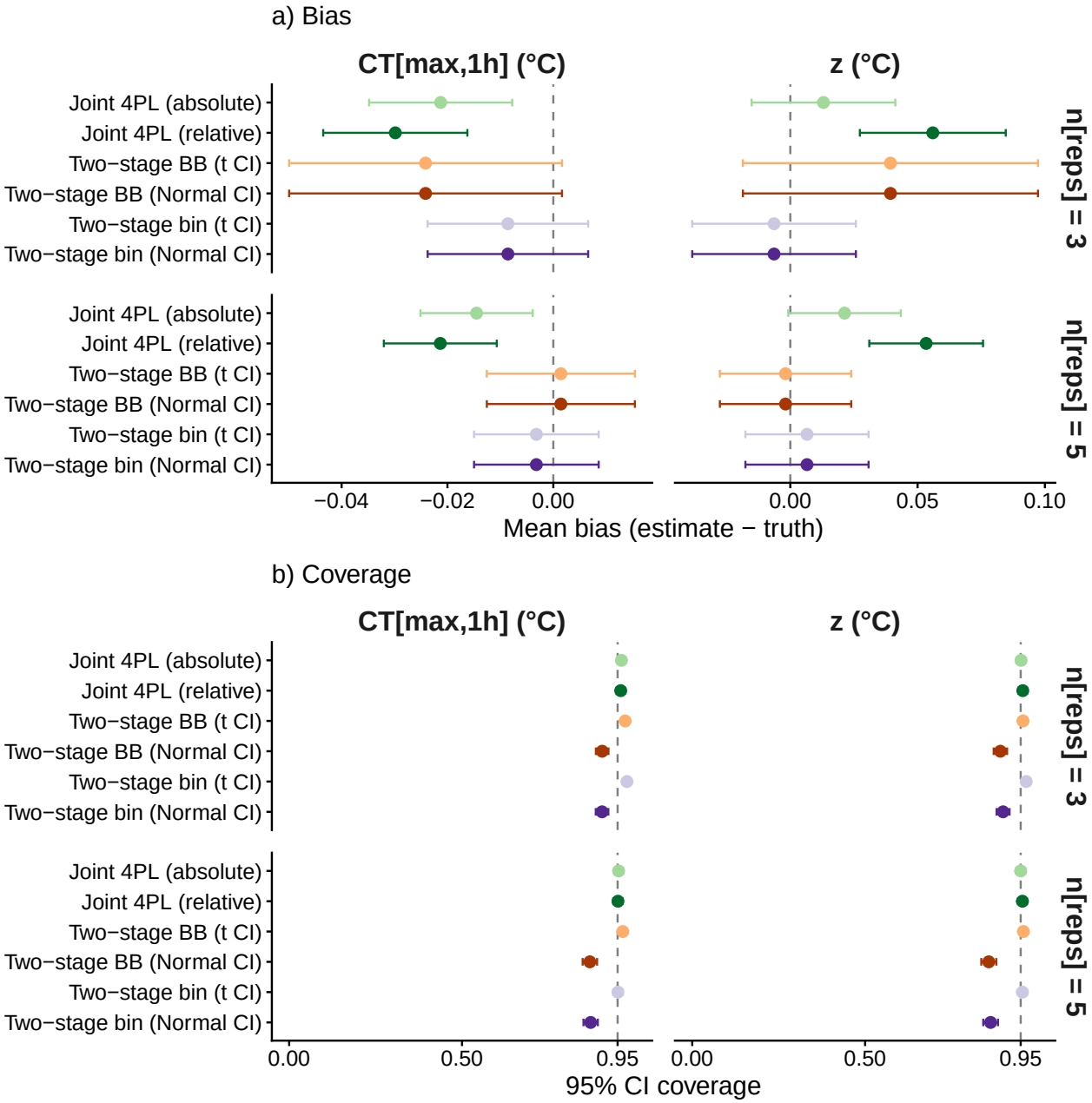


Figure S18: Bias and coverage with likelihood misspecification only (beta-binomial data-generating process with $\phi = 5$; curve shape as in the strict-equivalence baseline). Rows: $n_{reps} = 3$ (top) and $n_{reps} = 5$ (bottom).

```
# Format the RMSE table for simulation scenario 2.
.load_screened("scen2") |>
  dplyr::mutate(Cell = factor(sub(".*_n([0-9]+)$", "\\1", scenario),
                              levels = c("3", "5"))) |>
  .rmse_table(cell_name = "n_reps")
```

Table S29: RMSE (°C) of z and $CT_{max,1h}$ under likelihood misspecification only (beta-binomial data-generating process, $\phi = 5$; curve shape as in the baseline), by point estimator and replication budget.

n_reps	Method	z (°C)	CTmax (°C)
3	Two-stage (binomial)	0.52	0.24
3	Two-stage (beta-binomial)	0.93	0.41
3	Joint 4PL (relative)	0.47	0.22
3	Joint 4PL (absolute)	0.46	0.22
5	Two-stage (binomial)	0.39	0.19
5	Two-stage (beta-binomial)	0.42	0.23
5	Joint 4PL (relative)	0.36	0.17
5	Joint 4PL (absolute)	0.36	0.17

10.3 *Asymptotes compress symmetrically*

In this scenario, both asymptotes move toward each other as temperature increases: `up` declines by 0.01 per °C from 0.92, while low increases by 0.01 per °C from 0.05. This preserves the approximate logit-difference but compresses the response range. Although this symmetric compression is not biologically realistic, it is useful as a diagnostic test because it changes the dose-response range while largely preserving the midpoint-temperature relationship. We used two levels of replication, $n_{\text{reps}} \in \{3, 5\}$, with $N_{\text{sim}} = 1000$ datasets.

The two-stage fits showed clear bias in this scenario. Bias was negative in $CT_{max,1h}$ and positive in z. In addition, coverage was poor because the classical model cannot represent changing asymptotes directly. The joint 4PL remained close to unbiased with near-nominal coverage.

Of the 1,000 datasets, models in the two-stage analytical approach failed to fit or resulted in extreme bias for 0 binomial and 0 beta-binomial (glmmTMB) Stage-1 fits, versus 0 for the joint 4PL, across the simulations.

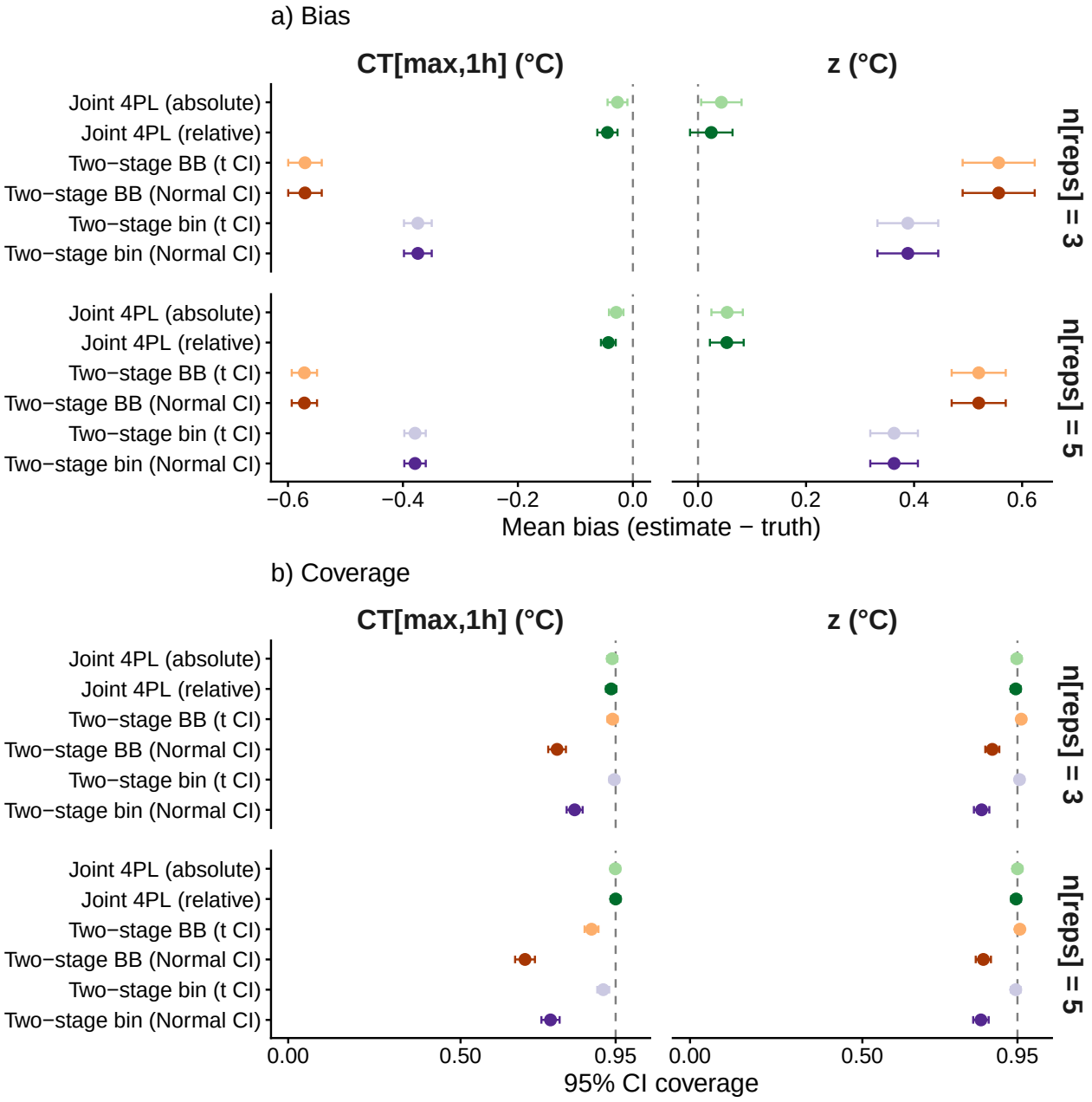


Figure S19: Bias and coverage when both asymptotes compress symmetrically with temperature. Rows: $n_{reps} = 3$ (top) and $n_{reps} = 5$ (bottom).

```
# Format the RMSE table for simulation scenario 4.
.load_screened("scen4") |>
  dplyr::mutate(Cell = factor(sub(".*_n([0-9]+)$", "\\1", scenario),
                              levels = c("3", "5"))) |>
  .rmse_table(cell_name = "n_reps")
```

Table S30: RMSE (°C) of z and $CT_{max,1h}$ when both asymptotes compress symmetrically with temperature, by point estimator and replication budget.

n_reps	Method	z (°C)	CTmax (°C)
3	Two-stage (binomial)	0.99	0.54
3	Two-stage (beta-binomial)	1.21	0.74
3	Joint 4PL (relative)	0.63	0.29
3	Joint 4PL (absolute)	0.60	0.28
5	Two-stage (binomial)	0.80	0.48
5	Two-stage (beta-binomial)	0.96	0.67
5	Joint 4PL (relative)	0.51	0.21
5	Joint 4PL (absolute)	0.47	0.21

10.4 *Coverage for shifting up across temperatures (β_{up})*

This figure complements Figure 3b in the main-text, where the upper asymptote varies with temperature with a coefficient β_{up}.

Of the 1,000 datasets, models in the two-stage analytical approach failed to fit or resulted in extreme bias for 0 binomial and 0–1 (0.0–0.1%) beta-binomial (glmmTMB) Stage-1 fits, versus 0 for the joint 4PL, across the simulations.

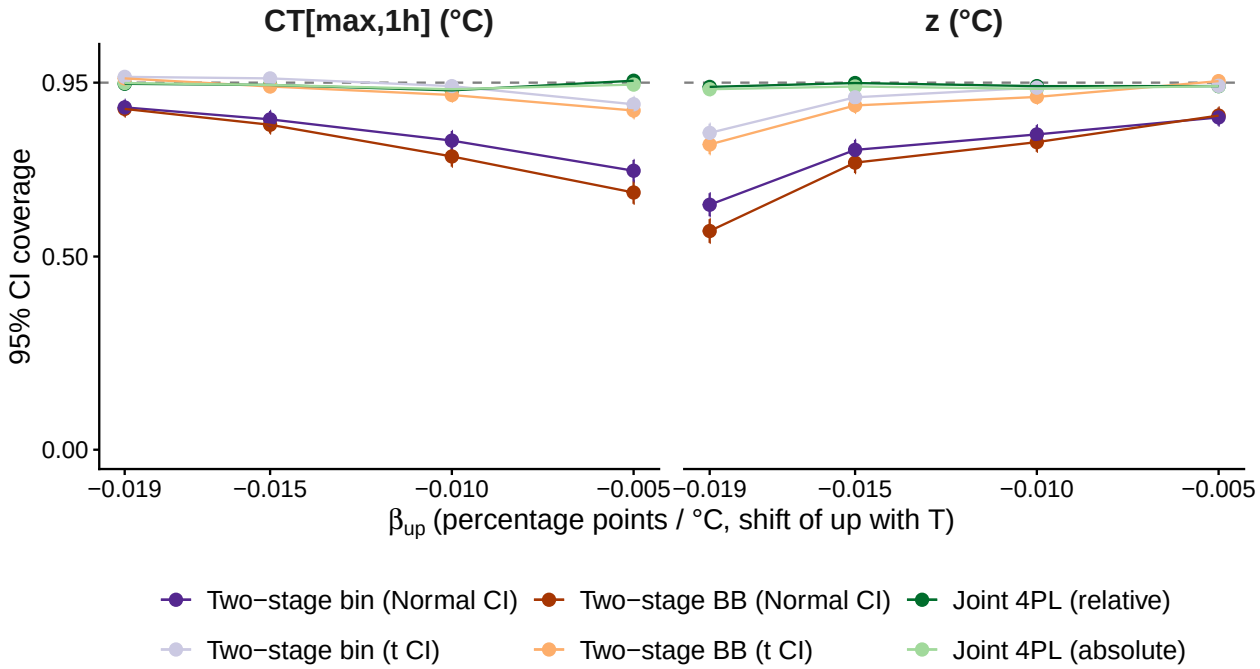


Figure S20: 95% CI coverage when the upper asymptote value varies with temperature (Scen 6). Left: z. Right: $CT_{max,1h}$. Dashed line at nominal 0.95. Same data generating process and method as in Figure 3. Normal- and t-quantile two-stage CIs differ in coverage even though their point estimates are identical.

Table S31: RMSE (°C) of z and $CT_{max,1h}$ when the upper asymptote decreases with temperature (Scen 6), by point estimator. β_{up} is the per-°C shift of the upper asymptote (percentage points per °C).

beta_up (pp/°C)	Method	z (°C)	CTmax (°C)
-0.019	Two-stage (binomial)	0.69	0.25
-0.019	Two-stage (beta-binomial)	0.78	0.28
-0.019	Joint 4PL (relative)	0.52	0.22
-0.019	Joint 4PL (absolute)	0.47	0.20
-0.015	Two-stage (binomial)	0.60	0.30
-0.015	Two-stage (beta-binomial)	0.66	0.35
-0.015	Joint 4PL (relative)	0.49	0.22
-0.015	Joint 4PL (absolute)	0.45	0.21
-0.010	Two-stage (binomial)	0.59	0.37
-0.010	Two-stage (beta-binomial)	0.63	0.45
-0.010	Joint 4PL (relative)	0.50	0.23
-0.010	Joint 4PL (absolute)	0.46	0.22
-0.005	Two-stage (binomial)	0.65	0.49
-0.005	Two-stage (beta-binomial)	0.71	0.62
-0.005	Joint 4PL (relative)	0.52	0.23
-0.005	Joint 4PL (absolute)	0.47	0.23

```
# Format the RMSE table for simulation scenario 6.
.load_screened("scen6") |>
  dplyr::mutate(bu   = -as.numeric(sub("scen6_ub_m", "", scenario)) / 1000,
                Cell = factor(sprintf("%.3f", bu),
                              levels = sprintf("%.3f", sort(unique(bu))))) |>
  .rmse_table(cell_name = "beta_up (pp/°C)")
```

10.5 *Coverage for constant-but-reduced* up *(up$_0$)*

This figure complements Figure 3a in the main-text, with variation in the value of the upper asymptote. With a low up$_0$, the two-stage Stage-1 GLMs miscalibrate per temperature, the Stage-2 OLS inherits that bias, and CI coverage on $CT_{max,1h}$ collapses well below nominal even when using the t-quantile small-sample correction.

Of the 1,000 datasets, models in the two-stage analytical approach failed to fit or resulted in extreme bias for 0–89 (0.0–8.9%) binomial and 0–101 (0.0–10.1%) beta-binomial (glmmTMB) Stage-1 fits, versus 0 for the joint 4PL, across the simulations.

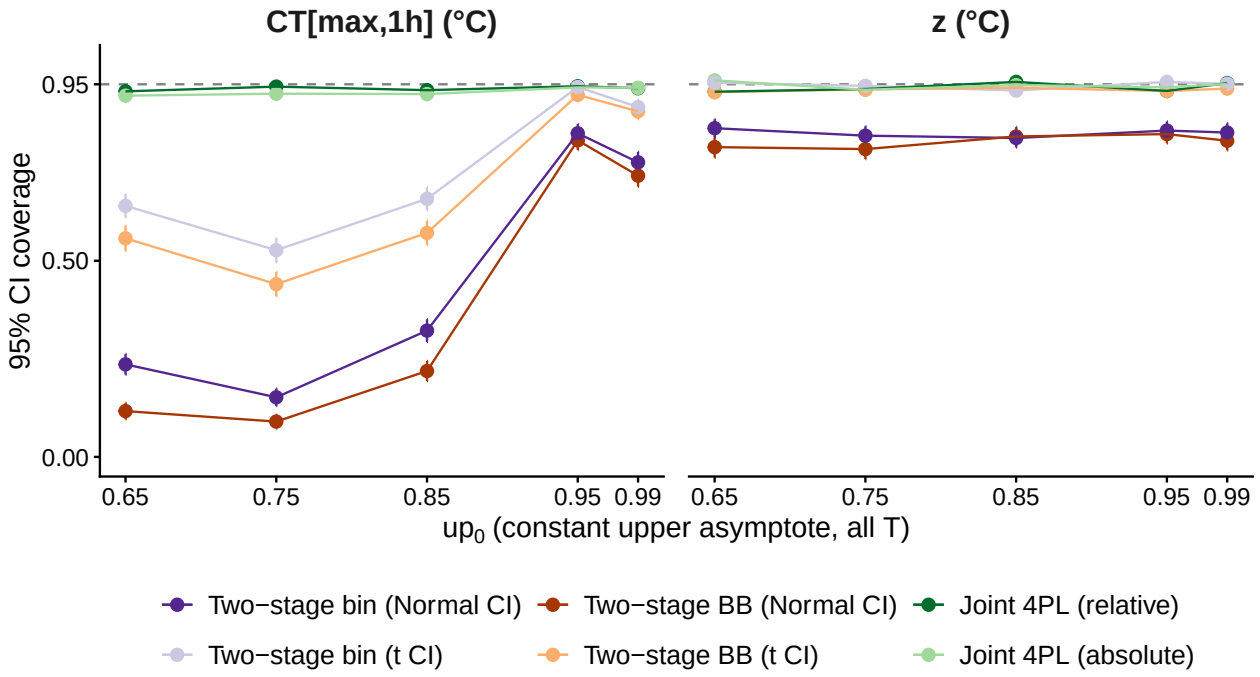


Figure S21: 95% CI coverage when the upper asymptote value varies (Scen 7). Same data generating process and method as in Figure 3. Two-stage CTmax coverage collapses as up_0 departs from 1; the t-correction widens CIs without recovering nominal calibration. Joint 4PL coverage holds at 92–96% throughout.

```
# Format the RMSE table for simulation scenario 7.
.load_screened("scen7") |>
  dplyr::mutate(u0   = as.numeric(sub("scen7_u0_", "", scenario)) / 100,
                Cell = factor(sprintf("%.2f", u0),
                              levels = sprintf("%.2f", sort(unique(u0))))) |>
  .rmse_table(cell_name = "up_0")
```

10.6 *Experimental Design Impacts*

10.6.0.1 *Different temperature time duration levels*

This scenario varies experimental design while holding the data-generating process fixed. We crossed two temperature-duration grids (full: 5 temperatures × 6 durations; sparse: 3 temperatures × 4 durations) with three levels of replication, $n_{\text{reps}} \in \{1, 3, 5\}$. Here, n_{reps} is the number of replicate cups per temperature-by-duration treatment. Each cup contained $N \sim \text{Uniform}\{10, \ldots, 20\}$ organisms. The data-generating process used the baseline beta-binomial shape (up = 0.92, low = 0.05, $k = 8$, midpoint linear in T, $\phi = 5$) and the sparsest design had only 12 'cups' total.

We included this scenario to test how much information each estimator needs before the two-stage reduction becomes unstable. Performance improved with denser designs and more replication, but the sparse one-replicate design was especially problematic for the two-stage pipeline. The joint 4PL was more stable across the design gradient because it shares information across temperatures and durations rather than estimating each temperature-specific curve separately.

Table S32: RMSE (°C) of z and $CT_{max,1h}$ when the upper asymptote value differs from 1 (Scen 7), by point estimator. up_0 is the constant upper asymptote held across all temperatures.

up_0	Method	z (°C)	CTmax (°C)
0.65	Two-stage (binomial)	7.90	9.28
0.65	Two-stage (beta-binomial)	3.15	5.94
0.65	Joint 4PL (relative)	1.03	0.60
0.65	Joint 4PL (absolute)	1.39	0.73
0.75	Two-stage (binomial)	1.86	2.94
0.75	Two-stage (beta-binomial)	2.15	3.54
0.75	Joint 4PL (relative)	0.78	0.37
0.75	Joint 4PL (absolute)	0.86	0.48
0.85	Two-stage (binomial)	1.17	1.38
0.85	Two-stage (beta-binomial)	1.40	1.78
0.85	Joint 4PL (relative)	0.58	0.30
0.85	Joint 4PL (absolute)	0.57	0.31
0.95	Two-stage (binomial)	0.73	0.35
0.95	Two-stage (beta-binomial)	0.87	0.43
0.95	Joint 4PL (relative)	0.47	0.23
0.95	Joint 4PL (absolute)	0.45	0.22
0.99	Two-stage (binomial)	0.61	0.29
0.99	Two-stage (beta-binomial)	0.76	0.34
0.99	Joint 4PL (relative)	0.43	0.21
0.99	Joint 4PL (absolute)	0.41	0.20

Of the 1,000 datasets, models in the two-stage analytical approach failed to fit or resulted in extreme bias for 0–37 (0.0–3.7%) binomial and 0–63 (0.0–6.3%) beta-binomial (glmmTMB) Stage-1 fits, versus 0 for the joint 4PL, across the simulations.

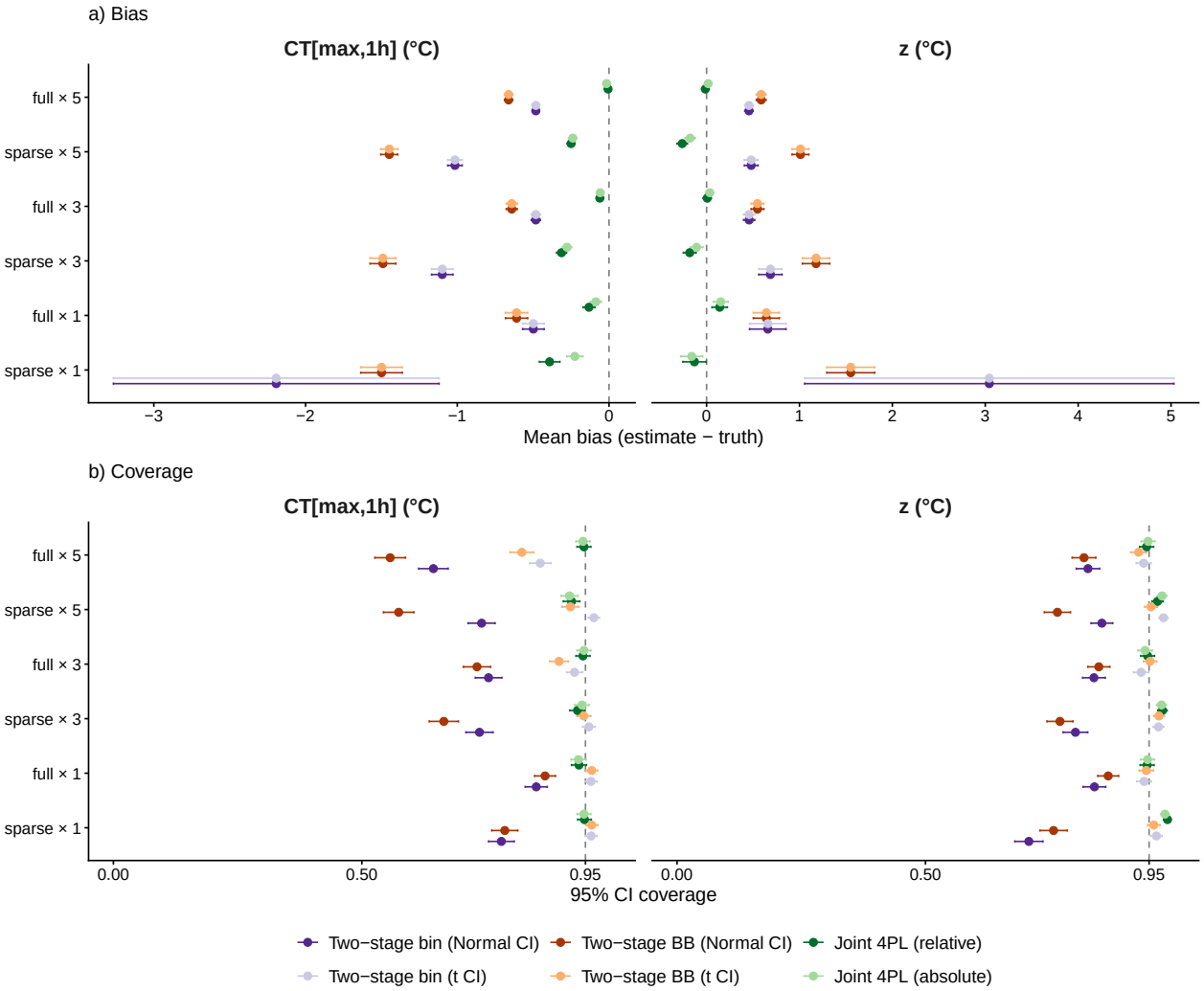


Figure S22: Bias and coverage across six design × replication cells (Scen 8). Two-stage at sparse × 1 produces large, unstable estimates with implausibly wide t-corrected CIs; joint 4PL returns coherent intervals of appropriate width across all six cells.

```
# Format the RMSE table for simulation scenario 8.
.load_screened("scen8") |>
  dplyr::mutate(design = ifelse(grepl("full", scenario), "full", "sparse"),
                n_reps = as.integer(sub(".*_n([0-9]+)$", "\\1", scenario)),
                Cell   = factor(sprintf("%s x %d", design, n_reps),
                                levels = c("sparse x 1", "full x 1",
                                           "sparse x 3", "full x 3",
                                           "sparse x 5", "full x 5"))) |>
  .rmse_table(cell_name = "Design x reps")
```

Table S33: RMSE (°C) of z and $CT_{max,1h}$ across the six design × replication cells (Scen 8), by point estimator. The sparse, single-replicate design is the most demanding for the two-stage reduction.

Design x reps	Method	z (°C)	CTmax (°C)
sparse x 1	Two-stage (binomial)	32.04	17.36
sparse x 1	Two-stage (beta-binomial)	4.32	2.61
sparse x 1	Joint 4PL (relative)	2.00	1.16
sparse x 1	Joint 4PL (absolute)	1.91	0.88
full x 1	Two-stage (binomial)	3.24	1.25
full x 1	Two-stage (beta-binomial)	2.34	1.32
full x 1	Joint 4PL (relative)	1.34	0.64
full x 1	Joint 4PL (absolute)	1.30	0.59
sparse x 3	Two-stage (binomial)	2.13	1.58
sparse x 3	Two-stage (beta-binomial)	2.65	2.02
sparse x 3	Joint 4PL (relative)	1.12	0.62
sparse x 3	Joint 4PL (absolute)	1.11	0.56
full x 3	Two-stage (binomial)	1.07	0.68
full x 3	Two-stage (beta-binomial)	1.24	0.87
full x 3	Joint 4PL (relative)	0.70	0.34
full x 3	Joint 4PL (absolute)	0.67	0.33
sparse x 5	Two-stage (binomial)	1.28	1.27
sparse x 5	Two-stage (beta-binomial)	1.77	1.71
sparse x 5	Joint 4PL (relative)	0.94	0.48
sparse x 5	Joint 4PL (absolute)	0.90	0.46
full x 5	Two-stage (binomial)	0.86	0.61
full x 5	Two-stage (beta-binomial)	1.03	0.79
full x 5	Joint 4PL (relative)	0.50	0.24
full x 5	Joint 4PL (absolute)	0.48	0.23

10.6.0.2 *Performance when estimating under short timeframes*

Thermal-tolerance assays are often limited by how long they can run. Long exposures at cooler assay temperatures are costly, so experiments may stop before those treatments reach their LT50. This scenario isolates that cost. We held the data-generating process fixed at the baseline beta-binomial shape (up = 0.92, low = 0.05, $k = 8$, midpoint linear in T, $\phi = 5$), used the full five-temperature design (30–38 °C) with $n_{\text{reps}} = 5$, and varied only the maximum exposure duration. Each design used six log-spaced durations capped at 60, 120, 240, or 405 min; the 405-min cap reproduces the full set of durations. Because the truth is identical across caps, changes in bias, coverage, and RMSE reflect only how well each design recovers the same z and $CT_{max_{1hr}}$.

Shorter windows remove the long-duration observations that anchor the cooler-temperature LT50s. At the shortest caps, the coolest temperatures may never reach 50% mortality, so the LT50 must be extrapolated rather than observed. This scenario therefore tests whether each estimator degrades as duration information is withdrawn, and whether the joint 4PL retains calibration better than the two-stage reduction by borrowing strength across temperatures.

Across the 1,000 datasets, the two-stage Stage-1 fits failed or produced extreme bias for 0–281 (0.0–28.1%) binomial and 0–287 (0.0–28.7%) beta-binomial (glmmTMB) fits, compared with 0 for the joint 4PL.

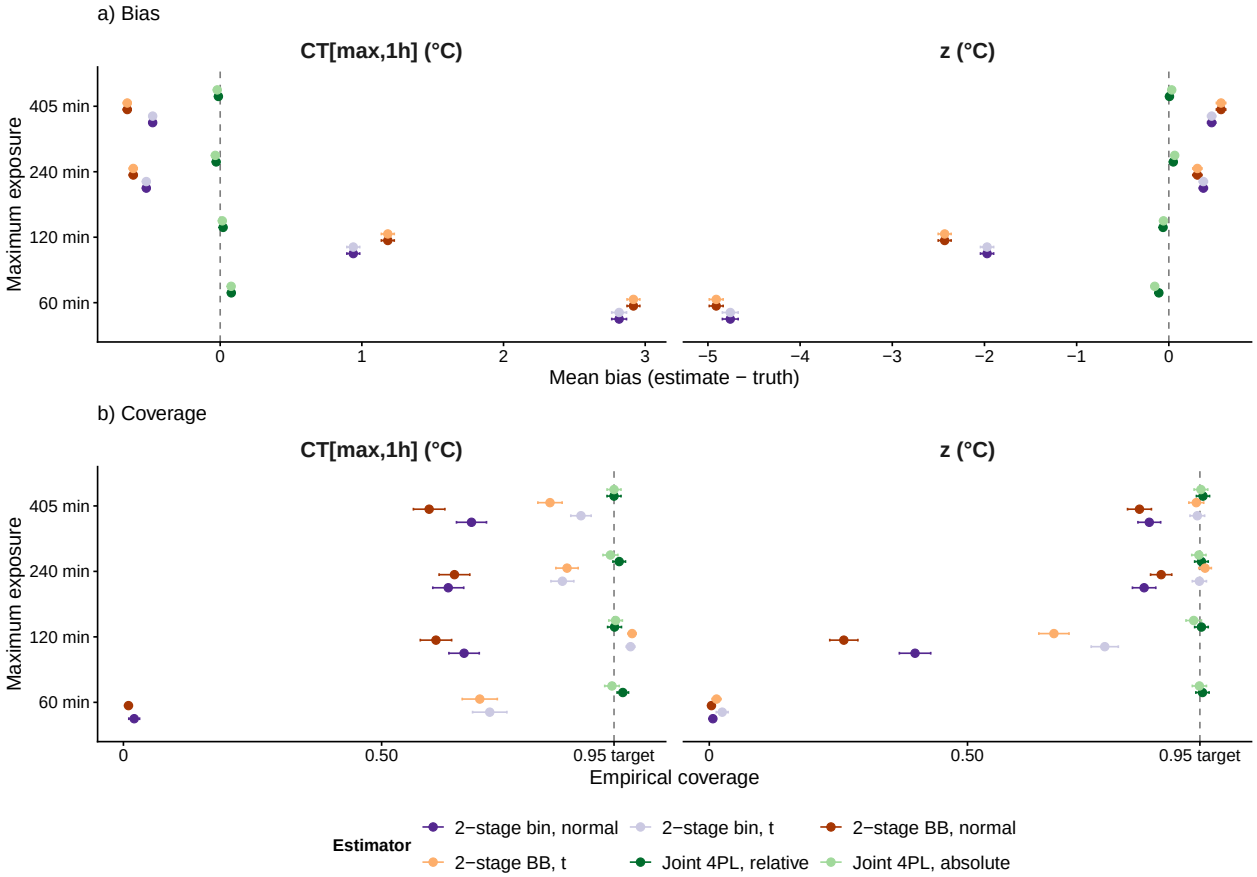


Figure S23: Effect of maximum exposure duration on estimator performance (Scen 9). Rows show mean bias (a; estimate minus truth) and 95% interval coverage (b) for z and $CT_{max_{1hr}}$. The 405-min cap is the full design; shorter caps progressively remove the long-duration observations that anchor LT50 at cooler assay temperatures.

```
# Format the RMSE table for simulation scenario 9.
.load_screened("scen9") |>
  dplyr::mutate(tmax = as.integer(sub(".*_tmax_([0-9]+)$", "\\1", scenario)),
                Cell = factor(sprintf("%d min", tmax),
                              levels = c("60 min", "120 min",
                                         "240 min", "405 min"))) |>
  .rmse_table(cell_name = "Max exposure")
```

11 Manuscript Figure 5: cross-case-study summary

This figure compiles the joint Bayesian 4PL posterior distributions of the TDT quantities (z and CT_{max}) from the four case studies into a single multi-panel display. Every panel is a group comparison drawn from a single joint fit: three cereal-aphid species (panel a), snow gum under dark vs light post-heat recovery (panel b), zebrafish across two oxygen treatments (panel c), and the vinegar fly by sex (panel d). The CT_{max} reference exposure follows each case study's own analysis

Table S34: RMSE (°C) of z and $CT_{max,1h}$ by maximum exposure duration (Scen 9). Lower RMSE indicates more accurate point estimates. The 60-min cap is the most demanding because the coolest assay temperatures may not reach 50% mortality within the observation window.

Max exposure	Method	z (°C)	CTmax (°C)
60 min	Two-stage (binomial)	4.90	2.90
60 min	Two-stage (beta-binomial)	5.02	2.98
60 min	Joint 4PL (relative)	0.57	0.28
60 min	Joint 4PL (absolute)	0.55	0.28
120 min	Two-stage (binomial)	2.28	1.19
120 min	Two-stage (beta-binomial)	2.68	1.39
120 min	Joint 4PL (relative)	0.49	0.22
120 min	Joint 4PL (absolute)	0.46	0.20
240 min	Two-stage (binomial)	0.79	0.64
240 min	Two-stage (beta-binomial)	0.86	0.76
240 min	Joint 4PL (relative)	0.51	0.22
240 min	Joint 4PL (absolute)	0.48	0.21
405 min	Two-stage (binomial)	0.85	0.60
405 min	Two-stage (beta-binomial)	0.99	0.79
405 min	Joint 4PL (relative)	0.51	0.24
405 min	Joint 4PL (absolute)	0.48	0.24

— 1 h for aphids, snow gum, and zebrafish, 4 h for the vinegar fly — so the right-column strips are labelled $CT_{max_{1hr}}$ and $CT_{max_{4hr}}$ respectively; the panels carry independent x-axes, so the columns are not on a shared scale.

```
# Assemble the manuscript cross-case posterior-density figure from the four joint
# 4PL case-study fits. Per-group z and CTmax draws come from the tidy-long tls()
# output (aphids, leaf, zebrafish) or the per-sex tls() objects (fly).
suppressPackageStartupMessages({
  library(ggplot2); library(dplyr); library(tibble); library(ggridges)
  library(patchwork); library(ggtext); library(cowplot); library(magick); library(here)
})

# A tls() result -> fig4 long draws + summary, relabelling CTmax to the study's
# reference (1 h here). `modcol` is the moderator column name in the tls output.
tls_to_fig4 <- function(tls_obj, modcol, case, ctmax_lab = "CTmax_1hr") {
  relab <- function(q) ifelse(q == "z", "z", ctmax_lab)
  dr <- tls_obj$draws |>
    dplyr::filter(quantity %in% c("z", "CTmax")) |>
    dplyr::transmute(case_study = case, group = as.character(.data[[modcol]]),
                     quantity = relab(quantity), value, .draw)
  sm <- tls_obj$summary |>
    dplyr::filter(quantity %in% c("z", "CTmax")) |>
    dplyr::transmute(case_study = case, group = as.character(.data[[modcol]]),
                     quantity = relab(quantity), median, lower, upper)
  list(draws = dr, summary = sm)
}

aphid_f4 <- tls_to_fig4(aphid_tls, "species",  "Cereal aphids")
leaf_f4  <- tls_to_fig4(leaf_tls,  "recovery", "Snow gum leaf PSII")
zf_f4    <- tls_to_fig4(zf_tls,    "oxygen",   "Zebrafish (oxygen)")

# Vinegar fly mortality, by sex: separate per-sex fits (absolute 4 h threshold).
fly_draws <- dplyr::bind_rows(
    Female = get_tls_est(tdt_dros_f, "draws", c("z", "CTmax")),
    Male   = get_tls_est(tdt_dros_m, "draws", c("z", "CTmax")),
    .id    = "group"
  ) |>
  dplyr::transmute(
    case_study = "Vinegar fly (mortality)",
    group,
    quantity = ifelse(quantity == "CTmax", "CTmax_4hr", "z"),
    value, .draw
  )
fly_summary <- dros_sex_sum |>
  dplyr::filter(quantity %in% c("z", "CTmax")) |>
  dplyr::transmute(
    case_study = "Vinegar fly (mortality)",
    group      = Sex,
```

```
    quantity   = ifelse(quantity == "CTmax", "CTmax_4hr", "z"),
    median, lower, upper
  )

fig4_draws <- dplyr::bind_rows(aphid_f4$draws, leaf_f4$draws, zf_f4$draws, fly_draws) |>
  dplyr::filter(is.finite(value))
# Panel a only: trim the aphid display tails to the central 98% per group so its
# axes stay tight and the contrast brackets sit cleanly (the 95% CIs are still
# drawn explicitly). Other case studies (b/c/d) are left untouched.
fig4_draws <- fig4_draws |>
  dplyr::group_by(case_study, group, quantity) |>
  dplyr::filter(case_study != "Cereal aphids" |
                (value <= stats::quantile(value, 0.99) &
                 value >= stats::quantile(value, 0.01))) |>
  dplyr::ungroup()
fig4_summary <- dplyr::bind_rows(aphid_f4$summary, leaf_f4$summary, zf_f4$summary, fly_summary)

# Persist the per-group z/CTmax medians so the manuscript can report figures
# (e.g. the CTmax_1hr range across case studies) as inline code, not magic
# numbers. Read back in ms/ms.qmd.
dir.create(here::here("output", "data"), showWarnings = FALSE, recursive = TRUE)
saveRDS(fig4_summary, here::here("output", "data", "fig4_summary.rds"))

# Facet order: z (left), CTmax (right); fly uses the 4 h CTmax reference.
fig4_draws   <- mutate(fig4_draws,   quantity = factor(quantity, levels = c("z", "CTmax_1hr",
fig4_summary <- mutate(fig4_summary, quantity = factor(quantity, levels = c("z", "CTmax_1hr",

# Group display order. Colour by level; the CI bar + point use a darker tone of
# the fill. Aphid species reuse the original 3-level viridis palette; snow gum
# keeps its olive, the fly its green/orange, zebrafish a blue oxygen pair.
sp_lv  <- c("M_dirhodum", "S_avenae", "R_padi")
rec_lv <- c("Dark", "Light")
oxy_lv <- c("normoxia", "hyperoxia")
vz <- viridisLite::viridis(3, begin = 0.15, end = 0.85)
fig4_pal <- c(setNames(vz, sp_lv),
              "Dark" = "#4F7942", "Light" = "#8FB07A",
              "normoxia" = "#2C7FB8", "hyperoxia" = "#7FCDBB",
              "Female" = "#009E73", "Male" = "#E69F00")
darken <- function(col, f = 0.34) { v <- grDevices::col2rgb(col) / 255 * (1 - f); grDevices::rg
fig4_pal_dark <- sapply(fig4_pal, darken)
qlab <- as_labeller(c(z = "italic(z)~(degree*C)", CTmax_1hr = "CT[max[1*hr]]~(degree*C)",
                      CTmax_4hr = "CT[max[4*hr]]~(degree*C)"), default = label_parsed)
P <- function(x) here("pics", x)

# Patrice's v2 illustrations are vector SVGs. Rasterise each to a high-resolution
# temporary PNG so the 600-dpi figure stays crisp and the existing
# draw_image()/<img> paths (which expect a raster file) work unchanged. Cached so
```

```r
# each SVG is rasterised once per session.
svg_png <- local({
  cache <- new.env(parent = emptyenv())
  function(svg, width = 1600) {
    key <- paste0(svg, "@", width)
    if (is.null(cache[[key]])) {
      o <- tempfile(fileext = ".png")
      magick::image_write(magick::image_read_svg(P(svg), width = width), o)
      cache[[key]] <- o
    }
    cache[[key]]
  }
})

# Trim white margins so the aphid species illustrations are size-comparable
# (mirrors the original danio-stage y-label treatment for the 3-group panel).
aphid_src  <- c(M_dirhodum = "aphid_Mdirhodum_V2.png", S_avenae = "aphid_Savenae_V2.png", R_pa
aphid_trim <- vapply(aphid_src, function(f) {
  o <- tempfile(fileext = ".png"); image_write(image_trim(image_read(P(f)), fuzz = 8), o); o
}, character(1))
imgH <- function(p, h) sprintf("<img src='%s' height='%d'/>", p, h)
imgW <- function(p, w) sprintf("<img src='%s' width='%d'/>", p, w)
sym  <- function(ch, pt) sprintf("<span style='font-size:%dpt'>%s</span>", pt, ch)

# pMCMC = twice the smaller posterior tail probability of the paired difference.
pmcmc <- function(d) { n <- length(d); max(2 * min(sum(d > 0) / n, sum(d < 0) / n), 1 / n) }
fmtp  <- function(p) ifelse(p < 0.001, "p<0.001", paste0("p=", sub("^0", "", sprintf("%.3f", p
make_brackets <- function(case, glevels, base = 0.10, step = 0.20, cbase = base, cstep = step)
  ylev <- rev(glevels); ypos <- setNames(seq_along(ylev), ylev); prs <- combn(glevels, 2, simp
  sp_all <- sapply(prs, function(pr) abs(ypos[[pr[1]]] - ypos[[pr[2]]])); maxsp <- max(sp_all)
  # x-offset keyed to the bracket's VERTICAL SPAN: adjacent-pair (inner) brackets
  # share one thin x-column and the widest (outer) bracket steps right by `step`.
  # INNER p-values print horizontally to the LEFT of their bar (tucked toward the
  # data, in the between-row gaps); the OUTER p-value prints to the RIGHT. `base`
  # offsets the inner column past the CI labels.
  dc <- filter(fig4_draws, case_study == case); rows <- list()
  for (q in as.character(unique(dc$quantity))) {
    b <- if (grepl("^CTmax", q)) cbase else base
    s <- if (grepl("^CTmax", q)) cstep else step
    dq <- filter(dc, quantity == q); rng <- range(dq$value); w <- diff(rng); tick <- 0.04 * w
    for (i in seq_along(prs)) {
      pr <- prs[[i]]; ya <- ypos[[pr[1]]]; yb <- ypos[[pr[2]]]; span <- abs(ya - yb)
      xbar <- rng[2] + w * (b + s * (span - 1))
      m <- inner_join(filter(dq, group == pr[1])[, c(".draw", "value")],
                      filter(dq, group == pr[2])[, c(".draw", "value")],
                      by = ".draw", suffix = c("_a", "_b"))
      p    <- pmcmc(m$value_a - m$value_b)
```

```
      side <- if (span == maxsp) "R" else "L"
      rows[[length(rows) + 1]] <- tibble(quantity = factor(q, levels = levels(fig4_draws$quanti
        x = xbar, xt = xbar - tick, side = side,
        xtext = if (side == "R") xbar + 0.04 * w else xbar - tick - 0.04 * w,
        y = min(ya, yb), yend = max(ya, yb), label = fmtp(p))
    }
  }
  bind_rows(rows)
}
add_brackets <- function(p, case, glevels, base = 0.10, step = 0.20, cbase = base, cstep = step
  br <- make_brackets(case, glevels, base, step, cbase, cstep)
  p + geom_segment(data = br, aes(x = x, xend = x, y = y, yend = yend), inherit.aes = FALSE, li
      geom_segment(data = br, aes(x = xt, xend = x, y = y, yend = y), inherit.aes = FALSE, line
      geom_segment(data = br, aes(x = xt, xend = x, y = yend, yend = yend), inherit.aes = FALSE
      geom_text(data = subset(br, side == "L"), aes(x = xtext, y = (y + yend) / 2, label = labe
      geom_text(data = subset(br, side == "R"), aes(x = xtext, y = (y + yend) / 2, label = labe
}
fig4_panel <- function(cs, glevels, ylabels, title, sci, tag,
                       texp = 1.15, xexp_r = 0.05, xexp_l = 0.18, ridge_scale = 0.65,
                       ridge_scale_ctmax = ridge_scale) {
  d <- filter(fig4_draws,   case_study == cs) |> mutate(group = factor(group, levels = rev(glev
  s <- filter(fig4_summary, case_study == cs) |> mutate(group = factor(group, levels = rev(glev
  ggplot(d, aes(value, group, fill = group)) +
    geom_density_ridges(
      data = dplyr::filter(d, !as.character(quantity) %in% c("CTmax_1hr", "CTmax_4hr")),
      aes(height = after_stat(ndensity)), stat = "density", adjust = 2.5,
      colour = "white", linewidth = .2, alpha = .9, scale = ridge_scale,
      rel_min_height = .01
    ) +
    geom_density_ridges(
      data = dplyr::filter(d, as.character(quantity) %in% c("CTmax_1hr", "CTmax_4hr")),
      aes(height = after_stat(ndensity)), stat = "density", adjust = 2.5,
      colour = "white", linewidth = .2, alpha = .9, scale = ridge_scale_ctmax,
      rel_min_height = .01
    ) +
    geom_linerange(data = s, aes(y = group, xmin = lower, xmax = upper, colour = ci_col), inher
    geom_point(data = s, aes(median, group, fill = group), inherit.aes = FALSE, shape = 21, col
    geom_label(data = s, aes(median, group, label = sprintf("%.1f [%.1f, %.1f]", median, lower
    facet_wrap(~ quantity, scales = "free_x", nrow = 1, strip.position = "bottom", labeller = l
    scale_fill_manual(values = fig4_pal, guide = "none") + scale_colour_identity() +
    scale_x_continuous(expand = expansion(mult = c(xexp_l, xexp_r))) +
    scale_y_discrete(labels = ylabels, expand = expansion(mult = c(.06, texp))) +
    labs(x = NULL, y = NULL, title = title, subtitle = sci, tag = tag) +
    theme_classic(base_size = 13) +
    theme(strip.background = element_blank(), strip.placement = "outside", strip.text = element
          axis.text.y = element_markdown(), plot.tag = element_text(face = "bold", size = 16),
          plot.title = element_text(face = "bold", size = 15), plot.subtitle = element_text(fac
```

```
          panel.border = element_rect(colour = "grey75", fill = NA, linewidth = .4), plot.margi
}
ins <- function(p, f, l, b, r, t) p + inset_element(ggdraw() + draw_image(f), left = l, bottom

# All panel icons are placed in the central gap BETWEEN the z and CTmax facets
# (centred horizontally on the facet divider, upper-middle vertically), where
# each panel has whitespace. Top expansion (texp) is reduced accordingly, since
# the icons no longer sit above the top ridge.

# a) Cereal aphids (3 species) - species illustration + italic name as each
#    y-label, PLUS a general aphid icon raised into the top whitespace (clear of
#    the posterior ridges). Extra right expansion makes room for the three
#    right-stacked contrast brackets.
fig4_A <- add_brackets(fig4_panel("Cereal aphids", sp_lv,
            setNames(c(paste0(imgH(aphid_trim["M_dirhodum"], 40), "<br>*M. dirhodum*"),
                       paste0(imgH(aphid_trim["S_avenae"],   38), "<br>*S. avenae*"),
                       paste0(imgH(aphid_trim["R_padi"],     38), "<br>*R. padi*")), sp_lv),
            "Cereal aphids", "three species", "a)", texp = 0.50, xexp_r = 0.38, xexp_l = 0.30)
            "Cereal aphids", sp_lv, base = 0.30, step = 0.12, cbase = 0.55, cstep = 0.14)
fig4_A <- ins(fig4_A, P("aphid_Mdirhodum_V2.png"), .41, .55, .66, .95)

# b) Snow gum, dark vs light recovery. Icon enlarged ~30%.
fig4_B <- add_brackets(fig4_panel("Snow gum leaf PSII", rec_lv,
            c(Dark = "Dark", Light = "Light"), "Snow gum", "Eucalyptus pauciflora", "b)",
            texp = 0.5, xexp_r = 0.34), "Snow gum leaf PSII", rec_lv)
fig4_B <- ins(fig4_B, svg_png("snowgum_v4.svg"), .40, .52, .72, .93)

# c) Zebrafish x oxygen - LARVAL illustration raised and shrunk a touch; ridges
#    scaled DOWN (0.7) so the tight hyperoxia peaks are not so tall.
fig4_C <- add_brackets(fig4_panel("Zebrafish (oxygen)", oxy_lv,
            c(normoxia = "Normoxia", hyperoxia = "Hyperoxia"), "Zebrafish", "Danio rerio", "c)"
            texp = 0.5, xexp_r = 0.34, ridge_scale = 0.7), "Zebrafish (oxygen)", oxy_lv)
fig4_C <- ins(fig4_C, svg_png("danio_larvae_v2.svg"), .41, .54, .68, .90)

# d) Vinegar fly by sex.
fig4_D <- add_brackets(fig4_panel("Vinegar fly (mortality)", c("Female", "Male"), c(Female = sy
fig4_D <- ins(fig4_D, svg_png("dros_male_v2.svg"), .44, .48, .66, .90)

fig4_final <- (fig4_A | fig4_B) / (fig4_C | fig4_D)

# Also write a standalone high-res PNG so the assembled figure can be imported
# directly (e.g. into the manuscript or slides) without re-running the chunk.
# Wider canvas (13.5 in) so the estimate labels are no longer clipped/overlapping.
dir.create(here("output", "figs"), recursive = TRUE, showWarnings = FALSE)
ggsave(here("output", "figs", "fig4_case_study_densities.png"), fig4_final,
       width = 13.5, height = 9.6, dpi = 600, bg = "white")
```

```
# Display the saved PNG rather than the live plot object, so the figure's tuned
# canvas (11.5 x 9.6 in) and inset/bracket positions are not re-flowed by the
# chunk's fig dimensions (mirrors the ms.qmd include_graphics approach).
knitr::include_graphics(here("output", "figs", "fig4_case_study_densities.png"))
```

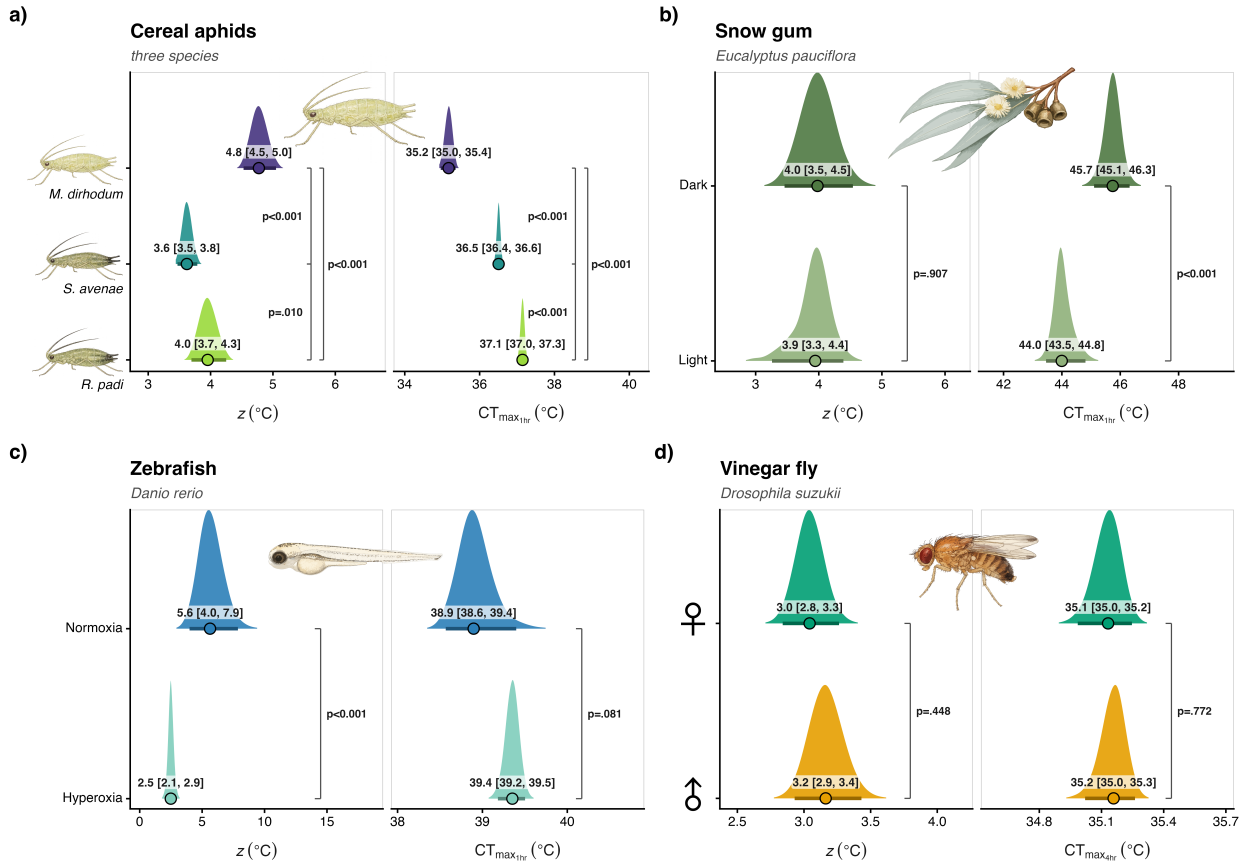


Figure S24: Manuscript Figure 5. Posterior distributions of thermal sensitivity z (left of each panel) and the critical thermal limit CT_{max} (right) from the joint Bayesian 4PL fits for four case studies: a) three cereal-aphid species (*Metopolophium dirhodum*, *Sitobion avenae*, *Rhopalosiphum padi*), b) snow gum (*Eucalyptus pauciflora*) leaf photosystem II function under dark vs 90-min-light post-heat recovery, c) zebrafish (*Danio rerio*) larvae under normoxia vs hyperoxia, and d) the vinegar fly (*Drosophila suzukii*) by sex. Filled ridges are the posterior densities, coloured by group; the black-outlined point and bar give the posterior median and 95% credible interval, also printed beside each ridge. The CT_{max} reference exposure follows each study's own analysis: 1 h for aphids, snow gum, and zebrafish (relative midpoint threshold) and 4 h for the vinegar fly (absolute LT50, following Ørsted *et al.* (2024)). Within each panel the groups are fit jointly, so pairwise posterior contrasts are annotated as brackets labelled with the pMCMC (twice the smaller posterior tail probability of the difference; smaller values indicate stronger evidence the groups differ). Panels carry independent x-axes.

12 Manuscript Figure 6: Heat injury and survival

Manuscript Figure 6 illustrates how a fitted joint 4PL translates a *natural* temperature trajectory into cumulative heat-injury (HI) accumulation and predicted survival, with uncertainty, for two case studies side by side: the vinegar fly (*Drosophila suzukii*, by sex) and the three cereal aphids (*Metopolophium dirhodum*, *Sitobion avenae*, *Rhopalosiphum padi*). It reuses the fly and aphid fits built above (`wf_dros_mort_F`/`wf_dros_mort_M` and `wf_aphid`) and the package's `predict_heat_injury()`, so no model is refit here.

For the fly we use the shaded hourly **microclimate** trace from the Ørsted et al. (2024) NicheMapR workflow (Rennes, France; June–August 2018; Zenodo record 10821572) and predict HI for each sex with and without damage repair. For the aphids we use the **unmodified** hourly air-temperature trace for Wuhan — the hottest of Li *et al.* (2023)'s three field sites (May–June 2016, Open-Meteo ERA5; shipped with the package) — and project all three species at once with a single `predict_heat_injury(by = "species")` call. The aphid trace crosses each species' T_{crit} without any warming offset, so it is used as observed. We show the May–June window because, over the full May–August record, the most heat-vulnerable species (*M. dirhodum*, the lowest T_{crit}) accrues heat injury into the tens of thousands of percent of an LT_{50} dose, which swamps the axis; May–June keeps the scale readable and is the window in which the repair model still keeps the two more tolerant species (*R. padi*, *S. avenae*) alive.

Repair is modelled with an **illustrative Sharpe–Schoolfield kernel that is not empirically fitted** — it shows the *qualitative* effect of repair, not a calibrated rate. We set the Arrhenius (uninhibited) rate scale to $r_{ref} = 0.005$ LT50-dose h^{-1}; the realised repair rate is a little below this because the inactivation arms of the kernel suppress it. Bands are ±1 SE across posterior draws (not 95% credible intervals) to keep the exponential HI envelope readable. T_{crit} is the relative-threshold critical temperature from each fit (per species for the aphids).

```
# Prepare temperature traces and posterior summaries for the manuscript heat-injury figure.
## Fly: shaded hourly microclimate temperature from the Oersted et al. (2024)
## Zenodo/NicheMapR workflow (Rennes, France; Jun-Aug 2018). The cached CSV is
## built by data-raw/build_orsted_microclimate_trace.R from the original
## microclimate_injury_accumulation.R script in Zenodo record 10821572.
orsted_micro <- utils::read.csv(here::here(
  "inst", "extdata", "orsted_2024",
  "orsted2024_nichemapr_rennes_2018_hourly.csv.gz"
))
fly_trace <- orsted_micro |>
  # The column is ISO-8601 with a "T" separator and trailing "Z". as.POSIXct()'s
  # default formats do NOT match that, so it silently falls back to "%Y-%m-%d" and
  # collapses every hour of a day onto midnight (which made the Fig. 6 trace plot as
  # one vertical min-max bar per day). Give the format explicitly.
  dplyr::mutate(datetime_utc = as.POSIXct(datetime_utc,
                                          format = "%Y-%m-%dT%H:%M:%SZ", tz = "UTC")) |>
  dplyr::filter(format(datetime_utc, "%Y") == "2018",
                format(datetime_utc, "%m") %in% c("06", "07", "08"),
                is.finite(micro_temp_c)) |>
  dplyr::arrange(datetime_utc) |>
```

```
  dplyr::transmute(time = dplyr::row_number() - 1, temp = micro_temp_c,
                   datetime = datetime_utc)

## Relative-threshold T_crit per group via tls() (matches the relative HI integration).
Tc_F <- get_tls_est(tls(wf_dros_mort_F, target_surv = "relative", t_ref = 60,
                        lethal = TRUE, ndraws = 150), "summary", "Tcrit")$median
Tc_M <- get_tls_est(tls(wf_dros_mort_M, target_surv = "relative", t_ref = 60,
                        lethal = TRUE, ndraws = 150), "summary", "Tcrit")$median

## Illustrative Sharpe-Schoolfield repair kernel (doses h^-1 at TREF; NOT fitted).
## TH sets the high-temperature decline: TH = 28 puts the repair optimum near
## 26 C, comfortably below the D. suzukii T_crit (~29 C), rather than right at it.
## TREF (25 C) is only the magnitude anchor for r_ref, not the optimum.
fly_repair <- list(TA = 14065, TAL = 50000, TAH = 100000,
                   TL = 15 + 273.15, TH = 28 + 273.15,
                   TREF = 25 + 273.15, r_ref = 0.005)

ND <- 150
set.seed(123)  # reproducible posterior-draw subsample for both taxa
agg_draws <- function(hi, ...) {
  hi$draws |> dplyr::group_by(time) |>
    dplyr::summarise(hi_med = median(hi), hi_se = sd(hi),
                     surv_med = median(survival), surv_se = sd(survival),
                     .groups = "drop") |>
    dplyr::mutate(...)
}

fly_rows <- list()
for (sx in c("Female", "Male")) {
  wf <- if (sx == "Female") wf_dros_mort_F else wf_dros_mort_M
  Tc <- if (sx == "Female") Tc_F else Tc_M
  for (rep in c(FALSE, TRUE)) {
    hi <- predict_heat_injury(fly_trace[, c("time", "temp")], wf,
            target_surv = "relative", T_c = Tc, trace_unit = "hours",
            ndraws = ND, repair = rep,
            repair_pars = if (rep) fly_repair else NULL, save_draws = TRUE)
    fly_rows[[length(fly_rows) + 1]] <-
      agg_draws(hi, sex = sx, repair = ifelse(rep, "With repair", "No repair"))
  }
}
fdat <- dplyr::bind_rows(fly_rows) |> dplyr::left_join(fly_trace, by = "time")
fTc  <- mean(c(Tc_F, Tc_M))

## Aphids (d-f): UNMODIFIED hourly Wuhan air-temperature trace (no warming
## offset), all three species projected at once with by = "species". Per-species
## T_crit comes from the aphid fit; repair uses the same illustrative kernel.
## Trimmed to May-June: over the full May-Aug record the heat injury for the most
```

```
## vulnerable species (M. dirhodum, lowest T_crit) runs to ~28000% of an LT50 dose,
## swamping the axis; May-June keeps it readable and spans the window in which the
## repair model still protects the two more tolerant species (R. padi, S. avenae).
aphid_trace <- readr::read_csv(
  here::here("inst", "extdata", "data_temp_trace_aphid_summer2016.csv"),
  show_col_types = FALSE) |>
  dplyr::filter(city == "Wuhan",
                as.integer(format(datetime, "%m")) <= 6) |>   # May-June only
  # readr already parses this ISO-8601 column to POSIXct. The old as.character()
  # round-trip rendered the first (midnight) element as a bare date, which made
  # as.POSIXct() choose a date-only format for the WHOLE vector -> hours lost.
  dplyr::transmute(time = time_h, temp = temp_c, datetime = datetime)
aphid_repair <- list(TA = 14065, TAL = 50000, TAH = 100000,
                     TL = 15 + 273.15, TH = 35 + 273.15,
                     TREF = 25 + 273.15, r_ref = 0.005)
aphid_Tc <- aphid_tls$summary |> dplyr::filter(quantity == "Tcrit") |>
  dplyr::transmute(species = as.character(species), Tc = median)
aphid_rows <- list()
for (rep in c(FALSE, TRUE)) {
  hi <- predict_heat_injury(aphid_trace[, c("time", "temp")], wf_aphid,
          target_surv = "relative", trace_unit = "hours", by = "species",
          ndraws = ND, repair = rep,
          repair_pars = if (rep) aphid_repair else NULL, save_draws = TRUE)
  aphid_rows[[length(aphid_rows) + 1]] <- hi$draws |>
    dplyr::group_by(species, time) |>
    dplyr::summarise(hi_med = median(hi), hi_se = sd(hi),
                     surv_med = median(survival), surv_se = sd(survival),
                     .groups = "drop") |>
    dplyr::mutate(repair = ifelse(rep, "With repair", "No repair"))
}
adat <- dplyr::bind_rows(aphid_rows) |> dplyr::mutate(species = as.character(species)) |>
  dplyr::left_join(aphid_trace, by = "time")

dir.create(here::here("output", "data"), recursive = TRUE, showWarnings = FALSE)
# `provenance` marks this file as written by the authoritative supplement
# pipeline; ms.qmd refuses the cached values (falls back) without it.
saveRDS(list(fly = fdat, fly_trace = fly_trace, fly_Tc = fTc,
             aphid = adat, aphid_trace = aphid_trace, aphid_Tc = aphid_Tc,
             repair_pars = list(fly = fly_repair, aphid = aphid_repair),
             provenance = "supplement.qmd", rendered = Sys.time()),
        here::here("output", "data", "fig5_hi_data.rds"))
```

```
# Build the manuscript heat-injury figure from the in-session fly + aphid data
# (fdat / adat / traces / Tc) prepared in the chunk above. svg_png() and P() are
# the icon helpers defined in the Figure 5 chunk. Fly: colour = sex; aphids:
# colour = species; line type = repair. Colours match the Figure 5 ridge panels.
fdat$repair <- factor(fdat$repair, levels = c("No repair", "With repair"))
```

```r
adat$repair <- factor(adat$repair, levels = c("No repair", "With repair"))
sp_pal  <- c("M_dirhodum" = "#E69F00", "S_avenae" = "#D55E00", "R_padi" = "#CC79A7")
sp_lab  <- c(M_dirhodum = "M. dirhodum", S_avenae = "S. avenae", R_padi = "R. padi")
sex_pal <- c(Female = "#009E73", Male = "#7B3294")
tcrit_col <- "#7B3294"

f6_thm <- theme_classic(base_size = 13) +
  theme(panel.grid.major.y = element_line(colour = "grey93"), plot.margin = margin(2, 7, 2, 3)
        legend.position = "none", axis.title = element_text(size = 13), axis.text = element_tex
f6_xsc <- function() scale_x_datetime(date_breaks = "1 month", date_labels = "%b")
f6_tg  <- function(p, lab) p + labs(tag = lab) +
  theme(plot.tag = element_text(face = "bold", size = 12), plot.tag.position = c(0.015, 0.98))

# fly temperature panel (single Tcrit line)
f6_temp1 <- function(tr, Tc, title, ylab) ggplot(tr, aes(datetime, temp)) +
  geom_line(colour = "grey45", linewidth = 0.3) +
  geom_hline(yintercept = Tc, linetype = "dotted", colour = tcrit_col, linewidth = 0.9) +
  annotate("text", x = min(tr$datetime), y = Tc, label = "T[crit]", hjust = -0.1, vjust = -0.4
           size = 3.7, colour = tcrit_col, fontface = "bold", parse = TRUE) +
  labs(y = ylab, x = NULL, title = title) + f6_xsc() + f6_thm +
  theme(axis.text.x = element_blank(), plot.title = element_markdown(face = "bold", size = 13,
# aphid temperature panel (per-species Tcrit lines)
f6_tempA <- function(tr, tc_df, title, ylab) ggplot(tr, aes(datetime, temp)) +
  geom_line(colour = "grey45", linewidth = 0.3) +
  geom_hline(data = tc_df, aes(yintercept = Tc, colour = species), linetype = "dotted", linewi
  annotate("text", x = min(tr$datetime), y = max(tc_df$Tc), label = "T[crit]", hjust = -0.1, v
           size = 3.7, colour = "grey15", fontface = "bold", parse = TRUE) +
  scale_colour_manual(values = sp_pal) +
  labs(y = ylab, x = NULL, title = title) + f6_xsc() + f6_thm +
  theme(axis.text.x = element_blank(), plot.title = element_markdown(face = "bold", size = 13,
# fly HI / survival (colour = sex, line type = repair)
f6_hiF <- function(dd) ggplot(dd, aes(datetime, hi_med, colour = sex, fill = sex)) +
  geom_ribbon(aes(ymin = pmax(0, hi_med - hi_se), ymax = hi_med + hi_se, group = interaction(se
  geom_hline(yintercept = 100, linetype = "dotted", colour = "grey55") + geom_line(aes(linetype
  scale_colour_manual(values = sex_pal) + scale_fill_manual(values = sex_pal, guide = "none") +
  scale_linetype_manual(values = c("No repair" = "solid", "With repair" = "22")) +
  labs(y = "Heat injury (%)", x = NULL) + f6_xsc() + f6_thm + theme(axis.text.x = element_blank
f6_survF <- function(dd) ggplot(dd, aes(datetime, surv_med, colour = sex, fill = sex)) +
  geom_ribbon(aes(ymin = pmax(0, surv_med - surv_se), ymax = pmin(1, surv_med + surv_se), group
  geom_line(aes(linetype = repair), linewidth = 0.55) +
  scale_colour_manual(values = sex_pal) + scale_fill_manual(values = sex_pal, guide = "none") +
  scale_linetype_manual(values = c("No repair" = "solid", "With repair" = "22")) +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent) + labs(y = "Survival", x = NUL
# aphid HI / survival (colour = species, line type = repair)
f6_hiA <- function(dd) ggplot(dd, aes(datetime, hi_med, colour = species, fill = species)) +
  geom_ribbon(aes(ymin = pmax(0, hi_med - hi_se), ymax = hi_med + hi_se, group = interaction(sp
  geom_hline(yintercept = 100, linetype = "dotted", colour = "grey55") + geom_line(aes(linetype
```

```
  scale_colour_manual(values = sp_pal, labels = sp_lab) + scale_fill_manual(values = sp_pal, gu
  scale_linetype_manual(values = c("No repair" = "solid", "With repair" = "22")) +
  labs(y = "Heat injury (%)", x = NULL) + f6_xsc() + f6_thm + theme(axis.text.x = element_blank
f6_survA <- function(dd) ggplot(dd, aes(datetime, surv_med, colour = species, fill = species))
  geom_ribbon(aes(ymin = pmax(0, surv_med - surv_se), ymax = pmin(1, surv_med + surv_se), group
  geom_line(aes(linetype = repair), linewidth = 0.55) +
  scale_colour_manual(values = sp_pal, labels = sp_lab) + scale_fill_manual(values = sp_pal, gu
  scale_linetype_manual(values = c("No repair" = "solid", "With repair" = "22")) +
  scale_y_continuous(limits = c(0, 1), labels = scales::percent) + labs(y = "Survival", x = NUL
f6_ins     <- function(p, f, l, b, r, t) p + inset_element(ggdraw() + draw_image(svg_png(f)), l
f6_ins_png <- function(p, png, l, b, r, t) p + inset_element(ggdraw() + draw_image(P(png)),

fly_hi   <- f6_ins(f6_tg(f6_hiF(fdat), "b)"), "dros_male_v2.svg", .02, .55, .34, .98)
aphid_hi <- f6_ins_png(f6_tg(f6_hiA(adat), "e)"), "aphid_Mdirhodum_V2.png", .02, .46, .44, .99)
col_f <- f6_tg(f6_temp1(fly_trace,  fTc,      "Vinegar fly (*D. suzukii*)", "Temperature (°C)")
col_a <- f6_tg(f6_tempA(aphid_trace, aphid_Tc, "Cereal aphids (Wuhan)",      "Temperature (°C)"

mk_leg <- function(p, which) {
  g <- p + theme(legend.position = "bottom", legend.title = element_blank())
  g <- if (which == "colour") g + guides(fill = "none", linetype = "none", colour = guide_legen
       else g + guides(fill = "none", colour = "none", linetype = guide_legend(order = 1))
  cowplot::get_legend(g)
}
leg_fly_col <- mk_leg(f6_hiF(fdat), "colour")   # Female / Male
leg_aph_col <- mk_leg(f6_hiA(adat), "colour")   # three species
leg_repair  <- mk_leg(f6_hiA(adat), "linetype") # No repair / With repair -- shared
f6_body <- (col_f | col_a) + plot_layout(heights = c(0.5, 1, 1))
fig5_final <- cowplot::plot_grid(
  f6_body,
  cowplot::plot_grid(leg_fly_col, leg_aph_col, leg_repair, ncol = 3, rel_widths = c(1, 1.3, 1)),
  ncol = 1, rel_heights = c(1, 0.09))

ggsave(here::here("output", "figs", "fig5_heat_injury.png"), fig5_final,
       width = 9.8, height = 8.0, dpi = 300, bg = "white")

# Display the saved PNG (not the live object) so the tuned 9.8 x 8.0 in canvas
# and inset positions are not re-flowed by the chunk's fig dimensions.
knitr::include_graphics(here::here("output", "figs", "fig5_heat_injury.png"))
```

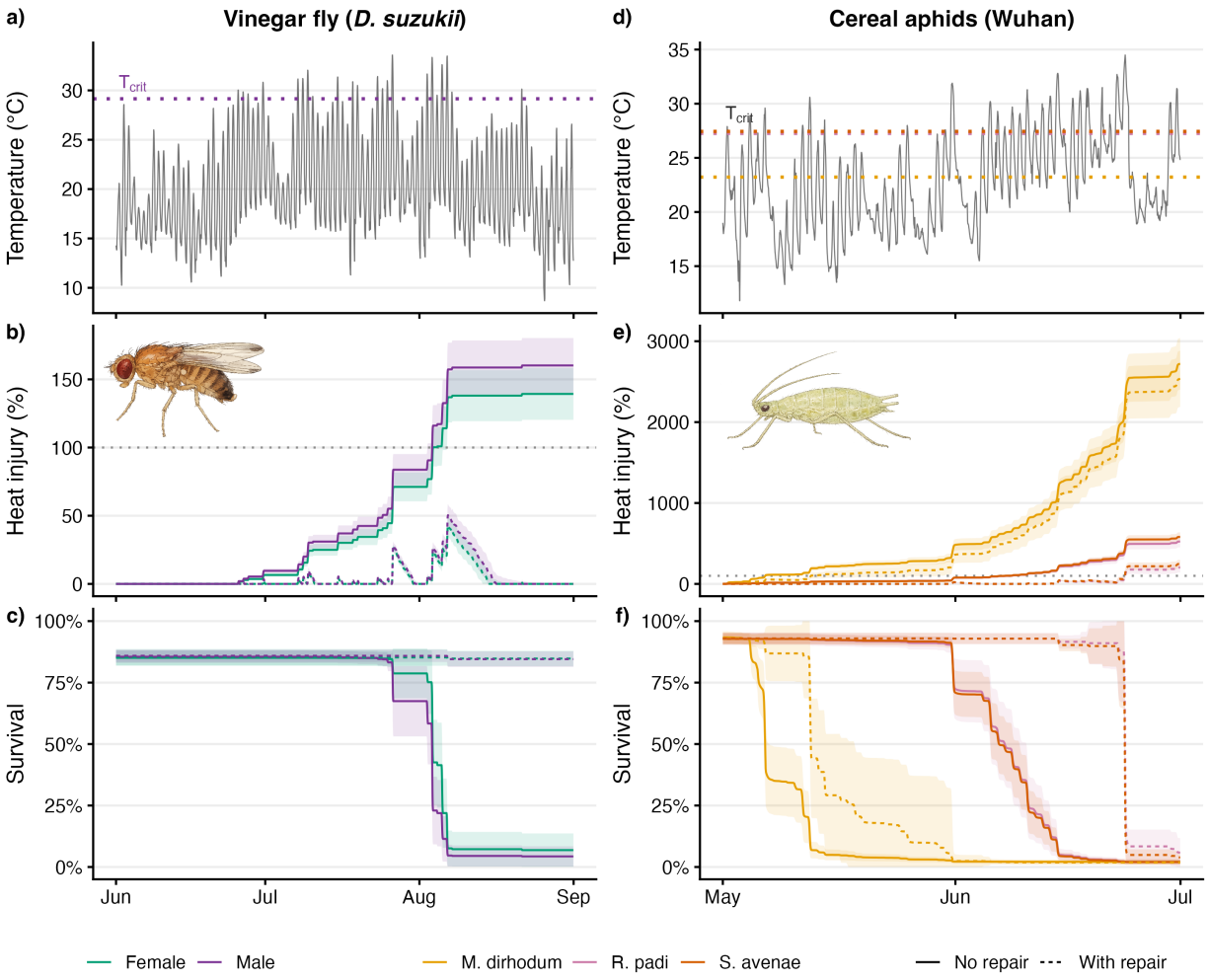


Figure S25: Manuscript Figure 6. Heat-injury accumulation and predicted survival under field temperature trajectories for the vinegar fly (*Drosophila suzukii*, by sex; panels a-c) and three cereal aphids (*Metopolophium dirhodum*, *Sitobion avenae*, *Rhopalosiphum padi*; panels d-f). (a, d) the temperature trace — shaded NicheMapR (Kearney & Porter 2017) microclimate temperature for the fly (Rennes, France, Jun–Aug 2018; Ørsted *et al.* (2024)) and the hourly air-temperature trace for the aphids (Wuhan, May–June 2016; Li *et al.* (2023), Open-Meteo ERA5 reanalysis) — with the critical temperature threshold T_{crit} (dotted line). (b, e) cumulative heat injury (100% = one LT_{50} dose, dotted grey) and (c, f) predicted survival, each with and without damage repair (line type). Fly panels are coloured by sex, aphid panels by species. Bands are ±1 SE across posterior draws. Predicted survival begins at each group's fitted upper asymptote rather than at 100% (the vinegar-fly 4PL estimates a maximum survival of 85%, $up < 1$). Repair uses an illustrative Sharpe–Schoolfield kernel with an Arrhenius (uninhibited) rate scale $r_{ref} = 0.005$ LT50-dose h^{-1}; it is not empirically fitted.

13 Fitting models with `freqTLS`

The companion package `freqTLS` fits the same models by maximum likelihood instead of MCMC, and mirrors the `bayesTLS` interface on purpose: `standardize_data()`, `fit_4pl()` and `tls()` take the same arguments and return the same tidy-long summaries — so the *same* call fits the model, only the package changes. We refit the three-species aphid model from Case Study 2 (Section 6) and compare. Because `freqTLS` deliberately shares `bayesTLS`'s function names, we call it with the `freqTLS::` prefix here so it never masks the `bayesTLS` functions used elsewhere in this supplement.

```
# freqTLS is installed from GitHub: remotes::install_github("itchyshin/freqTLS").
# We call it with the freqTLS:: prefix (rather than library(freqTLS)) so it does
# not mask the identically named bayesTLS functions used elsewhere.

# The SAME fit_4pl() call as Case Study 2 - fit by maximum likelihood, not MCMC.
fit_aphid_freq <- freqTLS::fit_4pl(aphid_std,
                                   ctmax = ~ 0 + species, z = ~ 0 + species,
                                   k = ~ temp_c, up = ~ 1, low = ~ 1,
                                   t_ref = 60, family = freqTLS::beta_binomial_tls())

# Per-species z and CTmax, extracted exactly as with bayesTLS.
freq_sum <- freqTLS::tls(fit_aphid_freq, by = "species")$summary
freq_sum
```

```
# A tibble: 6 x 5
  species    quantity median lower upper
  <chr>      <chr>     <dbl> <dbl> <dbl>
1 M_dirhodum CTmax     35.2  35.0  35.4
2 S_avenae   CTmax     36.5  36.4  36.6
3 R_padi     CTmax     37.2  37.1  37.3
4 M_dirhodum z          4.75  4.52  5.02
5 S_avenae   z          3.61  3.46  3.77
6 R_padi     z          3.97  3.70  4.25
```

We already have the Bayesian estimates for these species from Case Study 2 (`aphid_est`). Stacking the two side by side shows they coincide (Table S35):

```
compare <- dplyr::bind_rows(bayesTLS = aphid_est, freqTLS = freq_sum, .id = "Package") |>
  dplyr::filter(quantity %in% c("z", "CTmax")) |>
  dplyr::transmute(
    Package,
    Species  = dplyr::recode(as.character(species), M_dirhodum = "M. dirhodum",
                             R_padi = "R. padi", S_avenae = "S. avenae"),
    Quantity = dplyr::recode(quantity, z = "z (°C)", CTmax = "CTmax, 1h (°C)"),
    Estimate = format_interval(median, lower, upper, digits = 2)) |>
  dplyr::arrange(Species, Quantity, Package)
```

Table S35: Per-species thermal sensitivity (z) and 1-hour heat tolerance ($CT_{max,1h}$, °C) for the three aphids from the identical joint 4PL, estimated by `bayesTLS` (posterior median with 95% credible interval) and `freqTLS` (maximum-likelihood estimate with 95% profile-likelihood interval).

Package	Species	Quantity	Estimate
bayesTLS	M. dirhodum	CTmax, 1h (°C)	35.17 [34.96, 35.36]
freqTLS	M. dirhodum	CTmax, 1h (°C)	35.22 [35.01, 35.39]
bayesTLS	M. dirhodum	z (°C)	4.77 [4.53, 5.05]
freqTLS	M. dirhodum	z (°C)	4.75 [4.52, 5.02]
bayesTLS	R. padi	CTmax, 1h (°C)	37.15 [37.04, 37.25]
freqTLS	R. padi	CTmax, 1h (°C)	37.17 [37.06, 37.28]
bayesTLS	R. padi	z (°C)	3.95 [3.69, 4.25]
freqTLS	R. padi	z (°C)	3.97 [3.7, 4.25]
bayesTLS	S. avenae	CTmax, 1h (°C)	36.51 [36.41, 36.6]
freqTLS	S. avenae	CTmax, 1h (°C)	36.53 [36.43, 36.62]
bayesTLS	S. avenae	z (°C)	3.62 [3.47, 3.79]
freqTLS	S. avenae	z (°C)	3.61 [3.46, 3.77]

```
tinytable::tt(compare)
```

The two packages agree to within 0.04 °C on every estimate, with essentially identical intervals. The choice between them is practical: `freqTLS` is faster for large datasets and quick exploration, while `bayesTLS` retains the posterior draws needed to propagate uncertainty into derived quantities — group contrasts, heat-injury trajectories and T_{crit} — as used throughout this supplement.

14 Derivation of the correction factors for threshold summaries

The manuscript states two correction factors without showing the algebra: an asymmetry correction *added* to mid(T) to recover $\log_{10} t_{50}(T)$ from a 4PL whose asymptotes are not 0 and 1 (Eq. (4)), and the corresponding correction *subtracted* from the naive $CT_{max}(t_{\text{ref}})$ (Eq. (5)). Here we derive both expressions, extend them to an arbitrary survival threshold $x/100$, and check them against direct 4PL inversion.

14.1 Why a correction is needed

The 4PL midpoint is the $\log_{10}$-time value at which survival reaches the average of the two asymptotes, $(\text{up} + \text{low})/2$. When low ≈ 0 and up ≈ 1, this point is also near 50% survival. With partial endpoints, however, the midpoint need not occur at $p = 0.5$: if up $= 0.85$ and low $= 0.05$, it occurs

at 0.45. The absolute $\log_{10} t_{50}$ therefore comes from inverting the 4PL at $p = 0.5$. The signed offset between $\mathrm{mid}(T)$ and this absolute crossing is the **asymmetry correction**.

14.2 Step-by-step derivation of the $\log_{10} t_{50}$ correction

Starting from Eq. (2) of the manuscript with the temperature-dependent midpoint from Eq. (3), the survival probability is

$$p(t, T) \;=\; \mathrm{low} \;+\; \frac{\mathrm{up} - \mathrm{low}}{1 + \exp\!\big(k\,(\log_{10} t - \mathrm{mid}(T))\big)}.$$

By definition $\log_{10} t_{50}(T)$ is the value of $\log_{10} t$ at which $p = 0.5$. Solving for it gives:

Step 1 — set $p = 0.5$:

$$0.5 \;=\; \mathrm{low} \;+\; \frac{\mathrm{up} - \mathrm{low}}{1 + \exp\!\big(k\,(\log_{10} t_{50}(T) - \mathrm{mid}(T))\big)}.$$

Step 2 — move low to the left:

$$0.5 - \mathrm{low} \;=\; \frac{\mathrm{up} - \mathrm{low}}{1 + \exp\!\big(k\,(\log_{10} t_{50}(T) - \mathrm{mid}(T))\big)}.$$

Step 3 — invert both sides:

$$\frac{1}{0.5 - \mathrm{low}} \;=\; \frac{1 + \exp\!\big(k\,(\log_{10} t_{50}(T) - \mathrm{mid}(T))\big)}{\mathrm{up} - \mathrm{low}}.$$

Step 4 — multiply both sides by $(\mathbf{up} - \mathbf{low})$:

$$\frac{\mathrm{up} - \mathrm{low}}{0.5 - \mathrm{low}} \;=\; 1 + \exp\!\big(k\,(\log_{10} t_{50}(T) - \mathrm{mid}(T))\big).$$

Step 5 — subtract 1 to isolate the exponential:

$$\frac{\mathrm{up} - \mathrm{low}}{0.5 - \mathrm{low}} \;-\; 1 \;=\; \exp\!\big(k\,(\log_{10} t_{50}(T) - \mathrm{mid}(T))\big).$$

The left-hand side simplifies:

$$\frac{\mathrm{up} - \mathrm{low}}{0.5 - \mathrm{low}} - 1 \;=\; \frac{(\mathrm{up} - \mathrm{low}) - (0.5 - \mathrm{low})}{0.5 - \mathrm{low}} \;=\; \frac{\mathrm{up} - 0.5}{0.5 - \mathrm{low}}.$$

Hence

$$\frac{\mathrm{up} - 0.5}{0.5 - \mathrm{low}} \;=\; \exp\!\big(k\,(\log_{10} t_{50}(T) - \mathrm{mid}(T))\big).$$

Step 6 — take natural logs of both sides:

$$\ln\!\left(\frac{\mathrm{up} - 0.5}{0.5 - \mathrm{low}}\right) \;=\; k\,(\log_{10} t_{50}(T) - \mathrm{mid}(T)).$$

Step 7 — divide by k and add mid(T):

$$\log_{10} t_{50}(T) \;=\; \mathrm{mid}(T) \;+\; \frac{1}{k}\ln\!\left(\frac{\mathrm{up}-0.5}{0.5-\mathrm{low}}\right). \tag{S7}$$

Thus $\log_{10} t_{50}$ is the midpoint at that temperature plus a correction that depends only on $(\mathrm{low}, \mathrm{up})$ and k, not on T. This is Eq. (4) of the manuscript.

Step 8 — substitute the temperature-dependent midpoint from Eq. (3) of the manuscript:

$$\log_{10} t_{50}(T) \;=\; [\beta_0 + \beta_1(T-\bar{T})] \;+\; \frac{1}{k}\ln\!\left(\frac{\mathrm{up}-0.5}{0.5-\mathrm{low}}\right).$$

Step 9 — distribute and group all T-independent terms into an intercept α:

$$\boxed{\log_{10} t_{50}(T) \;=\; \alpha + \beta_1 T, \qquad \alpha \;=\; \beta_0 - \beta_1\bar{T} \;+\; \frac{1}{k}\ln\!\left(\frac{\mathrm{up}-0.5}{0.5-\mathrm{low}}\right).} \tag{S8}$$

This recovers the classical TDT relation: $\log_{10} t_{50}$ is linear in T.

14.3 From $\log_{10} t_{50}$ to the CT_{max} correction

By definition, $CT_{max}(t_{\mathrm{ref}})$ is the temperature at which $t_{50} = t_{\mathrm{ref}}$, or $\log_{10} t_{50}(T) = \log_{10} t_{\mathrm{ref}}$. From Equation S8:

$$\log_{10} t_{\mathrm{ref}} \;=\; \alpha \;+\; \beta_1\, CT_{max}(t_{\mathrm{ref}}) \quad\Longrightarrow\quad CT_{max}(t_{\mathrm{ref}}) \;=\; \frac{\log_{10} t_{\mathrm{ref}} - \alpha}{\beta_1}.$$

Substituting the full α from Equation S8 and tidying:

$$CT_{max}(t_{\mathrm{ref}}) \;=\; \bar{T} \;+\; \frac{\log_{10} t_{\mathrm{ref}} - \beta_0}{\beta_1} \;-\; \frac{1}{\beta_1}\cdot\frac{1}{k}\ln\!\left(\frac{\mathrm{up}-0.5}{0.5-\mathrm{low}}\right). \tag{S9}$$

The first two terms are the *naive* CT_{max}: the temperature at which $\beta_0 + \beta_1(T-\bar{T})$ crosses $\log_{10} t_{\mathrm{ref}}$. The final term is the asymmetry correction divided by β_1, appearing as a subtraction because a vertical shift on the log-time axis becomes a horizontal temperature shift when divided by the TDT slope. This is Eq. (5) of the manuscript.

14.4 Generalisation to arbitrary survival thresholds $x/100$

Replacing $p = 0.5$ with $p = x/100$ gives

$$\log_{10} t_x(T) \;=\; \mathrm{mid}(T) \;+\; \frac{1}{k}\ln\!\left(\frac{\mathrm{up}-x/100}{x/100-\mathrm{low}}\right), \tag{S10}$$

which reduces to Equation S7 when $x = 50$. The correction is still added to mid(T); its sign is set by the ratio inside the log. For $x < 50$ (e.g. LT10), the correction is positive and LT_x occurs later than the midpoint. For $x > 50$ (e.g. LT90), it is negative and LT_x occurs earlier. The corresponding $CT_{max,x}(t_{\mathrm{ref}})$ follows by substituting this LT_x correction into Equation S9.

14.5 The slope (and hence z) is robust to the correction

Because the correction in Equation S7 does not depend on T, it shifts $\log_{10} t_{50}(T)$ vertically without changing the slope:

$$z \;=\; -\frac{1}{\beta_1}$$

holds provided the 4PL is correctly specified. This is why the joint Bayesian 4PL and the classical two-stage pipeline can recover the same z even when their CT_{max} estimates differ. CT_{max} has no analogous protection: the same asymmetry term enters Equation S9 divided by β_1, so errors in low, up, or k shift the temperature estimate directly.

14.6 When the correction vanishes

When the experimental design anchors low ≈ 0 and up ≈ 1:

$$\frac{\text{up} - 0.5}{0.5 - \text{low}} \;\approx\; \frac{1 - 0.5}{0.5 - 0} \;=\; 1 \quad\Longrightarrow\quad \ln\!\left(\frac{\text{up} - 0.5}{0.5 - \text{low}}\right) \;\approx\; \ln 1 \;=\; 0.$$

The correction then shrinks to zero and $\text{mid}(T) \approx \log_{10} t_{50}(T)$. This is the equivalence regime in Section 3.0.2 and Section 3.0.3, and the strict-equivalence baseline of the simulation in §3.1. The correction matters when the design is partial: sublethal endpoints, sparse survival-range coverage, or asymptotes that vary with T (the last case is analysed in Section 3.0.6).

14.7 Numerical sanity check

The chunk below uses asymmetric asymptotes so the correction is non-zero. It checks that Equation S7 matches direct 4PL inversion at $p = 0.5$, and that Equation S9 matches the temperature obtained by direct inversion at $t_{\text{ref}} = 60$ min.

```
# Check the one-step correction against numerical inversion.
low_a  <- 0.05
up_a   <- 0.85          # asymmetric: (up - 0.5) != (0.5 - low), so the correction is non-zero
kk     <- 4
beta_0 <- log10(60)     # midpoint at T_bar sits exactly at log10(60 min)
beta_1 <- -0.25         # so z = -1/beta_1 = 4 °C
T_bar  <- 36

correction <- (1 / kk) * log((up_a - 0.5) / (0.5 - low_a))

# Closed form (eq-supp-corr-lt50) at T = 38 °C.
T_test <- 38
mid_T  <- beta_0 + beta_1 * (T_test - T_bar)
log10_LT50_closed <- mid_T + correction

# Direct numerical inversion: find log10 t such that p(t, T) = 0.5.
p_4pl <- function(log10_t, T_val) {
  midT <- beta_0 + beta_1 * (T_val - T_bar)
```

```
  low_a + (up_a - low_a) / (1 + exp(kk * (log10_t - midT)))
}
log10_LT50_direct <- stats::uniroot(
  function(x) p_4pl(x, T_test) - 0.5,
  interval = c(-5, 10)
)$root

# Closed-form CTmax_1hr from eq-supp-corr-ctmax. Because mid(T) was written
# with centred temperature, beta_0 + beta_1 * (T - T_bar), alpha converts the
# line to the uncentred form alpha + beta_1 * T before inversion.
alpha             <- beta_0 - beta_1 * T_bar + correction
CT_max_1hr_closed <- (log10(60) - alpha) / beta_1
z_from_b1         <- -1 / beta_1

# Direct cross-check for CTmax_1hr: find T such that the 4PL crosses p = 0.5
# exactly at t = 60 min.
CT_max_1hr_direct <- stats::uniroot(
  function(T_val) p_4pl(log10(60), T_val) - 0.5,
  interval = c(20, 50)
)$root

data.frame(
  Quantity = c("Asymmetry correction (1/k) log[(up-0.5)/(0.5-low)]",
               "log10(LT50) at T = 38 °C, closed form (eq-supp-corr-lt50)",
               "log10(LT50) at T = 38 °C, direct 4PL inversion",
               "  |closed - direct|",
               "z derived from beta_1 (expected: 4)",
               "CTmax_1hr, closed form (eq-supp-corr-ctmax)",
               "CTmax_1hr, direct 4PL inversion at t = 60 min",
               "  |closed - direct|"),
  Value    = c(sprintf("%+.6f", correction),
               sprintf("%.6f", log10_LT50_closed),
               sprintf("%.6f", log10_LT50_direct),
               sprintf("%.3e", abs(log10_LT50_closed - log10_LT50_direct)),
               sprintf("%.4f °C", z_from_b1),
               sprintf("%.4f °C", CT_max_1hr_closed),
               sprintf("%.4f °C", CT_max_1hr_direct),
               sprintf("%.3e", abs(CT_max_1hr_closed - CT_max_1hr_direct)))
) |>
  tinytable::tt(escape = TRUE)
```

Here the correction is **non-zero** (-0.0628), and the closed forms match direct inversion to floating-point precision. The slope still gives $z = -1/\beta_1 = 4$ °C. In contrast, $CT_{max,1h}$ is shifted from $\bar{T} = 36$ °C by about $-\text{correction}/\beta_1$, showing why the correction matters when low and up are not equidistant from 0.5.

Table S36: Verification that the closed-form $\log_{10} t_{50}$ correction (Equation S7) agrees with direct numerical inversion of the 4PL at $p = 0.5$, and that $CT_{max,1h}$ from Equation S9 agrees with direct inversion at $t = 60$ min.

Quantity	Value
Asymmetry correction (1/k) log[(up-0.5)/(0.5-low)]	-0.062829
log10(LT50) at T = 38 °C, closed form (eq-supp-corr-lt50)	1.215323
log10(LT50) at T = 38 °C, direct 4PL inversion	1.215323
|closed - direct|	1.624e-08
z derived from beta_1 (expected: 4)	4.0000 °C
CTmax_1hr, closed form (eq-supp-corr-ctmax)	35.7487 °C
CTmax_1hr, direct 4PL inversion at t = 60 min	35.7487 °C
|closed - direct|	8.151e-08

15 References

Arnold, P.A., Harris, R.J., Aitken, S.M., Hoek, M.M., Cook, A.M., Leigh, A., *et al.* (2026). Towards a standard approach to investigating the Thermal Load Sensitivity of photosystem II via chlorophyll fluorescence. *bioRxiv : the preprint server for biology.*

Arnold, P.A., Noble, D.W.A., Nicotra, A.B., Kearney, M.R., Rezende, E.L., Andrew, S.C., *et al.* (2025). A framework for modelling thermal load sensitivity across life. *Global Change Biology*, 31, e70315.

Faber, A.H., Møller, F.D., Ørsted, M., Ehlers, B.K. & Overgaard, J. (2026). Separating good from bad – a methodological assessment of the critical temperature that separates stressful and permissive temperatures in ectotherms.

Jørgensen, L.B., Malte, H. & Overgaard, J. (2021). A unifying model to estimate thermal tolerance limits in ectotherms across static, dynamic and fluctuating exposures to thermal stress. *Scientific Reports*, 11, 12840.

Kearney, M.R. & Porter, W.P. (2017). NicheMapR – an R package for biophysical modelling: The microclimate model. *Ecography*, 40, 664–674.

Li, Y.-J., Chen, S.-Y., Jørgensen, L.B., Overgaard, J., Renault, D., Colinet, H., *et al.* (2023). Interspecific differences in thermal tolerance landscape explain aphid community abundance under climate change. *Journal of Thermal Biology*, 114, 103583.

Noble, D.W.A., Mayfield, M.M., Hoffmann, A.A., Chen, Z.-H., Lade, S.J., Bai, X., *et al.* (2026). A systems modelling approach to predict biological responses to extreme heat. *Trends in Ecology & Evolution*, 41, 451–464.

Ørsted, M., Jørgensen, L.B. & Overgaard, J. (2022). Finding the right thermal limit: A framework to reconcile ecological, physiological and methodological aspects of CTmax in ectotherms. *Journal of Experimental Biology*, 225, jeb244514.

Ørsted, M., Willot, Q., Olsen, A.K., Kongsgaard, V. & Overgaard, J. (2024). Thermal limits of survival and reproduction depend on stress duration: A case study of Drosophila suzukii. *Ecology Letters*, 27, e14421.

Rezende, E.L., Bozinovic, F., Szilágyi, A. & Santos, M. (2020). Predicting temperature mortality and selection in natural Drosophila populations. *Science*, 369, 1242–1245.

Rezende, E.L., Castañeda, L.E. & Santos, M. (2014). Tolerance landscapes in thermal ecology. *Functional Ecology*, 28, 799–809.

Saruhashi, S., Boerrigter, J.G.J., Hooymans, M.H.L., Sinclair, B.J. & Verberk, W.C.E.P. (2026). Oxygen availability and oxygen delivery but not oxidative stress shape heat tolerance in diploid and triploid zebrafish larvae. *Journal of Experimental Biology*, 229, jeb251548.

16 Session Information

```
# Record package versions and session details for reproducibility.
pander::pander(sessionInfo())
```

R version 4.5.3 (2026-03-11)

Platform: aarch64-apple-darwin20

locale: C.UTF-8||C.UTF-8||C.UTF-8||C||C.UTF-8||C.UTF-8

attached base packages: *stats*, *graphics*, *grDevices*, *utils*, *datasets*, *methods* and *base*

other attached packages: *magick(v.2.9.1)*, *cowplot(v.1.2.0)*, *ggtext(v.0.1.2)*, *ggridges(v.0.5.7)*, *bayesTLS(v.1.0.0)*, *readxl(v.1.4.5)*, *pander(v.0.6.6)*, *patchwork(v.1.3.2)*, *tinytable(v.0.16.0)*, *tidybayes(v.3.0.7)*, *posterior(v.1.7.1)*, *brms(v.2.23.0)*, *Rcpp(v.1.1.2)*, *ggplot2(v.4.0.3)*, *purrr(v.1.2.2)*, *tibble(v.3.3.1)*, *tidyr(v.1.3.2)*, *dplyr(v.1.2.1)* and *here(v.1.0.1)*

loaded via a namespace (and not attached): *RColorBrewer(v.1.1-3)*, *tensorA(v.0.36.2.1)*, *jsonlite(v.2.0.0)*, *magrittr(v.2.0.5)*, *TH.data(v.1.1-3)*, *estimability(v.1.5.1)*, *farver(v.2.1.2)*, *nloptr(v.2.2.1)*, *rmarkdown(v.2.31)*, *ragg(v.1.5.2)*, *vctrs(v.0.7.3)*, *minqa(v.1.2.8)*, *tinytex(v.0.59)*, *htmltools(v.0.5.9)*, *distributional(v.0.8.1)*, *curl(v.7.1.0)*, *cellranger(v.1.1.0)*, *StanHeaders(v.2.36.0.9000)*, *desc(v.1.4.3)*, *plyr(v.1.8.9)*, *sandwich(v.3.1-1)*, *emmeans(v.2.0.3)*, *zoo(v.1.8-14)*, *TMB(v.1.9.17)*, *commonmark(v.2.0.0)*, *lifecycle(v.1.0.5)*, *pkgconfig(v.2.0.3)*, *Matrix(v.1.7-4)*, *R6(v.2.6.1)*, *fastmap(v.1.2.0)*, *rbibutils(v.2.3)*, *digest(v.0.6.39)*, *numDeriv(v.2016.8-1.1)*, *rsvg(v.2.6.2)*, *rprojroot(v.2.1.1)*, *textshaping(v.1.0.5)*, *freqTLS(v.0.1.0)*, *labeling(v.0.4.3)*, *abind(v.1.4-8)*, *mgcv(v.1.9-4)*, *compiler(v.4.5.3)*, *remotes(v.2.5.0)*, *bit64(v.4.8.0)*, *withr(v.3.0.3)*, *S7(v.0.2.2)*, *backports(v.1.5.1)*, *inline(v.0.3.21)*, *QuickJSR(v.1.10.0)*, *pkgbuild(v.1.4.8)*, *MASS(v.7.3-65)*, *loo(v.2.10.0.9000)*, *tools(v.4.5.3)*, *otel(v.0.2.0)*, *glue(v.1.8.1)*, *callr(v.3.8.0)*, *nlme(v.3.1-168)*, *gridtext(v.0.1.5)*, *grid(v.4.5.3)*, *cmdstanr(v.0.9.0)*, *checkmate(v.2.3.4)*, *reshape2(v.1.4.5)*, *generics(v.0.1.4)*, *isoband(v.0.3.0)*, *glmmTMB(v.1.1.14)*, *gtable(v.0.3.6)*, *tzdb(v.0.5.0)*, *hms(v.1.1.4)*, *xml2(v.1.5.2)*, *utf8(v.1.2.6)*, *markdown(v.2.0)*, *pillar(v.1.11.1)*, *ggdist(v.3.3.3)*, *stringr(v.1.6.0)*, *vroom(v.1.7.1)*, *splines(v.4.5.3)*, *lattice(v.0.22-9)*, *survival(v.3.8-6)*, *bit(v.4.6.0)*, *tidyselect(v.1.2.1)*, *knitr(v.1.51)*, *reformulas(v.0.4.4)*, *arrayhelpers(v.1.1-0)*, *gridExtra(v.2.3.1)*, *V8(v.6.0.4)*, *litedown(v.0.7)*, *stats4(v.4.5.3)*, *xfun(v.0.57)*, *bridgesampling(v.1.2-1)*, *matrixStats(v.1.5.0)*, *rstan(v.2.36.0.9000)*, *stringi(v.1.8.7)*, *yaml(v.2.3.12)*, *pacman(v.0.5.1)*, *boot(v.1.3-32)*, *evaluate(v.1.0.5)*, *codetools(v.0.2-20)*, *cli(v.3.6.6)*, *RcppParallel(v.5.1.11-2)*, *systemfonts(v.1.3.2)*, *xtable(v.1.8-8)*, *Rdpack(v.2.6.4)*, *processx(v.3.9.0)*, *png(v.0.1-8)*, *coda(v.0.19-4.1)*, *svUnit(v.1.0.8)*, *parallel(v.4.5.3)*, *rstantools(v.2.6.0.9000)*, *readr(v.2.2.0)*, *bayesplot(v.1.15.0.9000)*, *Brobdingnag(v.1.2-9)*, *lme4(v.1.1-37)*, *viridisLite(v.0.4.3)*, *mvtnorm(v.1.4-1)*, *scales(v.1.4.0)*, *crayon(v.1.5.3)*, *rlang(v.1.3.0)* and *multcomp(v.1.4-28)*